

CodeWithHarry's

DATA ANALYTICS HANDBOOK

The Ultimate Job Ready Data Analytics Resource



Excel



Python



SQL



Git & GitHub



AI Tools



Power BI



Statistics



Dashboards



JOB READY
SKILLS



PRACTICAL
PROJECTS



REAL WORLD
TOOLS



CAREER
FOCUSED

Section 1: Introduction to Data Analytics

Chapter 1: What is Data Analytics?

Data analytics means using data to find useful insights that help you make better decisions.

Think of it like this:

- **You collect data:** Collect data like sales numbers, website traffic, survey responses, sensor readings, app usage, etc
- **You clean and organize the data:** Remove errors, fill missing values, standardize formats.
- **You analyze the data:** Use formulas, statistics, tools, and visualizations to understand patterns.
- **You interpret the results:** Convert numbers into actions.
- **You make decisions:** Answer questions like:
 - Which product sells best?
 - Which marketing campaign is working?
 - Where your business is losing money?
 - What customers really want?

Everyday examples

- Netflix analyzes what you watch to recommend the next movie.
- Amazon analyzes your purchases to show relevant products.
- A business analyzes sales to decide how much stock to order.
- A YouTuber analyzes analytics to see which videos perform best.



Chapter 2: What data analysts typically do

SECTION 01

- Work with Excel, SQL, Python, and dashboards
- Create reports, charts, and visualizations
- Solve business questions using data
- Present insights clearly so teams can act on them

In one line:

Data analytics is the science of turning raw data into meaningful information that drives smarter decisions.



The data analytics lifecycle describes how raw data becomes real business decisions. There are **6 core stages**:

1. Collect

We gather relevant data from different sources.

Examples of data sources

- Excel or CSV files
- SQL Databases
- APIs (e.g., Twitter, YouTube)
- Sensors or CRM systems
- Surveys or web analytics (Google Analytics)

Key points

- Collect only what is needed
- Understand data ownership and privacy

2. Clean

Raw data always contains errors. We fix issues to make it ready for analysis.

Common cleaning tasks

- Removing duplicates
- Fixing inconsistent formats (dates, currency, categories)
- Handling missing data
- Standardizing units (Kg vs. g, km vs. miles)

Why it matters

Analysts spend 60 to 80 percent of their time cleaning data. Cleaner data = better insights.

3. Analyze

This is where we start answering business questions using formulas, statistics, or Python/SQL queries.

What we do



- Identify patterns and trends
- Do aggregations (sum, average, counts)
- Compare categories
- Correlation analysis
- Build initial models if needed

Questions we answer

- What happened? (Descriptive)
- Why did it happen? (Diagnostic)

4. Visualize

We convert numbers into charts so teams can understand insights quickly.

Tools used

- Excel charts
- Power BI/Tableau dashboards
- Matplotlib/Seaborn charts

Good visuals show

- Trends
- Comparisons
- Segmentation
- Outliers

5. Report

We communicate insights in a clear and actionable way, typically to non-technical stakeholders.

This includes

- Summary dashboards
- Slide decks
- Business recommendations
- Email updates

Important skill **Storytelling**: Present insights like a narrative, not just numbers.

6. Decision Making

The final business decisions are taken based on what the data shows.

Examples

- Improve product pricing
- Change marketing strategy
- Fix customer dissatisfaction points
- Remove low-performing features

Real Example Workflow: Online Store Sales Analysis

Business Goal: Improve monthly revenue of an e-commerce store.

Step	What happens	Example
Collect	Sales data from SQL database + customer data from CRM	Orders table, Users table
Clean	Remove duplicate orders, fix date formats, fill missing values	Correct wrong prices
Analyze	Find top-selling products, customer repeat rate	Calculation of revenue by category
Visualize	Charts showing sales by category, month, customer type	Line chart + bar chart
Report	Create dashboard + summary recommendations	Category A drives 45 percent revenue
Decision	Increase stock + run ads on best products	Revenue improves next month



Data analysts use a variety of tools to collect, clean, analyze, visualize, and report data. Here are some of the most common ones:

- **Data Collection:** SQL, Google Analytics, APIs, Web Scraping
- **Data Cleaning:** Python & Python libraries like Pandas, Numpy, etc.
- **Data Analysis:** Excel, SQL, Python (NumPy, Pandas) etc.
- **Data Visualization:** Tableau, Power BI, Matplotlib, Seaborn
- **Reporting:** Excel, PowerPoint, etc.

Each tool has its strengths and is chosen based on the specific needs of the project.



Chapter 5: Mini Project: The Profitable Car Showroom

SECTION 01

A Mahindra showroom manager wants to know which car models are bringing higher profit and which models need better marketing or offers.

Dataset

Below are the car models sold in the showroom along with units sold per month and profit per car.

Model	Units Sold per Month	Profit per Car (₹)
Bolero	25	1,50,000
Scorpio	40	1,90,000
Thar	30	2,20,000
XUV700	45	2,50,000
XUV300	50	1,30,000

Task

Add a new column → Monthly Profit Formula: Units Sold × Profit per Car

By quick mental math or calculator:

Model	Units Sold	Profit per Car	Monthly Profit
Bolero	25	1,50,000	₹37,50,000
Scorpio	40	1,90,000	₹76,00,000
Thar	30	2,20,000	₹66,00,000
XUV700	45	2,50,000	₹1,12,50,000
XUV300	50	1,30,000	₹65,00,000

Basic Insights

- XUV700 generates the highest total profit for the showroom.
- Even though XUV300 sells the most units, its lower profit per car gives it less total profit.

Message for Students

Data analytics is not about guessing. It is about using data to help the business take decisions such as:

- Which car to promote more
- Which car needs more marketing
- Which car generates maximum revenue

This is what a data analyst does. We use data to tell the business which product brings money and which product needs help. This is only the start. Soon you will learn Excel, SQL, AI and Python to work with much bigger and complex data.

CodeWithHarry



Section 2: Basic Excel for Data Analytics

Chapter 1: Introduction to Excel for Data Analytics

Welcome to the first module of our Data Analytics journey. Before we dive into advanced tools and techniques, we start with Excel since it is one of the most widely used tools in the world for analyzing data. From small startups to large enterprises, everyone uses Excel to store, organize, and gain insights from data.

How to Get Microsoft Excel

- Most laptops come with Microsoft Office pre-installed
- Excel is available on Windows, macOS, and even on tablets and mobile devices
- You can also use the free online version at **Office.com**



Before doing any analytics in Excel, let's get familiar with the main interface and layout. This will help you navigate quickly and efficiently while working on datasets.

Starting Excel

When you open Excel, you will see two main options:

1. Blank Workbook

This is the most commonly used option. It opens a fresh sheet with rows and columns where you can directly start entering and analyzing data.

2. Templates

Excel also offers built-in templates such as:

- Budget planners
- Calendars
- To-Do lists
- Project trackers

Templates can save time when you need a ready-made structure.

For now, choose "Blank Workbook" to get started

Excel Interface Overview

Once your workbook opens, take a moment to explore the interface:

The Ribbon and Tabs

At the top of Excel, you'll see a **ribbon** with multiple tabs, each containing different tools:

Tab Name	Purpose
Home	Basic tasks like formatting text, alignment, copy-paste, number formats
Insert	Add charts, tables, images, shapes
Draw <i>(may not be available on every version)</i>	Drawing tools for touch/stylus users
Page Layout, Formulas, Data, Review, View	More advanced functionality we will explore later

The ribbon is your toolbox for everything you want to do in Excel.

Hiding the Ribbon

If the ribbon feels distracting:

- **Double-click** on any tab name (Home/Insert etc.)
- The ribbon collapses and gives you more working space
- **Double-click** again to bring it back

This is useful when working on smaller screens.

Quick Access Toolbar

You'll also notice a small toolbar at the very top left of Excel. This is the **Quick Access Toolbar**.

You can customize it to include your frequently used actions like:

- Save
- Undo
- Redo
- Quick Print
- New Workbook

Right-click any command > **Add to Quick Access Toolbar** This helps speed up your workflow.



Excel organizes data in a structured format using **workbooks**, **sheets**, and **cells**.

What is a Workbook?

- A **workbook** is an Excel file
- It can contain **one or multiple sheets**
- Example: A project file with separate sheets for Sales, Expenses, and Summary

You can think of a workbook like a notebook with multiple pages.

What is a Sheet?

- A **sheet** (or **worksheet**) is a single page inside a workbook
- It is made up of **rows** and **columns**
- Each sheet has a separate space to store different sets of data

By default, most new workbooks start with **Sheet1**.

What is a Cell?

- A **cell** is the smallest unit of Excel where data is stored
- It is identified by a **column letter + row number**
- Example: **A1**, **B15**, **C3**

Cells can store:

- Text
- Numbers
- Dates
- Formulas
- Functions

Naming Workbooks and Sheets

Rename Workbook

Rename your file when saving:

[Go to File](#) → [Save As](#)

- Enter a meaningful name like: **Sales_Report_2025.xlsx**

Rename a Sheet

- **Double-click** the sheet name at the bottom
- Type a new name Example: Sales, Employees, Products, etc.

Always use names that clearly describe the data inside.

Inserting Data into a Sheet

To insert data:

1. Click any cell
2. Type the value
3. Press **Enter** or **Tab**

Excel automatically adjusts cell size based on data. You can also adjust column width and row height manually later.

Data Size and Performance

- Excel is extremely powerful but **can lag with very large datasets**
 - Especially when using many formulas or graphics
 - For big data analytics, tools like **SQL, Python, and Power BI** work faster (which we will cover later in this course)
-

Saving Workbooks and AutoSave

- Frequently press **Ctrl + S** to avoid losing work
 - If using **OneDrive or SharePoint**, AutoSave can automatically update changes
 - Local files need manual saving
-

Excel Size Limits

Excel supports a huge amount of data:

- **Rows:** 1,048,576
- **Columns:** 16,384 (Column XFD)

That means each sheet can handle millions of cells of data.



Managing files is an essential part of working with Excel. You should know how to create, save, open, and share your Excel files properly.

Save vs Save As

Save

- Updates the currently open workbook
- If the file is already saved once, all changes are stored in the same file **Shortcut:** `Ctrl + S`

Save As

- Used for **new** workbooks that have never been saved before
- Also useful when saving a **copy** with a different name or location **Note:** For a new document, clicking **Save** will automatically open **Save As** first

Creating New Files

`Go to File` → `New` → `Blank Workbook`

- Or simply use the shortcut: `Ctrl + N`

Opening Workbooks

You can open existing Excel files using:

- `File` → `Open` → `Browse` to select a file
- **Recent files** list to quickly open previously used files **Shortcut:** `Ctrl + O`

Distributing (Sharing) Files

After saving your workbook, you can share the `.xlsx` file through:

- Email
- Cloud services like OneDrive, Google Drive
- USB drives
- Teams or other collaboration tools

Difference between .xlsx vs .xls File Types

Format	Used Since	Notes
.xlsx	2006 onwards	Default format in modern Excel, supports more features
.xls	Before 2006	Older format, limited compatibility

Excel supports many file types, but in this course, we will mainly use `.xlsx`.

Closing Workbooks

When done with a file:

- Go to **File > Close**
- Or use the shortcut: `Ctrl + W`

(This closes only the workbook, not Excel itself.)

Shortcut Summary

Action	Shortcut
Save	<code>Ctrl + S</code>
Open	<code>Ctrl + O</code>
New Workbook	<code>Ctrl + N</code>
Close Workbook	<code>Ctrl + W</code>



Entering data in Excel is simple and efficient. Each cell can store numbers, text, dates, times and more.

Typing Data into Cells

1. Click on a cell to select it.
2. Start typing your value.
3. Press Enter to confirm the entry.

Navigating While Entering Data

- Press **Tab** to move to the next cell in the same row.
- Press **Enter** to move to the next row in the same column.
- Use the arrow keys for quick navigation across the sheet.

Inserting Rows and Columns

You can add new rows and columns when you need more space for data.

- Right click on a row number and select **Insert** to add a row above it.
- Right click on a column letter and select **Insert** to add a column to the left.

When working with tables, always include **headers** for the columns. Headers make sorting, filtering and data analysis much easier.

Quick Access Toolbar Shortcut

Any command that you frequently use can be added to the Quick Access Toolbar.

- Right click on a command in the Ribbon.
- Select **Add to Quick Access Toolbar**.

This toolbar helps keep your most used commands accessible at all times.



Alignment

- **Left Align** keeps text and data toward the left side of the cell. Text is left aligned by default.
- **Right Align** positions values to the right. Numbers are usually right aligned.
- **Center Align** places content in the middle of the cell.

You can either align text horizontally (left, center, right) or vertically (top, middle, bottom) within a cell.

Colors

- You can change **font color** to highlight important text.
- You can change **fill color** to shade the background of cells for better visibility.

Fonts

- You can choose different **font styles, sizes** and **bold, italic or underline** to format text clearly.

Format Painter

- **Format Painter** copies formatting from one cell to another.
- Select a formatted cell, click Format Painter, then apply it to other cells to keep styles consistent.



Excel stores data in cells but controls how that data is displayed using **formats**. Formatting does not change the actual value, only how it looks.

Common Number Formats

- **General** Default format. Excel decides how to show the value.
- **Number** Can show decimals and comma separators. Example: 1,500.00
- **Currency** Shows currency symbols. Example: ₹1,500.00
- **Accounting** Aligns currency symbols and decimal points neatly in a column.
- **Percentage** Converts a number to percent form. Example: 0.25 becomes 25 percent
- **Fraction** Shows numbers as fractions. Example: 0.5 becomes 1/2
- **Scientific** Displays very large or small numbers using E notation. Example: 2.5E+06

Date and Time Formats

Dates and times are stored as numbers in Excel but displayed in readable formats.

- Dates: 04 Dec 2025, 4/12/25
- Time: 12:30 PM, 23:30

Text Format

Content is treated as plain text. Numbers stored as text do not calculate.

Choosing Formats

Select the cell or range Home tab > Number group > Choose the desired format



Excel is often called calculator on steroids because it can perform calculations instantly on large amounts of data.

Entering Formulas

All formulas start with an = sign.

- Example using numbers: `=10+20`
- Example using cell references: `=A1+B1` When you double click the formula cell, Excel highlights the referenced cells in different colors so you can check the formula easily.

AutoSum and Basic Functions

Excel provides built in functions for quick calculations:

- **SUM** adds numbers
- **AVERAGE** calculates the mean value
- **AutoSum** button automatically selects a range and inserts a SUM formula

Fill Handle for Quick Calculations

The small square at the bottom right of a selected cell is called the **Fill Handle**. You can drag it to copy a formula or pattern:

- If you select 12 then drag, it may create a sequence like 13, 14, 15
- If you select 1 and drag, it may repeat like 1, 1, 1, 1 depending on the pattern Excel recognizes

Copying Formulas

Excel uses **relative references** by default. When you copy a formula down, references change automatically based on position.

- Example: `=A1+B1` becomes `=A2+B2` in the next row

Use **absolute references** when you want a cell reference to stay fixed.

- Use `$` symbol Example: `=$A$1+B1` keeps A1 constant when copying



Quick Tip: You can create a new Excel sheet in Windows by right clicking on the desktop or inside a folder, selecting **New**, then **Microsoft Excel Worksheet**.

What Are Relative References

Relative references change automatically when you copy a formula to another cell. Example: `=A1+B1` becomes `=A2+B2` when copied one row down.

What Are Absolute References

Absolute references stay fixed even when you copy the formula. Example: `=$A$1+B1` keeps A1 constant everywhere.

The Dollar Symbol

The `$` sign is used to lock a reference.

- `$A$1` locks both column A and row 1
- `$A1` locks the column only
- `A$1` locks the row only

What Happens When You Copy

- **Relative reference copied:** Excel shifts the reference based on the new location. Formulas update automatically.
- **Absolute reference copied:** Excel keeps the locked part unchanged, so the formula always points to the same cell.

This makes it easy to create formulas that adjust when needed or stay fixed when required.



Chapter 10: Order of Operations in Excel

SECTION 02

Excel follows a standard order of operations when evaluating formulas. This is commonly known as **BODMAS** or **PEMDAS**. Both represent the same calculation sequence.

B/P

Brackets / Parentheses

O/E

Orders / Exponents

D

Division

M

Multiplication

A

Addition

S

Subtraction

Excel always evaluates formulas in this order unless parentheses change the sequence.

Use Parentheses to Be Sure

Always use parentheses to control the calculation and avoid mistakes. Example: `=(10+5)*2` ensures the addition happens before multiplication.

Evaluate Formula Tool

Excel provides an **Evaluate Formula** option to check how Excel solves a formula step by step. You can find it under: Formulas tab > Formula Auditing group > Evaluate Formula

This helps you understand and debug complex formulas.



Functions are built in formulas that perform specific calculations. Instead of writing long formulas yourself, you use a function that already knows how to do the work.

Structure of a Function

Every function follows the same pattern:

```
=FUNCTION_NAME(arguments)
```

Example:

```
=SUM(A1:A10)
```

Using Entire Rows and Columns

You can apply functions to full rows and columns:

- Sum an entire column B `=SUM(B:B)`
- Sum an entire row 1 `=SUM(1:1)`

Multiple Arguments

Functions can take more than one argument:

```
=SUM(J20:J22,4)
```

This adds the range J20 to J22 and the number 4.

Function Arguments Window

Clicking the **F(x)** button next to the formula bar opens the Function Arguments window. This helps you enter values step by step.

You can also click **Help on this function** inside that window to open detailed documentation.

Autocomplete Suggestions

When you start typing a function name, Excel shows suggestions. This makes it easy to find the correct function and reduces spelling mistakes.

Where to Find Functions

Go to: Formulas tab > Function Library group > Insert Function

This is where you can browse functions by category such as Math, Text, Logical and more.

Excel provides many built in functions, but a few are used in almost every worksheet.

SUM Function

The SUM function adds numbers quickly.

```
=SUM(A1:A10)
```

You do not need to type each cell reference again and again. Use **AutoSum** to add a range instantly.

Shortcut: **Alt + =** inserts a SUM formula automatically.

MIN Function

Finds the smallest value in a range.

```
=MIN(A1:A10)
```

You can also use an entire column:

```
=MIN(B:B)
```

MAX Function

Finds the largest value in a range.

```
=MAX(A1:A10)
```

AVERAGE Function

Calculates the mean of numbers.

```
=AVERAGE(A1:A10)
```

COUNT Function

Counts how many cells contain numbers.

```
=COUNT(A1:A10)
```

Copying Formulas with Auto Fill Handle

Select the cell containing the formula, then drag the fill handle (small square at the bottom-right corner) across nearby cells. Excel copies the formula and automatically updates the cell references for each new row or column, making it easy to perform the same calculation on multiple sets of data.

CodeWithHarry



SECTION 02

You will also use functions like **SUM**, **AVERAGE**, **COUNT**, and **COUNTIF** to analyze your data and generate useful insights.

S.No	Description	Type	Amount	Food	Fun	Office	Investment
1	Tea	food	15	15	0	0	0
2	Snacks	food	8	8	0	0	0
3	Gulab Jamun	food	20	20	0	0	0
4	Lunch	food	80	80	0	0	0
5	Tea	food	10	10	0	0	0
6	Stationary	office	40	0	0	40	0
7	Zerodha	investment	1000	0	0	0	1000
8	Icici	investment	2000	0	0	0	2000
9	Dinner	food	90	90	0	0	0
10	Birthday of a friend	fun	300	0	300	0	0

	Amount	Percentage
Total for the Day	3563	-
Average Transaction Value	356.3	-
Food Expenses	223	6.258770699
Fun Expenses	300	8.419870895
Office Expense	40	1.122649453
Investments	3000	84.19870895

Moving, Copying and Deleting Cells

- You can **drag and drop a cell** to move its contents.
- You can **drag and drop a selected range** to move multiple cells together.
- Use **Cut (Ctrl + X)** and **Paste (Ctrl + V)** to move data. Press **Ctrl + D** to deselect after pasting.
- Deleting cells removes the data and may shift surrounding cells.

Inserting Rows and Columns

- Use **Ctrl + Plus (+)** to insert a row or column.
- Use **Ctrl + Minus (-)** to delete a row or column.
- If you do not have a number pad, use **Ctrl + Shift + =** to insert.

Resizing Rows and Columns

- Resize rows or columns by dragging their borders.
- If a column is too narrow, Excel may show **#####**. Increase the column width to display the value correctly.

Hide and Unhide

- You can hide rows or columns to reduce clutter.
- Hidden rows or columns can be unhidden later from the same menu.

Working with Worksheets

- Deleted worksheets cannot be undone.
- Hold **Ctrl** and drag a worksheet tab to create a copy.
- You can move a worksheet to a different workbook by dragging its tab to the other workbook window.



Font Formatting

- You can apply **bold**, **italic** and **underline** to text.
- Change **font color** to highlight important values.
- Use **cell fill color** to shade the background of cells.

Themes

- Go to [Page Layout](#) → [Themes](#) to apply a consistent look across the entire worksheet.
- Themes control fonts, colors and effects together.

Borders

- Borders help separate and organize data.
- You can change **border line color** and **line styles**.
- Use **More Borders** to access advanced border options.

Number Formatting

- Percentages can be applied directly or by using the **fraction** format when appropriate.
- Formatting affects display only, not the underlying value.

Cell Styles

- **Cell Styles** provide pre defined formatting for titles, headings, totals and more.
- Using styles helps keep formatting consistent across the sheet.



Conditional formatting changes the appearance of cells based on their values. It helps you quickly spot patterns, trends and outliers.

Highlight Cells Rules

- Highlight cells based on conditions such as greater than, less than or equal to.
- You can also highlight duplicate values, text or dates.

Data Bars

- Data bars add visual bars inside cells.
- The bar length represents the value relative to other cells in the range.

Manage Rules

- Use **Manage Rules** to view all conditional formatting rules applied to a sheet.
- You can control rule priority and where each rule applies.

Edit Rules

- Edit a rule to change conditions, formatting or ranges.
- This is useful when your data or logic changes.

Clear Rules

- Clear rules to remove conditional formatting.
- You can clear rules from selected cells or from the entire worksheet.



Sorting and filtering help you organize and analyze data more effectively.

Sorting Data

- Go to **Data** → **Sort** to arrange data in ascending or descending order.
- You can sort by numbers, text, dates or custom criteria.

Filtering Data

- Go to **Data** → **Filter** to enable filters on column headers.
- Filters allow you to view only the data that meets specific conditions.

Keyboard Shortcut

- **Alt + D + F + F** is the shortcut to apply filters.
- This shortcut toggles the filter on and off.
- Once filters are applied, you can also use them to sort data quickly from the header dropdowns.



Excel provides several functions to generate random numbers for testing and analysis.

RAND Function

- `=RAND()` Returns a random decimal number between 0 and 1.
- The value changes every time the worksheet recalculates.

RANDBETWEEN Function

- `=RANDBETWEEN(min, max)` Returns a random whole number between the specified minimum and maximum values.
- Example: `=RANDBETWEEN(10, 50)`

RANDARRAY Function

- `=RANDARRAY(rows, columns, min, max, whole_number)` Generates an array of random numbers.
- Example: `=RANDARRAY(5, 1, 1, 100, TRUE)`

Random Number Using Min and Max Technique

You can generate a random number between any two values using:

```
=min + (RAND() * (max - min))
```

Example:

```
=10 + (RAND() * (50 - 10))
```

This returns a random decimal number between 10 and 50.

QuickTip: Selecting Data Quickly

- Use **Ctrl + Shift + Down Arrow** to select all continuous data below the current cell.



Excel allows you to add visual elements like images, shapes and diagrams to make your sheets more informative.

Inserting Images

Go to Insert → Illustrations → Pictures to insert pictures.

- **Online Pictures** are powered by Bing Image Search.

Working with Shapes

Go to Insert → Shapes to add shapes.

- Available shape categories include:
- Basic Shapes
- Block Arrows
- Equation Shapes
- Flowchart
- Stars and Banners
- Callouts

Resizing and Adjusting Shapes

- **White markers** are used to resize the shape.
- **Orange markers** are used to adjust the thickness or internal structure of certain shape elements.

Shape Effects

- Use **Shape Effects** to add shadows, reflections and other visual styles.

SmartArt

- Go to **Insert** → **SmartArt** to insert diagrams such as processes, hierarchies and relationships.
- SmartArt is useful for explaining workflows and structured information visually.



Excel charts help visualize data and identify trends quickly.

Common Chart Types

Column, Line, Pie, Bar, Area and Scatter charts are commonly used to compare values, show trends, display proportions and analyze relationships between data.

- Column Charts: Useful for comparing values across categories.
- Line Charts: Ideal for showing trends over time.
- Pie Charts: Show proportions of a whole.
- Bar Charts: Similar to column charts but oriented horizontally.
- Area Charts: Emphasize the magnitude of change over time.
- Scatter Charts: Show relationships between two variables.

Inserting a Chart

- Select the data.
- Go to **Insert** → **Charts** and choose the required chart type.

Chart Design and Data

- Use **Chart Design > Select Data** to modify the data source.
- You can add or remove data series from here.

Formatting Charts

- Use **Format Axis** to change scale, labels and number formatting.
- You can format chart elements such as colors, fonts, gridlines and legends.

Moving Charts

- Go to **Chart Design** → **Move Chart** to place the chart on a new worksheet or move it back into the current sheet.

Pie Chart Data Labels

- Use **Format Data Labels** to show values, percentages or category names.
- To pull out a slice, click the pie once, click the slice again and drag it outward.



Printing in Excel gives you control over layout, scaling and what content appears on paper.

Print Preview and Page Breaks

- After opening **Print Preview** and returning to the sheet, Excel displays **page breaks**.
- Page breaks show how the sheet will be split across printed pages.

Printing Charts

- If a chart is selected while printing, **only the chart** will be printed.

Scaling Options

- Use **custom scaling** to fit content on one page or a specific number of pages.
- Scaling options are available in the Print settings.

Page Setup and Margins

- Go to **Page Setup** → **Margins** to adjust spacing around the content.
- Proper margins help prevent content from being cut off.

Page Layout and Zoom

- Use **Page Layout** view to see how the sheet fits on pages.
- Go to **View** → **Zoom** to adjust how large or small the sheet appears on screen.

Headers and Footers

- You can add **page numbers** using Header and Footer options.
- This is useful for multi page printouts.

Print Selection

- Use **Print Selection** to print only the selected cells instead of the entire worksheet.

Microsoft Print to PDF

- Select **Microsoft Print to PDF** as the printer to save the document as a PDF file instead of printing on paper.



You can use templates in Excel to create pre designed spreadsheets for various purposes. Templates save time and help maintain consistency across documents.

Using Built In Templates

Excel provides ready made templates for budgets, invoices, trackers, calendars and more. - Go to

File



New

- Search for the required template type and select one

Creating a Template from an Existing Sheet

- Right click on the sheet tab
- Select **Move or Copy**
- Choose **New Book**
- Enable **Create a copy**

Saving as an Excel Template

Go to File



Save As

- Select file type **Excel Template (*.xltx)**
- Save the file

Custom Office Templates Location

Save your templates in the following directory on your computer (Windows):

```
C:\Users\YourUsername\Documents\Custom Office Templates
```

Using this location is important because when you create a new document in Excel and select **Personal** under the template options, Excel shows the templates saved in this folder.



Excel allows you to use data from one worksheet inside another worksheet within the same workbook.

Basic Cross Worksheet Reference

To reference a cell from another sheet, use:

```
=SheetName!CellReference
```

Example:

```
=Sheet2!A1
```

Using References in Formulas

You can perform calculations using cells from different sheets:

```
=Sheet1!A1 + Sheet2!A1
```

Referencing Ranges

You can also reference a full range from another worksheet:

```
=Sheet1!A1:A10
```

How Excel Helps You

- Type `=` and switch to another sheet.
- Click the required cell or range.
- Excel automatically inserts the correct reference.

Important Notes

- If the sheet name contains spaces, Excel adds quotes automatically:

```
= 'Sales Data' !B2
```

- Renaming a sheet updates references automatically.

Cross worksheet referencing is useful for summaries, dashboards and keeping raw data separate from calculations.

Section 3: Basic Python (AI-powered)

Chapter 1: Introduction to Python for Data Analytics

Why Python for Data Analytics

Python is one of the most popular languages in data analytics because it is simple, readable, and extremely powerful for handling data.

Key reasons Python is used in data analytics: - Easy to learn and write - Large ecosystem of analytics libraries - Strong community support - Works well with Excel, databases, and visualization tools

What You Will Learn in This Section

In this section, learners will build a strong Python foundation focused only on what is actually required for data analytics.

You will learn: - How Python fits into the data analytics workflow - Writing and running Python code - Working with numbers, text, and variables - Making decisions using conditions - Repeating tasks using loops - Organizing logic using functions

AI Powered Python for Data Analytics

What Is AI Powered Python

AI powered Python means using artificial intelligence tools to write, understand, debug, and improve Python code faster. Instead of replacing learning, AI acts as a smart assistant that increases productivity.

Why AI Matters for Data Analysts

Modern data analysts do not work alone. AI tools help in: - Generating Python code quickly - Explaining complex logic in simple terms - Debugging errors faster - Improving code quality and efficiency

This reflects how analytics is actually done in real companies today.

How AI Is Used in This Course

In this section, learners will see practical usage of AI while coding in Python.

You will learn: - How to ask AI the right questions for analytics problems - Converting problem statements into Python logic using AI - Understanding AI generated code line by line - Fixing bugs and edge cases with AI assistance

AI as a Learning Accelerator

AI helps beginners overcome fear and confusion by: - Breaking problems into smaller steps - Explaining errors in simple language - Providing multiple approaches to the same problem

Learners still think and decide, AI only speeds up the process.

Why Proper Setup Matters

Before writing any code, it is important to set up the right tools. A correct Python and editor setup ensures a smooth learning experience and avoids common beginner issues.

Installing Python

Python is the language we will use for all data analytics tasks.

In this section, you will learn:

- How to download and install Python
- How to verify that Python is installed correctly
- Understanding the Python version and basic setup

Python is available for Windows, macOS, and Linux. Always install the **latest stable version** from the official website.

Installation

1. Open your browser and go to <https://www.python.org>.
2. Click on **Downloads** and download the latest Python version for Windows.
3. Once the download is complete, double-click the installer to begin the installation.
4. On the first setup screen, check the **Add Python to PATH** checkbox. This is a very important step.
5. Click **Install Now**.
6. Wait while Python is installed on your computer.
7. When the installation finishes, click **Close**.
8. Open **Command Prompt** or **Terminal** and run the following command to verify the installation:

```
python --version
```

If Python has been installed successfully, the terminal will display the installed Python version.

Your Python installation is now complete and ready to use.

Installing Visual Studio Code

Visual Studio Code is a lightweight and powerful code editor widely used by developers and data analysts.

In this section, you will learn:

- How to install Visual Studio Code
- How to enable Python support in VS Code
- How to run Python programs inside the editor

Installation

1. Open your browser and visit <https://code.visualstudio.com>.
2. Click the **Download** button.
3. Select your operating system.
4. Download the installer.
5. Run the installer.
6. Accept the license agreement.
7. On the Additional Tasks screen, enable the following options:
 - Add "Open with Code" action to Windows Explorer file context menu.
 - Add "Open with Code" action to Windows Explorer directory context menu.
 - Register Code as an editor for supported file types.
 - Add to PATH.
8. Click **Install**.
9. After the installation is complete, click **Finish** to launch Visual Studio Code.

You now have both Python and Visual Studio Code installed and are ready to start writing Python programs.



What Are Python Packages

Python packages are reusable pieces of code that add extra functionality to Python. In data analytics, packages help with tasks like data handling, calculations, and visualization.

What Is Pip

Pip is the official package manager for Python. It allows you to download and install Python packages from the Python Package Index.

Installing Packages Using Pip

To install a package, you normally use the pip command in the terminal or command prompt.

```
pip install package_name
```

This will download and install the package so it can be used in your Python programs.

If Pip Command Does Not Work

On some systems, the pip command may not work directly due to path or environment issues. In such cases, you can use Python itself to run pip.

Use this command instead:

```
python -m pip install package_name
```

This ensures that pip runs using the same Python version that is installed on your system.

Verifying Installation

After installing a package, you can verify it by importing it in Python or checking the installed packages list.

By the end of this section, learners will be confident in installing and managing Python packages required for data analytics.



In the last lesson, we learned what Python packages are and how to install them using **pip**.

Now, let's understand the difference between **built-in** and **external** packages, and how to use both in real programs.

What Are Python Modules?

A **module** is a single Python file that contains reusable code such as: - Functions - Classes - Variables

Python uses modules to organize code and avoid writing the same logic again and again.

Example:

```
import math
```

Here, `math` is a module that provides mathematical functions.

A **package** is a collection of multiple related modules.

Built-in (Standard Library) Modules

Built-in modules come pre-installed with Python. You do **not** need to install them using **pip**.

They are part of Python's **Standard Library** and are available as soon as Python is installed.

Examples of Built-in Modules

- `os` – Interact with the operating system
- `sys` – Work with Python runtime
- `math` – Mathematical functions

External (Third-Party) Packages

External packages are not included with Python by default. They must be installed using **pip**.

These packages are created by the Python community and are widely used in real-world applications.

Examples of External Packages

- `pandas` – Data analysis and manipulation
- `numpy` – Numerical computing
- `matplotlib` – Data visualization

Using an External Package (pandas)

Install Pandas

```
pip install pandas
```

If pip does not work:

```
python -m pip install pandas
```

Use Pandas in Python

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [24, 30, 28]
}

df = pd.DataFrame(data)
print(df)
```

Explanation:

- `pandas` is imported using the alias `pd`
- `DataFrame` is used to work with tabular data

Built-in vs External: Key Differences

Feature	Built-in Modules	External Packages
Installation	Comes with Python	Installed using pip
Internet Required	No	Yes (for installation)
Examples	os, math, sys	pandas, numpy, matplotlib
Usage	General-purpose tasks	Specialized tasks (data, ML, web)

Finding More Built-in Modules

Python provides a large number of built-in modules in its standard library. You can explore the complete list here: <https://docs.python.org/3/py-modindex.html>

Before this, we have already installed Python, set up VS Code, and understood how packages work. Now we start with the core of Python.

This lecture is extremely important because almost everything in data analytics is built on variables and data types.

What is a Variable?

A variable is simply a container that stores a value.

Think of it like a labeled box.

The label is the variable name and the item inside the box is the value.

Example

```
age = 25
```

Here:

- `age` is the variable
- `25` is the value stored inside it

Another example:

```
name = "Harry"
```

Now Python remembers that `name` holds the value `"Harry"`.

In Python, you do not need any special keyword to create a variable. Just write the name, use `=` and assign a value.

What are Data Types?

A data type tells Python what kind of value a variable is holding.

In real life:

- Age is a number
- Name is text
- Price can be a decimal

Python follows the same logic.

Integer (int)

Integers are whole numbers without decimals.

```
students = 100  
orders = 45
```

Integers are commonly used for counts and quantities.

Float (decimal numbers)

Floats are numbers that contain decimals.

```
price = 2999.99  
rating = 4.5
```

In data analytics, floats are used for prices, averages, percentages, and measurements.

String (text data)

Text values in Python are called strings. Anything written inside single or double quotes is a string.

```
city = "Delhi"  
email = "harry@gmail.com"
```

Strings are heavily used for names, cities, emails, and categories.

Boolean (True or False)

Boolean values represent logical conditions.

They can have only two values:

- True
- False

```
is_active = True  
is_paid = False
```

Booleans are very common in analytics for flags and conditions.

How Python Identifies Data Types

Python automatically detects the data type of a value.

You do not need to specify it manually.

You can check the data type using the `type()` function:

```
type(age)
type(name)
```



In the previous lecture, we learned about variables and data types.

Now we will cover two small but extremely important concepts that you will use all the time in data analytics:

- Comments
- Type conversion

These concepts become even more important when you start working with AI generated code.

Comments in Python

Comments are written for humans, not for Python.

Python completely ignores comments while running the code.

They exist only to explain what the code is doing.

Single Line Comments

In Python, comments start with the `#` symbol.

```
# This stores the user's age  
age = 25
```

The line starting with `#` will not be executed.

Why Comments Matter in Data Analytics

In real analytics work:

- Code is read more times than it is written
- You might revisit your code after weeks or months
- AI generated code often needs explanation

Comments help you:

- Understand the logic quickly
- Modify AI generated code safely
- Debug issues faster

Writing comments is a professional habit.

What is Type Conversion?

Type conversion means changing one data type into another.

This is extremely common when working with real data.

Converting String to Integer

Data coming from CSV files or user input is often in string format.

```
age = "25"
```

Even though it looks like a number, it is still a string.

To convert it into an integer:

```
age = int(age)
```

Now you can perform calculations on it.

Converting String to Float

Decimal values are converted using `float()`.

```
price = "1999.99"  
price = float(price)
```

This is commonly used for prices, ratings, and averages.

Converting Number to String

Sometimes you need to convert numbers into strings, especially while generating reports or messages.

```
total = 500  
message = "Total sales: " + str(total)
```

Common Errors in Type Conversion

Not every string can be converted into a number.



```
value = "abc"  
int(value)
```

This will cause an error. Always validate your data before converting it. Later, we will handle such cases using error handling.

CodeWithHarry



In many real world programs, data does not always come from files or databases. Sometimes, the user needs to provide input while the program is running.

Python provides a simple way to take input from the user using the `input()` function.

The `input()` Function

The `input()` function pauses the program and waits for the user to type something.

```
name = input()
```

Whatever the user types is stored in the variable `name`.

Taking Input with a Prompt Message

You can guide the user by showing a message inside `input()`.

```
name = input("Enter your name: ")
```

This improves usability and avoids confusion.

Important Note About Input Data Type

By default, all input taken using `input()` is a string.

Even if the user enters a number, Python will treat it as text.

```
age = input("Enter your age: ")  
type(age)
```

The data type of `age` will be `str`.

Converting User Input to Integer

If you want to perform calculations, you must convert the input.

```
age = int(input("Enter your age: "))
```

Now `age` can be used in numeric operations.

Converting User Input to Float

For decimal values, use `float()`.

```
price = float(input("Enter product price: "))
```

This is common for prices, ratings, and measurements.

Using User Input in Calculations

```
quantity = int(input("Enter quantity: "))
price = float(input("Enter price: "))

total = quantity * price
print("Total amount:", total)
```

Common Input Errors

If the user enters invalid data, the program will crash.

```
age = int(input("Enter age: "))
```

If the user types text instead of a number, an error will occur.

Handling such cases will be covered later using error handling.



Operators are symbols used to perform operations on values and variables. They allow you to calculate, compare, assign, and combine data in different ways.

Python provides several types of operators, each serving a specific purpose.

Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Description	Example
+	Addition	10 + 5
-	Subtraction	10 - 5
*	Multiplication	10 * 5
/	Division	10 / 5
//	Floor Division	10 // 3
%	Modulus (remainder)	10 % 3
**	Exponent	2 ** 3

Example:

```
a = 10
b = 3

print(a + b)
print(a // b)
print(a ** b)
```

Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Description	Example
=	Assign	x = 5
+=	Add and assign	x += 2
-=	Subtract and assign	x -= 2
*=	Multiply and assign	x *= 2
/=	Divide and assign	x /= 2
%=	Modulus and assign	x %= 2
//=	Floor divide and assign	x //= 2
**=	Exponent and assign	x **= 2

Example:

```
x = 10
x += 5
```

Comparison Operators

Comparison operators compare two values and return either `True` or `False`.

Operator	Description	Example
=	Equal to	a == b
≠	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
≥	Greater than or equal	a ≥ b
≤	Less than or equal	a ≤ b

Example:

```
a = 10
b = 5

print(a > b)
```

Logical Operators

Logical operators are used to combine conditional statements.

Operator	Description	Example
and	Both true	a > 5 and b < 10
or	Either true	a > 5 or b > 10
not	Reverse	not(a > 5)

Example:

```
a = 10
b = 3

print(a > 5 and b < 5)
```

Bitwise Operators

Bitwise operators work on binary representations of numbers.

Operator	Description	
&	AND	
	OR	
^	XOR	
~	NOT	
<<	Left shift	
>>	Right shift	

Example:

```
a = 5
b = 3

print(a & b)
```

Membership Operators

Membership operators are used to test whether a value exists in a sequence.

Operator	Description
<code>in</code>	Exists in sequence
<code>not in</code>	Does not exist

Example:

```
numbers = [1, 2, 3]

print(2 in numbers)
```

Identity Operators

Identity operators compare memory locations of objects, not values.

Operator	Description
<code>is</code>	Same object
<code>is not</code>	Different object

Example:

```
a = [1, 2]
b = a

print(a is b)
```

Operator Precedence

Python evaluates operators in a specific order. Operators with higher precedence are evaluated first.

Precedence Order	Operators	Description
1	()	Parentheses
2	**	Exponentiation
3	*, /, //, %	Multiplication / Division
4	+, -	Addition / Subtraction
5	=, $\neq$, >, <	Comparison
6	not, and, or	Logical operators

Operators at the top are evaluated before operators lower in the table.

Using parentheses is the safest way to control evaluation order and make expressions easier to read.



Strings are used to store text data in Python.

Any value written inside single quotes or double quotes is treated as a string.

Strings are one of the most commonly used data types and Python provides many built in methods to work with them.

Creating Strings

Strings can be created using single quotes or double quotes.

```
name = "Harry"  
city = 'Delhi'
```

Both forms work the same way.

Multiline Strings

Multiline strings are created using triple quotes.

```
message = """This is a  
multi line  
string"""
```

Accessing Characters in a String

Strings are indexed, which means each character has a position.

```
text = "Python"  
  
print(text[0])  
print(text[3])
```

Indexing starts from 0 .

String Length

You can find the length of a string using the `len()` function.

```
text = "Python"  
length = len(text)
```

String Slicing

Slicing is used to extract a portion of a string.

```
text = "Python"
print(text[0:3])
print(text[2:])
print(text[:4])
```

When slicing a string with negative indexes, Python internally adds the length of the string to the negative value.

```
text[-3:]      # same as text[6-3 : 6]
text[-5:-2]    # same as text[6-5 : 6-2]
```

So Python converts negative indexes like this:

```
-3  →  6 - 3 = 3
-5  →  6 - 5 = 1
-2  →  6 - 2 = 4
```

Common String Methods

Python provides many built in string methods for common operations.

lower() and upper()

Convert string to lowercase or uppercase.

```
text = "Python"
print(text.lower())
print(text.upper())
```

strip()

Removes extra spaces from the beginning and end of a string.

```
text = "  hello  "
print(text.strip())
```

replace()

Replaces part of a string with another string.

```
text = "Hello World"
print(text.replace("World", "Python"))
```

split()

Splits a string into a list based on a separator.

```
text = "apple,banana,orange"
items = text.split(",")
```

join()

Joins elements of a list into a single string.

```
items = ["apple", "banana", "orange"]
text = ",".join(items)
```

find()

Finds the position of a substring.

```
text = "Hello World"
position = text.find("World")
```

Returns `-1` if the value is not found.

startswith() and **endswith()**

Checks whether a string starts or ends with a given value.

```
email = "test@gmail.com"

print(email.startswith("test"))
print(email.endswith(".com"))
```

String Concatenation

Strings can be combined using the `+` operator.

```
first = "Hello"
second = "World"

result = first + " " + second
```

String Formatting

String formatting allows inserting values into strings.

Using f strings

```
name = "Harry"
age = 25

message = f"My name is {name} and I am {age} years old"
```

Checking String Content

Python provides methods to check string content.

```
text = "Python123"

print(text.isalpha())
print(text.isdigit())
print(text.isalnum())
```

Strings are Immutable

Strings cannot be changed after creation.

```
text = "Python"
text[0] = "J"
```

This will cause an error. Any modification creates a new string.

f-Strings in Python

f-strings (formatted string literals) are an easy and modern way to format strings in Python. They were introduced in **Python 3.6**.

To use an f-string, you add an `f` before the string and place variables or expressions inside `{}`.

Basic Example

```
name = "Harry"
age = 25

print(f"My name is {name} and I am {age} years old")
```

Here, `{name}` and `{age}` are automatically replaced with their actual values.

Why f-Strings?

- Cleaner than string concatenation (`+`) and `.format()`
- Faster and easier to read
- Supports expressions and method calls

Lists are used to store multiple values in a single variable.

They are one of the most flexible and commonly used data structures in Python.

A list can store different types of data and can be modified after creation.

Creating a List

Lists are created using square brackets `[]`.

```
numbers = [1, 2, 3, 4]
names = ["Alice", "Bob", "Charlie"]
mixed = [1, "Python", 3.5, True]
```

Accessing List Elements

List elements are accessed using indexes.

```
items = ["apple", "banana", "orange"]

print(items[0])
print(items[2])
```

Indexing starts from `0`.

Negative Indexing

Negative indexes start from the end of the list.

```
items = ["apple", "banana", "orange"]

print(items[-1])
print(items[-2])
```

List Length

Use the `len()` function to get the number of elements in a list.

```
items = ["apple", "banana", "orange"]
count = len(items)
```


List Slicing

Slicing is used to extract part of a list.

```
items = ["apple", "banana", "orange", "mango"]

print(items[1:3])
print(items[:2])
print(items[2:])
```

Modifying List Elements

Lists are mutable, which means their values can be changed.

```
items = ["apple", "banana", "orange"]
items[1] = "grapes"
```

Adding Elements to a List

append()

Adds an element to the end of the list.

```
items = ["apple", "banana"]
items.append("orange")
```

insert()

Inserts an element at a specific index.

```
items = ["apple", "banana"]
items.insert(1, "orange")
```

extend()

Adds multiple elements from another list.

```
items = ["apple", "banana"]
more_items = ["orange", "mango"]

items.extend(more_items)
```

Removing Elements from a List

`remove()`

Removes the first occurrence of a value.

```
items = ["apple", "banana", "orange"]
items.remove("banana")
```

`pop()`

Removes and returns an element at a given index. If no index is provided, it removes the last element.

```
items = ["apple", "banana", "orange"]
items.pop()
items.pop(0)
```

`clear()`

Removes all elements from the list.

```
items = ["apple", "banana"]
items.clear()
```

Finding Elements in a List

`index()`

Returns the index of the first occurrence of a value.

```
items = ["apple", "banana", "orange"]
position = items.index("banana")
```

`count()`

Counts how many times a value appears in the list.

```
items = ["apple", "banana", "apple"]
count = items.count("apple")
```

Sorting Lists

sort()

Sorts the list in ascending order.

```
numbers = [3, 1, 4, 2]
numbers.sort()
```

To sort in reverse order:

```
numbers.sort(reverse=True)
```

sorted()

Returns a new sorted list without modifying the original list.

```
numbers = [3, 1, 4, 2]
new_list = sorted(numbers)
```

Reversing a List

reverse()

Reverses the order of elements in the list.

```
items = ["apple", "banana", "orange"]
items.reverse()
```

Copying Lists

copy()

Creates a shallow copy of the list.

```
items = ["apple", "banana"]
new_items = items.copy()
```

Checking Membership

Use the `in` keyword to check if an element exists in a list.

```
items = ["apple", "banana", "orange"]  
  
print("apple" in items)
```

Lists are Mutable

Lists can be modified after creation.

```
items = [1, 2, 3]  
items[0] = 10
```



Tuples are used to store multiple values in a single variable, similar to lists.

The key difference is that tuples are immutable, which means their values cannot be changed after creation.

Tuples are commonly used when the data should remain constant.

Creating a Tuple

Tuples are created using parentheses `()`.

```
numbers = (1, 2, 3)
names = ("Alice", "Bob", "Charlie")
mixed = (1, "Python", 3.5, True)
```

Single Element Tuple

A tuple with one element must include a trailing comma.

```
single = (5,)
```

Without the comma, Python will treat it as a normal value.

Accessing Tuple Elements

Tuple elements are accessed using indexes.

```
items = ("apple", "banana", "orange")

print(items[0])
print(items[2])
```

Indexing starts from `0`.

Negative Indexing

Negative indexes start from the end of the tuple.

```
items = ("apple", "banana", "orange")

print(items[-1])
print(items[-2])
```

Tuple Length

Use the `len()` function to get the number of elements in a tuple.

```
items = ("apple", "banana", "orange")
count = len(items)
```

Tuple Slicing

Slicing is used to extract part of a tuple.

```
items = ("apple", "banana", "orange", "mango")

print(items[1:3])
print(items[:2])
print(items[2:])
```

Tuples are Immutable

Tuples cannot be modified after creation.

```
items = ("apple", "banana", "orange")
items[1] = "grapes"
```

This will cause an error.

Tuple Methods

Tuples have fewer built in methods compared to lists.

`count()`

Returns the number of times a value appears in the tuple.

```
items = ("apple", "banana", "apple")
count = items.count("apple")
```

`index()`

Returns the index of the first occurrence of a value.


```
items = ("apple", "banana", "orange")
position = items.index("banana")
```

Looping Through a Tuple

You can loop through tuple elements using a `for` loop.

```
items = ("apple", "banana", "orange")

for item in items:
    print(item)
```

Tuple Packing and Unpacking

Packing

Multiple values can be packed into a tuple.

```
data = 10, 20, 30
```

Unpacking

Tuple values can be unpacked into separate variables.

```
a, b, c = data
```

Converting Between List and Tuple

You can convert a tuple to a list and vice versa.

```
items = ("apple", "banana")
items_list = list(items)

items_tuple = tuple(items_list)
```

When to Use Tuples

- When data should not change
- When you want to protect values from modification
- When returning multiple values from a function

CodeWithHarry



Sets are used to store multiple values in a single variable.

A set does not allow duplicate values and does not maintain any specific order.

Sets are commonly used when you want unique elements.

Creating a Set

Sets are created using curly braces `{}` or the `set()` function.

```
numbers = {1, 2, 3, 4}
names = {"Alice", "Bob", "Charlie"}
```

To create an empty set, use `set()`.

```
empty_set = set()
```

Using `{}` creates an empty dictionary, not a set.

Unique Elements in a Set

Sets automatically remove duplicate values.

```
numbers = {1, 2, 2, 3, 3, 4}
print(numbers)
```

Accessing Set Elements

Sets do not support indexing or slicing because they are unordered.

To access elements, you must loop through the set.

```
items = {"apple", "banana", "orange"}

for item in items:
    print(item)
```

Adding Elements to a Set

`add()`

Adds a single element to the set.

```
items = {"apple", "banana"}  
items.add("orange")
```

update()

Adds multiple elements from another iterable.

```
items = {"apple", "banana"}  
items.update(["orange", "mango"])
```

Removing Elements from a Set**remove()**

Removes a specified element. Raises an error if the element does not exist.

```
items = {"apple", "banana"}  
items.remove("banana")
```

discard()

Removes a specified element without raising an error.

```
items = {"apple", "banana"}  
items.discard("grapes")
```

pop()

Removes and returns a random element.

```
items = {"apple", "banana", "orange"}  
items.pop()
```

clear()

Removes all elements from the set.

```
items = {"apple", "banana"}  
items.clear()
```

Set Operations

Sets support mathematical set operations.

Union

Combines elements from both sets.

```
a = {1, 2, 3}
b = {3, 4, 5}

result = a.union(b)
```

Intersection

Returns common elements between sets.

```
result = a.intersection(b)
```

Difference

Returns elements present in the first set but not in the second.

```
result = a.difference(b)
```

Symmetric Difference

Returns elements present in either set but not in both.

```
result = a.symmetric_difference(b)
```

Membership Testing

Use the `in` keyword to check if an element exists in a set.

```
items = {"apple", "banana", "orange"}

print("apple" in items)
```

Set Methods Summary

Common set methods include:

- `add()`
- `update()`
- `remove()`
- `discard()`
- `pop()`
- `clear()`
- `union()`
- `intersection()`
- `difference()`
- `symmetric_difference()`

Sets are Mutable

Sets can be modified after creation.

```
items = {"apple", "banana"}  
items.add("orange")
```

Important Notes About Sets

- Sets do not maintain order
- Sets do not allow duplicate values
- Sets do not support indexing
- Elements in a set must be immutable



Dictionaries are used to store data in key value pairs.

Each key is linked to a value, making dictionaries ideal for structured data.

Dictionaries are one of the most important data structures in Python.

Creating a Dictionary

Dictionaries are created using curly braces `{}` with key value pairs separated by colons.

```
student = {  
    "name": "Harry",  
    "age": 25,  
    "city": "Delhi"  
}
```

You can also create a dictionary using the `dict()` function.

```
student = dict(name="Harry", age=25, city="Delhi")
```

Accessing Dictionary Values

Values are accessed using keys.

```
print(student["name"])  
print(student["age"])
```

Using a key that does not exist will cause an error.

Using `get()`

The `get()` method safely retrieves a value.

```
print(student.get("city"))  
print(student.get("email"))
```

If the key does not exist, `None` is returned instead of an error.

Adding and Updating Values

You can add new key value pairs or update existing ones.

```
student["email"] = "test@gmail.com"  
student["age"] = 26
```

Removing Dictionary Items

pop()

Removes a key and returns its value.

```
age = student.pop("age")
```

popitem()

Removes and returns the last inserted key value pair.

```
student.popitem()
```

del

Deletes a key value pair.

```
del student["city"]
```

clear()

Removes all items from the dictionary.

```
student.clear()
```

Dictionary Keys, Values, and Items

keys()

Returns all keys.

```
student.keys()
```

values()

Returns all values.

```
student.values()
```

items()

Returns key value pairs as tuples.

```
student.items()
```

Looping Through a Dictionary

Loop through keys

```
for key in student:  
    print(key)
```

Loop through values

```
for value in student.values():  
    print(value)
```

Loop through key value pairs

```
for key, value in student.items():  
    print(key, value)
```

Checking for Keys

Use the `in` keyword to check if a key exists.

```
print("name" in student)
```

Copying Dictionaries

copy()

Creates a shallow copy of the dictionary.

```
new_student = student.copy()
```

Updating Dictionaries

update()

Adds multiple key value pairs at once.

```
student.update({  
    "course": "Python",  
    "level": "Beginner"  
})
```

Nested Dictionaries

Dictionaries can contain other dictionaries.

```
user = {  
    "id": 1,  
    "profile": {  
        "name": "Harry",  
        "email": "test@gmail.com"  
    }  
}
```

Accessing nested values:

```
print(user["profile"]["email"])
```

Dictionary Characteristics

- Keys must be unique
- Keys must be immutable
- Values can be of any data type
- Dictionaries are mutable



Conditional statements are used to make decisions in a program. They allow the program to execute different blocks of code based on conditions.

Python uses `if`, `elif`, and `else` for conditional logic.

The if Statement

The `if` statement checks a condition. If the condition is true, the code inside the block runs.

```
age = 18

if age >= 18:
    print("Eligible")
```

The else Statement

The `else` block runs when the `if` condition is false.

```
age = 16

if age >= 18:
    print("Eligible")
else:
    print("Not eligible")
```

The elif Statement

`elif` stands for else if. It is used to check multiple conditions.

```
score = 75

if score >= 90:
    print("Grade A")
elif score >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Indentation in Conditionals

Python uses indentation to define code blocks.

```
if True:
    print("This runs")
    print("This also runs")
```

Incorrect indentation will cause an error.

Comparison Operators in Conditions

Conditions often use comparison operators.

```
x = 10

if x == 10:
    print("Equal")

if x != 5:
    print("Not equal")
```

Logical Operators in Conditions

Logical operators allow combining multiple conditions.

```
age = 25
country = "India"

if age >= 18 and country == "India":
    print("Allowed")
```

Nested Conditionals

Conditionals can be placed inside other conditionals.




```
age = 20

if age >= 18:
    if age < 60:
        print("Adult")
```

Using pass in Conditionals

The `pass` statement is used when a condition is required syntactically but no action is needed.

```
age = 15

if age >= 18:
    pass
else:
    print("Minor")
```

Conditional Expressions (Ternary Operator)

Python supports one line conditionals.

```
age = 20
status = "Adult" if age >= 18 else "Minor"
```

Common Mistakes

- Forgetting indentation
- Using `=` instead of `==` in conditions
- Writing overly complex nested conditions



Loops are used to execute a block of code multiple times. They help reduce repetition and make programs more efficient.

Python mainly provides two types of loops: - `for` loop - `while` loop

The for Loop

The `for` loop is used to iterate over a sequence.

```
items = ["apple", "banana", "orange"]

for item in items:
    print(item)
```

The loop runs once for each element in the sequence.

Using range() with for Loop

The `range()` function generates a sequence of numbers.

```
for i in range(5):
    print(i)
```

This will print numbers from `0` to `4`.

range() with Start and Step

```
for i in range(1, 10, 2):
    print(i)
```

This prints numbers from `1` to `9` with a step of `2`.

Looping Through a String

Strings can also be iterated character by character.

```
text = "Python"

for char in text:
    print(char)
```

The while Loop

The `while` loop runs as long as a condition remains true.

```
count = 1

while count ≤ 5:
    print(count)
    count += 1
```

Infinite Loop

If the condition never becomes false, the loop runs forever.

```
while True:
    print("Running")
```

Use infinite loops carefully.

Loop Control Statements

Python provides special statements to control loop execution.

`break`

Stops the loop completely.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

`continue`

Skips the current iteration and moves to the next one.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

`pass`

Acts as a placeholder where a statement is required.

```
for i in range(5):  
    pass
```

Nested Loops

A loop inside another loop is called a nested loop.

```
for i in range(3):  
    for j in range(2):  
        print(i, j)
```

Looping with else

The `else` block runs when the loop finishes normally.

```
for i in range(3):  
    print(i)  
else:  
    print("Loop finished")
```

The `else` block does not run if the loop is terminated using `break`.



Functions are used to group reusable code into a single block. They help make programs organized, readable, and easier to maintain.

Instead of writing the same code again and again, you can define a function and reuse it whenever needed.

Defining a Function

Functions are defined using the `def` keyword.

```
def greet():  
    print("Hello")
```

Calling a Function

To execute a function, you call it using its name followed by parentheses.

```
greet()
```

Functions with Parameters

Parameters allow you to pass data into a function.

```
def greet(name):  
    print("Hello", name)  
  
greet("Harry")
```

Here, `name` is a parameter and `"Harry"` is an argument.

Functions with Multiple Parameters

```
def add(a, b):  
    print(a + b)  
  
add(5, 3)
```

Return Statement

The `return` statement sends a value back from the function.

```
def add(a, b):  
    return a + b  
  
result = add(10, 5)
```

Once `return` is executed, the function stops running.

Default Parameter Values

You can provide default values to parameters.

```
def greet(name="User"):  
    print("Hello", name)  
  
greet()  
greet("Harry")
```

Keyword Arguments

Arguments can be passed using parameter names.

```
def user_info(name, age):  
    print(name, age)  
  
user_info(age=25, name="Harry")
```

Variable Length Arguments

`*args`

Used to pass multiple positional arguments.

```
def total(*args):  
    print(args)  
  
total(1, 2, 3, 4)
```

`**kwargs`

Used to pass multiple keyword arguments.

```
def user_details(**kwargs):  
    print(kwargs)  
  
user_details(name="Harry", age=25)
```

Function Docstrings

Docstrings are used to describe what a function does.

```
def add(a, b):  
    """Returns the sum of two numbers"""  
    return a + b
```

Scope of Variables

Variables defined inside a function are local to that function.

```
def test():  
    x = 10  
    print(x)
```

Variables defined outside functions are global.

Function Inside a Function

Functions can be defined inside other functions.

```
def outer():  
    def inner():  
        print("Inside inner function")  
    inner()
```

Lambda Functions

Lambda functions are small anonymous functions written in one line.

```
add = lambda a, b: a + b  
print(add(3, 5))
```


Variables in Python have a scope.

Scope defines where a variable can be accessed in the code.

The two most common scopes are local and global.

Local Variables

Local variables are created inside a function.

They can be used only within that function.

```
def show_value():  
    x = 10  
    print(x)  
  
show_value()
```

Here, `x` is a local variable.

If you try to access it outside the function, it will cause an error.

```
print(x)
```

Global Variables

Global variables are created outside all functions. They can be accessed from anywhere in the program.

```
x = 20  
  
def show_value():  
    print(x)  
  
show_value()
```

Here, `x` is a global variable.

Local vs Global Example

```
x = 10

def test():
    x = 5
    print(x)

test()
print(x)
```

Output:

- Inside function: 5
- Outside function: 10

The local variable does not affect the global variable.

The global Keyword

The `global` keyword is used to modify a global variable inside a function.

```
x = 10

def update():
    global x
    x = 20

update()
print(x)
```

Without the `global` keyword, Python treats `x` as a local variable.

Why global Should Be Used Carefully

Using global variables can make code harder to understand and debug.

It is generally better to:

- Pass values as parameters
- Return values from functions

Global Variables and Function Parameters

A better approach than using `global` is passing data explicitly.

```
def update(x):  
    return x + 10  
  
value = 5  
value = update(value)
```

Variable Scope Rules

- Local variables exist only inside functions
- Global variables exist throughout the program
- Local variables override global variables with the same name
- The `global` keyword allows modifying global variables



File input and output allows a program to read data from files and write data to files. Python provides built in functions and methods to work with files easily.

Opening a File

Files are opened using the `open()` function.

```
file = open("data.txt", "r")
```

The first argument is the file name. The second argument is the file mode.

File Modes

Commonly used file modes:

Mode	Description
<code>r</code>	Read mode
<code>w</code>	Write mode
<code>a</code>	Append mode
<code>x</code>	Create file
<code>rb</code>	Read binary
<code>wb</code>	Write binary

Reading a File

`read()`

Reads the entire file content as a string.

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

`readline()`

Reads one line at a time.

```
file = open("data.txt", "r")
line = file.readline()
print(line)
file.close()
```

`readlines()`

Reads all lines and returns them as a list.

```
file = open("data.txt", "r")
lines = file.readlines()
file.close()
```

Writing to a File

Write Mode (`w`)

Creates a new file or overwrites an existing file.

```
file = open("data.txt", "w")
file.write("Hello Python")
file.close()
```

Append Mode (`a`)

Adds content to the end of a file.

```
file = open("data.txt", "a")
file.write("\nNew line added")
file.close()
```

Using the with Statement

The `with` statement automatically closes the file.

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

This is the recommended way to work with files.

Writing Multiple Lines

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]

with open("data.txt", "w") as file:
    file.writelines(lines)
```

Checking File Existence

```
import os

if os.path.exists("data.txt"):
    print("File exists")
```

Reading and Writing Binary Files

```
with open("image.png", "rb") as file:
    data = file.read()
```

```
with open("copy.png", "wb") as file:
    file.write(data)
```



Errors are situations where a program cannot run as expected.

Error handling allows a program to deal with such situations without crashing.

Python uses `try`, `except`, `else`, and `finally` to handle errors.

What is an Error?

An error occurs when Python encounters a problem while executing code.

Common examples: - Dividing by zero - Accessing a file that does not exist - Converting invalid data types

The try and except Block

Code that may cause an error is placed inside the `try` block.

If an error occurs, the code inside `except` runs.

```
try:
    x = int("abc")
except:
    print("An error occurred")
```

The program continues running instead of stopping.

Handling Specific Errors

You can catch specific error types.

```
try:
    x = int("abc")
except ValueError:
    print("Invalid conversion")
```

This helps in handling errors more precisely.

Multiple except Blocks

Different errors can be handled separately.


```
try:
    x = int(input("Enter a number: "))
    y = 10 / x
except ValueError:
    print("Please enter a valid number")
except ZeroDivisionError:
    print("Division by zero is not allowed")
```

The else Block

The `else` block runs if no error occurs.

```
try:
    x = int("10")
except ValueError:
    print("Error")
else:
    print("Conversion successful")
```

The finally Block

The `finally` block always runs, whether an error occurs or not.

```
try:
    file = open("data.txt", "r")
except FileNotFoundError:
    print("File not found")
finally:
    print("Execution finished")
```

Raising Errors Manually

You can raise an error using the `raise` keyword.

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative")
```

Using `pass` in Error Handling

The `pass` statement can be used when you want to ignore an error.

```
try:
    x = int("abc")
except ValueError:
    pass
```

This is generally discouraged unless intentional.

Common Error Types

Some common built in exceptions:

- `ValueError`
- `TypeError`
- `ZeroDivisionError`
- `FileNotFoundError`
- `IndexError`
- `KeyError`

Why Error Handling is Important

- Prevents program crashes
- Makes programs more reliable
- Helps handle unexpected inputs
- Improves user experience



The **Snake–Water–Gun** game is a beginner-friendly project that strengthens logic-building skills and introduces real-world programming flow in Python

Objective

The goal of this mini project is to build a simple command-line game using Python where the user plays **Snake–Water–Gun** against the computer. This project helps reinforce basic Python concepts in a fun and practical way.

Game Rules

- Snake drinks Water → **Snake wins**
- Water disables Gun → **Water wins**
- Gun kills Snake → **Gun wins**
- Same choice by both → **Draw**

Program Explanation

1. The computer randomly selects **Snake**, **Water**, or **Gun** using the `random` module.
2. The user inputs their choice (`s` , `w` , or `g`).
3. A function compares both choices and returns:
4. `True` → User wins
5. `False` → User loses
6. `None` → Draw
7. The result is displayed using **f-strings** for better readability.

```
import random

def game_win(user, computer):
    if user == computer:
        return None
    # Snake vs Water
    if user == "s" and computer == "w":
        return True
    if user == "w" and computer == "s":
        return False
    # Water vs Gun
    if user == "w" and computer == "g":
        return True
    if user == "g" and computer == "w":
        return False
    # Gun vs Snake
    if user == "g" and computer == "s":
        return True
    if user == "s" and computer == "g":
        return False

rand_no = random.randint(1, 3)
print("Computer's turn: Snake(s), Water (w), Gun (g)")
if rand_no == 1:
    computer = "s"
elif rand_no == 2:
    computer = "w"
else:
    computer = "g"
user = input("Your turn: Snake(s), Water (w), Gun (g): ").lower()
result = game_win(user, computer)
# Returns True if you win, False for lose, None for draw
print(f"\nYou chose: {user}")
print(f"\nComputer chose: {computer}")
if result is None:
    print("Its a draw!")

elif(result):
    print("You win!")
else:
    print("You lose!")
```

AI has changed the way software is written.

Developers no longer rely only on manual coding and documentation.

Modern software development often combines human logic with AI assistance.

ChatGPT and Conversational Coding

ChatGPT allows developers to write code using natural language.

You can: - Ask for code examples - Get explanations for existing code - Debug errors by pasting code and error messages - Convert logic written in English into working code

ChatGPT works best as a thinking and explanation tool.

IDE Integrated AI Development

Tools like GitHub Copilot are directly integrated into code editors.

They: - Suggest code while you type - Complete functions automatically - Understand patterns within your codebase - Speed up repetitive coding tasks

These tools work in real time inside the IDE.

Context Aware Agents and Chat to Code

Modern AI systems can understand project context.

They: - Read multiple files - Understand project structure - Modify existing code safely - Generate code that fits into an ongoing project

Chat to code allows developers to describe changes and let AI implement them across files.

AI Powered Terminals

AI enhanced terminals allow developers to: - Convert plain English commands into terminal commands - Fix command line errors - Explain unfamiliar commands - Automate workflows using natural language

This reduces friction while working with systems and deployments.

Important Note on Using AI for Code

AI can generate code quickly, but it does not guarantee correctness.

1. Read the generated code carefully.
2. Understand the logic before using it.
3. Test the output to verify it works as expected.
4. Modify and improve the code when necessary.

AI should assist development, not replace understanding.

Never use AI-generated code blindly.

CodeWithHarry



What is ChatGPT?

ChatGPT is an AI model that can understand natural language and generate human like responses, including code, explanations, and solutions to problems.

ChatGPT vs Other AI Tools

In my personal opinion, ChatGPT is overall better than tools like Gemini and Claude.

Claude Code is very good specifically for coding tasks, but ChatGPT feels more balanced and reliable across coding, explanation, reasoning, and learning use cases.

This comparison is purely my personal opinion based on 4 Years usage experience.

ChatGPT Plans

At the time of writing this, the ChatGPT Go plan is available for free.

This makes it easy for anyone to start using premium features of ChatGPT without any upfront cost.

Generating Python Code with ChatGPT

ChatGPT can generate Python code for a wide range of tasks.

For example, you can ask it to build: - A simple calculator - A GUI based calculator using Tkinter - Small automation scripts - Data processing programs

A basic prompt like:

Create a calculator GUI using Tkinter in Python

can already generate working code.

Understanding AI Limitations

As your problem becomes more complex, unique, or loosely defined, it becomes harder for AI to generate exactly what you expect.

However, even in such cases, AI often surprises you with: - Well structured code - Logical approaches - Useful starting points

The key is to guide the AI with clear prompts and gradually refine the output.

Final Advice

ChatGPT is a powerful development assistant, not a replacement for thinking.

When used correctly, ChatGPT can significantly improve your productivity.

CodeWithHarry



GitHub Copilot is an AI coding assistant developed by GitHub in collaboration with OpenAI. It works directly inside supported code editors and provides intelligent code suggestions while you type.

Instead of writing every line manually, developers can describe what they want to build, and Copilot suggests functions, loops, classes, comments, and even complete files.

Copilot supports dozens of programming languages including Python, JavaScript, C++, Java, HTML, CSS, SQL, and many others. The better your code and comments, the better its suggestions become.

What Can GitHub Copilot Do?

CODE

Generates code snippets and complete functions from simple instructions.

COMPLETE

Predicts and completes the next lines of code as you type.

DEBUG

Suggests fixes for common errors and improves existing code.

EXPLAIN

Helps understand unfamiliar code by generating explanations.

How Copilot Helps Developers

Copilot is designed to reduce repetitive work rather than replace developers. It is especially useful for writing boilerplate code, generating test cases, creating documentation, and learning new APIs.

It can also suggest better implementations, improve readability, and help explore multiple approaches to solving the same programming problem.

Things to Remember

1. Review every suggestion before accepting it.
2. Test generated code using real inputs.
3. Understand the logic instead of memorizing it.
4. Modify the output according to your project requirements.
5. Never assume AI-generated code is always correct or secure.

GitHub Copilot is most effective when used as a programming partner that assists your thinking rather than replacing your problem-solving skills.

This project organizes files in a directory into folders based on file extensions using Python.

It scans the current folder, creates category folders like PDFs, Images, and Word files, and moves files into the correct folders automatically.

Code

```
import os
import shutil

FOLDER_PATH = os.getcwd()

FILE_TYPES = {
    "PDFs": [".pdf"],
    "Word_Files": [".docx", ".doc"],
    "Images": [".jpg", ".jpeg", ".png"],
    "Videos": [".mp4", ".mkv"],
    "Text_Files": [".txt"]
}

for folder in FILE_TYPES:
    if not os.path.exists(folder):
        os.mkdir(folder)

for file in os.listdir(FOLDER_PATH):
    if os.path.isdir(file):
        continue

    ext = os.path.splitext(file)[1].lower()

    for folder, extensions in FILE_TYPES.items():
        if ext in extensions:
            shutil.move(file, os.path.join(folder, file))
            break
```

This project is a command line typing speed tester built using Python.

It displays a random sentence, measures typing time, calculates words per minute, and checks accuracy.

Code

```
import time
import random

sentences = [
    "The quick brown fox jumps over the lazy dog.",
    "A journey of a thousand miles begins with a single step.",
    "This is the way for us to reference the object of the class."
]

def measure_accuracy(user_input, test_sentence):
    correct_chars = sum(1 for a, b in zip(user_input, test_sentence) if a == b)
    accuracy = (correct_chars / len(test_sentence)) * 100 if test_sentence else 0
    return accuracy

def typing_test():
    test_sentence = random.choice(sentences)
    print("Type the following sentence as fast as you can:")
    print(test_sentence)
    input("Press Enter when you are ready...")
    start_time = time.time()
    user_input = input("\nStart typing:\n")
    end_time = time.time()
    time_taken = end_time - start_time
    word_count = len(test_sentence.split(" "))

    print("Results:")
    print(f"Time taken: {time_taken} seconds")
    print(f"Words typed: {word_count}")
    print(f"Typing speed: {word_count / (time_taken / 60):.2f} words per minute")
    accuracy = measure_accuracy(user_input, test_sentence)
    print(f"Accuracy: {accuracy:.2f}%")

typing_test()
```

This project is a simple command line quiz application built using Python.

It displays multiple choice questions, takes user input, checks answers, and shows the final score.

Code

```
def run_quiz():
    questions = [
        {
            "question": "What is the capital of France?",
            "options": ["A) Berlin", "B) Madrid", "C) Paris", "D) Rome"],
            "answer": "C) Paris"
        },
        {
            "question": "What is 2 + 2?",
            "options": ["A) 3", "B) 4", "C) 5", "D) 6"],
            "answer": "B) 4"
        },
        {
            "question": "What is the largest planet in our solar system?",
            "options": ["A) Earth", "B) Jupiter", "C) Saturn", "D) Mars"],
            "answer": "B) Jupiter"
        },
        {
            "question": "Who wrote 'Hamlet'?",
            "options": ["A) Charles Dickens", "B) Mark Twain",
                       "C) William Shakespeare", "D) Jane Austen"],
            "answer": "C) William Shakespeare"
        }
    ]

    score = 0

    for index, q in enumerate(questions):
        print(f"Q{index + 1}: {q['question']}")
        for option in q['options']:
            print(option)

        user_answer = input("Your answer(A/B/C/D): ")
        if user_answer.strip().upper() == q['answer'][0]:
            print("Correct!\n")
            score += 1
```

```
print(f"Your final score is {score}/{len(questions)}")

run_quiz()
```

CodeWithHarry



This project is a simple GUI based PDF merger built using Python and Tkinter. It allows users to select multiple PDF files, merge them, and save the output as a single PDF.

Code

```
import tkinter as tk
from tkinter import filedialog, messagebox
from pypdf import PdfWriter
import os

def select_pdfs():
    files = filedialog.askopenfilenames(
        title="Select PDF files to merge",
        filetypes=[("PDF files", "*.pdf")]
    )
    if files:
        pdf_list.delete(0, tk.END)
        for file in files:
            pdf_list.insert(tk.END, file)

def merge_pdfs():
    pdfs = list(pdf_list.get(0, tk.END))
    if not pdfs:
        messagebox.showerror("Error", "No PDF files selected!")
        return

    merger = PdfWriter()
    try:
        for pdf in pdfs:
            merger.append(pdf)

        output_file = filedialog.asksaveasfilename(
            defaultextension=".pdf",
            filetypes=[("PDF files", "*.pdf")],
            title="Save merged PDF as"
        )
        if output_file:
            merger.write(output_file)
            merger.close()
            messagebox.showinfo(
                "Success",
                f"PDFs merged successfully into {os.path.basename(output_file)}"
            )
```



```
)  
except Exception as e:  
    messagebox.showerror("Error", f"An error occurred: {str(e)}")  
  
root = tk.Tk()  
root.title("PDF Merger")  
root.geometry("400x300")  
  
select_button = tk.Button(root, text="Select PDF Files", command=select_pdfs)  
select_button.pack(pady=10)  
  
pdf_list = tk.Listbox(root, selectmode=tk.MULTIPLE, width=50, height=10)  
pdf_list.pack(pady=10)  
  
merge_button = tk.Button(root, text="Merge PDFs", command=merge_pdfs)  
merge_button.pack(pady=10)  
  
root.mainloop()
```



This project is a Python based water drinking reminder that sends system notifications at regular intervals using the plyer library.

Code

```
import time
from plyer import notification

def water_reminder(interval_minutes):
    interval_seconds = interval_minutes * 60

    print("Water Drinking Reminder Started")
    print(f"You will get a notification every {interval_minutes} minutes\n")

    while True:
        time.sleep(interval_seconds)
        notification.notify(
            title="Drink Water",
            message="Time to drink water and stay hydrated.",
            timeout=10
        )

water_reminder(30)
```

This project is a simple command line password manager built using Python. It stores passwords in a text file and copies the password to the clipboard when requested without displaying it on screen.

Code

```
import pyperclip
import os

FILE_NAME = "passwords.txt"

def save_password():
    website = input("Enter website: ")
    password = input("Enter password: ")
    with open(FILE_NAME, "a") as f:
        f.write(f"{website} < > {password}\n")

def get_password():
    website = input("Enter website: ")
    with open(FILE_NAME, "r") as f:
        for line in f:
            if website in line:
                password = line.strip().split("< >")[1]
                pyperclip.copy(password)
                print("Password copied to clipboard!")
                break
    else:
        print("Website not found.")

def main():
    while True:
        print("1. Save Password")
        print("2. Get Password")
        print("3. Exit")
        choice = input("Choose an option: ")

        if choice == '1':
            save_password()
        elif choice == '2':
            get_password()
        elif choice == '3':
```

```
        print("Exiting...")
        break
    else:
        print("Invalid choice. Please try again.")

main()
```



Section 4: SQL for Data Analytics

Chapter 1: Introduction to Databases & SQL

When working with large amounts of data, databases are essential. Databases are used to store, manage, and retrieve data efficiently.

Why Not Use Excel or Text Files to Store Data?

At a small scale, Excel sheets or text files look convenient. But as data grows, serious problems start to appear.

Limitations of Excel

- Hard to manage large datasets efficiently
- Very slow when data grows into lakhs or millions of rows
- No proper multi user access control
- High chances of accidental data modification
- Poor support for complex queries and automation

Limitations of Text Files

- No structure or enforced data types
- Searching and filtering is slow
- No relationships between data
- No security or access control
- Data consistency is impossible to guarantee

These tools are fine for temporary or personal use, not for real applications or analytics.

Why Do We Need Databases?

A database is designed to store, manage, and retrieve large amounts of data efficiently and safely.

Databases solve problems that files and spreadsheets cannot.

What Databases Provide

- Fast data access even with millions of records
- Structured data with rules and constraints
- Data consistency and accuracy
- Secure access with permissions
- Multiple users working at the same time
- Easy data analysis using queries

For analytics, databases are essential because real world data is always large, messy, and constantly changing.

What Is a Database?

A **database** is an organized collection of data stored electronically so it can be easily accessed, managed, and updated.

Think of a database as: - A smart storage system - That understands your data - And allows you to ask questions about it efficiently

Example: Instead of scrolling through files, you can directly ask: - Give me total sales for last month - Find top 5 customers by revenue - Show users who signed up but never purchased

Databases make this possible in seconds.

Where SQL Fits In

SQL stands for Structured Query Language.

It is the language used to: - Store data - Retrieve data - Update data - Analyze data

SQL is the bridge between you and the database.

In this course, we will use **MySQL**, one of the most widely used SQL databases in the industry.

What You Should Remember From This Lecture

- Excel and text files do not scale
- Real applications and analytics need databases
- A database is structured, fast, secure, and reliable
- SQL is the language used to work with databases
- MySQL will be our tool to learn SQL practically



Before learning MySQL, it is important to understand the two major types of databases.

Relational Databases

Relational databases store data in the form of **tables**.

Each table has: - Rows called records - Columns called fields - A fixed structure (schema)

Tables can be connected to each other using relationships.

Example

- A `users` table stores user details
- An `orders` table stores purchase details
- Both tables are linked using a common column like `user_id`

This structure makes data organized and reliable.

Key Characteristics

- Data is stored in tables
- Fixed schema
- Relationships between tables
- Strong data consistency
- SQL is used to query data

Popular Relational Databases

- MySQL
- PostgreSQL
- SQL Server
- Oracle

For **data analytics**, relational databases are the most commonly used.

Non Relational Databases

Non relational databases do not use tables.

They store data in flexible formats like: - Key value pairs - Documents (JSON) - Graphs - Columns

There is no fixed structure. Each record can look different.

Example

A user document may contain: - Name - Email - Preferences - Nested objects

Another user document may have completely different fields.

Key Characteristics

- Flexible schema
- No fixed table structure
- Scales very well horizontally
- Designed for large scale applications
- Querying is not standardized like SQL

Popular Non Relational Databases

- MongoDB
- Redis
- Cassandra
- Neo4j

Relational vs Non Relational (Quick Comparison)

Feature	Relational	Non Relational
Data format	Tables	JSON, key value, graph
Schema	Fixed	Flexible
Relationships	Supported	Limited or manual
Consistency	Strong	Often eventual
Query language	SQL	Database specific
Analytics friendly	Yes	Not ideal

Which One Should You Learn First?

For **data analytics**, reporting, and business intelligence: - Relational databases are the standard - SQL is mandatory - MySQL is a perfect starting point

Non relational databases are more useful for: - Real time apps - High scale systems - Unstructured or rapidly changing data

We will be installing **MySQL Server (database engine)** first, and then **MySQL Workbench (GUI tool)** on a Windows system.

1. Install MySQL Server (Database Engine)

We will use the **official MySQL Installer**, which correctly installs and configures the MySQL Server.

Step 1: Download MySQL Server Installer

1. Open the official MySQL Installer page: <https://dev.mysql.com/downloads/installer/>
2. Download **mysql-installer-web-community** (recommended).

The file name will look like:

```
mysql-installer-web-community-8.0.xx.0.msi
```

1. If asked to log in, click **"No thanks, just start my download."**
-

Step 2: Run the Installer

1. Double-click the downloaded `.msi` file.
 2. If Windows asks for permission, click **Yes**.
-

Step 3: Choose Setup Type

1. When the installer opens, select **Server only**.
2. Click **Next**.

This installs **only MySQL Server**, without Workbench or extra tools.

Step 4: Install Required Components

1. If any required components are missing, the installer will show them.
 2. Click **Execute** to install them.
 3. After completion, click **Next**.
-

Step 5: Install MySQL Server

1. Review the selected products.
2. Click **Execute**.
3. Wait until the installation completes successfully.
4. Click **Next**.

Step 6: Configure MySQL Server

6.1 Server Type

- Select **Development Computer**
- Click **Next**

6.2 Connectivity & Port

- Keep the default port **3306**
- Click **Next**

6.3 Authentication Method

- Select **Use Strong Password Encryption**
- Click **Next**

6.4 Set Root Password

- Set a password for the **root** user
- Confirm the password
- Save it securely
- Do not add extra users
- Click **Next**

6.5 Windows Service

- Service Name: **MySQL80**
- Enable **Start MySQL Server at System Startup**
- Click **Next**

6.6 Apply Configuration

- Click **Execute**
- Wait for all steps to complete
- Click **Finish**

Step 7: Finish MySQL Server Installation

1. Click **Next**
2. Click **Finish**

At this point, **MySQL Server** is installed and running.

2. Install MySQL Workbench (GUI Tool)

Now we install **MySQL Workbench**, which is used to visually manage databases and run SQL queries.

Step 1: Download MySQL Workbench

1. Visit the MySQL Workbench download page: <https://dev.mysql.com/downloads/workbench/>
2. Select **Windows (x86, 64-bit)**.
3. Download the installer.
4. Click **"No thanks, just start my download."** if prompted.

Step 2: Install MySQL Workbench

1. Run the downloaded installer.
2. Click **Next** and keep default options.
3. Click **Install**.
4. Once installation completes, click **Finish**.

Step 3: Open MySQL Workbench

1. Open **MySQL Workbench** from the Start Menu.
2. On the home screen, under **MySQL Connections**, click: **Local instance MySQL80**
3. Enter the **root password** you created earlier.
4. Click **OK**

The SQL Editor screen will open successfully.

You have now **successfully installed MySQL Server and MySQL Workbench on Windows**.



In this lesson, we will write our very first SQL commands and understand what they do.

Creating a Database

```
CREATE DATABASE ecom;
```

This command creates a new database named `ecom`. A database is a structured place where we store our tables and data.

Using a Database

```
USE ecom;
```

This tells MySQL that all the queries we run now should apply to the `ecom` database.

If you do not use a database, MySQL will not know where to create tables or store data.

Saving and Opening SQL Scripts in MySQL Workbench

Saving a SQL Script

1. Write your queries in the SQL editor.

Click File → Save Script.

2. Save the file with a `.sql` extension Example: `first_queries.sql`

Opening a SQL Script

Click File → Open SQL Script.

1. Select your `.sql` file.
2. The queries will open in a new SQL editor tab.



So far we learnt that Relational databases use **SQL** to store data in tables, while non relational databases do **not use SQL** and store data in flexible formats like documents or key-value pairs.

CRUD Operations

C

Create

Add new records using `INSERT` .

R

Read

Retrieve data using `SELECT` .

U

Update

Modify existing data using `UPDATE` .

D

Delete

Remove records using `DELETE` .

Resetting the Database

```
DROP DATABASE IF EXISTS ecom;
```

Deletes the database if it already exists so we can start fresh.

Creating and Using the Database

```
CREATE DATABASE ecom;  
USE ecom;
```

Creates a new database named `ecom` and sets it as the active database.

Creating the orders Table

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_name VARCHAR(100),  
  city VARCHAR(50),  
  product VARCHAR(100),  
  category VARCHAR(50),  
  quantity INT,  
  price_per_unit DECIMAL(10,2),  
  discount_percent INT,
```



```
    order_date DATE,  
    delivery_date DATE,  
    payment_mode VARCHAR(30),  
    order_status VARCHAR(30),  
    rating INT  
);
```

This table stores order related data like customer details, product info, dates, payments, and ratings.

Inserting Data into orders

```
INSERT INTO orders  
(customer_name, city, product, category, quantity, price_per_unit,  
discount_percent, order_date, delivery_date, payment_mode,  
order_status, rating)  
VALUES  
(  
'Amit Sharma','Delhi','Laptop','Electronics',  
1,65000,10,  
'2025-01-05','2025-01-08',  
'Credit Card','Delivered',5),  
  
(  
'Neha Verma','Mumbai','Headphones','Electronics',  
2,2500,0,  
'2025-01-10','2025-01-12',  
'UPI','Delivered',4),  
  
(  
'Rahul Khan','Delhi','Office Chair','Furniture',  
1,12000,15,  
'2025-01-12','2025-01-20',  
'Debit Card','Delivered',5),  
  
(  
'Priya Singh','Bangalore','Notebook','Stationery',  
10,80,0,  
'2025-01-15','2025-01-16',  
'Cash','Delivered',3),  
  
(  
'Arjun Mehta','Ahmedabad','Smartphone','Electronics',  
1,30000,5,  
'2025-01-18',NULL,  
'UPI','Cancelled',NULL),
```



```
('Sara Ali','Delhi','Table Lamp','Home Decor',  
2,1500,20,  
'2025-01-20','2025-01-23',  
'Credit Card','Delivered',4),  
  
('Rohit Gupta','Mumbai','Water Bottle','Kitchen',  
5,500,0,  
'2025-01-22','2025-01-24',  
'Cash','Delivered',2),  
  
('Kavita Joshi','Pune','Backpack','Accessories',  
1,3500,10,  
'2025-01-25','2025-01-29',  
'Debit Card','Delivered',5),  
  
('Mohammed Faisal','Hyderabad','Keyboard','Electronics',  
1,1800,0,  
'2025-01-28','2025-02-01',  
'UPI','Delivered',4),  
  
('Ananya Roy','Kolkata','Study Table','Furniture',  
1,15000,25,  
'2025-02-01',NULL,  
'Credit Card','Pending',NULL),  
  
('Vikram Patel','Surat','Mixer Grinder','Appliances',  
1,4200,5,  
'2025-02-03','2025-02-06',  
'UPI','Delivered',4),  
  
('Pooja Nair','Chennai','Yoga Mat','Fitness',  
2,1200,0,  
'2025-02-05','2025-02-07',  
'Cash','Delivered',5);
```

This adds sample data so we can practice real SQL queries.

Viewing the Data

```
SELECT * FROM orders;
```

A table is where we store data inside a database. Each table has **columns**, and every column has a **data type**.

Example Table: customers

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  email VARCHAR(150),  
  age INT,  
  phone VARCHAR(15),  
  is_active BOOLEAN,  
  signup_date DATE,  
  created_at DATETIME,  
  total_spent DECIMAL(10,2)  
);
```

This table stores basic customer information.

Commonly Used MySQL Data Types

- **INT** Stores whole numbers Example: age, quantity, ids
- **VARCHAR(n)** Stores text with a fixed maximum length Example: name, email, city
- **DECIMAL(p, s)** Stores precise decimal numbers Example: price, salary, total_spent
- **BOOLEAN** Stores true or false values Example: is_active
- **DATE** Stores only date Example: signup_date
- **DATETIME** Stores date and time together Example: created_at
- **ENUM('val1', 'val2', ...)** Stores one value from a predefined list Example: status ('active', 'inactive', 'pending')

Removing the Table

After learning, we can delete the table.

```
DROP TABLE customers;
```

This removes the table completely from the database.

Renaming the Table

```
ALTER TABLE customers RENAME TO clients;
```

CodeWithHarry



Once a table is created, we can add data to it using the `INSERT` statement. Each row represents one record in the table.

Example Table: customers

Assume this table already exists.

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  email VARCHAR(150),  
  age INT,  
  phone VARCHAR(15),  
  is_active BOOLEAN,  
  signup_date DATE,  
  created_at DATETIME,  
  total_spent DECIMAL(10,2)  
);
```

Inserting a Single Row

```
INSERT INTO customers  
(name, email, age, phone,  
is_active, signup_date, created_at, total_spent)  
VALUES  
(  
'Amit Sharma', 'amit@gmail.com', 28, '9876543210',  
TRUE, '2025-01-10',  
'2025-01-10 10:30:00', 12000.50);
```

This adds one customer record to the table.

Inserting Multiple Rows

```
INSERT INTO customers
(name, email, age, phone,
is_active, signup_date, created_at, total_spent)
VALUES
('Neha Verma', 'neha@gmail.com', 25, '9123456789',
TRUE, '2025-01-12',
'2025-01-12 09:15:00', 5400.00),

('Rahul Khan', 'rahul@gmail.com', 32, '9988776655',
FALSE, '2025-01-15',
'2025-01-15 14:20:00', 0.00);
```

This inserts multiple rows in a single query.

Viewing Inserted Data

```
SELECT * FROM customers;
```

This shows all rows stored in the table.



To read data from a table, we use the `SELECT` statement.

Selecting All Columns

```
SELECT * FROM orders;
```

This returns all rows and all columns from the `orders` table.

Selecting Specific Columns

```
SELECT customer_name, product, city  
FROM orders;
```

This returns only the selected columns.

Filtering Rows Using WHERE

```
SELECT *  
FROM orders  
WHERE city = 'Delhi';
```

This returns orders placed from Delhi.

Using Conditions

```
SELECT customer_name, product, price_per_unit  
FROM orders  
WHERE price_per_unit > 5000;
```

This returns expensive products.

Here is another example using not equal to:

```
SELECT customer_name, product, price_per_unit  
FROM orders  
WHERE price_per_unit != 5000;
```

In SQL, we use `IS NULL` instead of `= NULL` because `NULL` represents an unknown or missing value, not an actual value, and any comparison using operators like `=`, `!=`, `<`, or `>` with `NULL` always results in `UNKNOWN` rather than `TRUE` or `FALSE`. Since a `WHERE` clause only selects rows where the condition evaluates to `TRUE`, expressions like `column = NULL` or `column != NULL` never return any rows. `IS NULL` and `IS NOT NULL` are special SQL conditions designed specifically to check whether a column has no value, avoiding this ambiguity and correctly identifying missing data.

```
SELECT *  
FROM orders  
WHERE delivery_date IS NULL;
```

Using AND and OR

```
SELECT *  
FROM orders  
WHERE city = 'Delhi' AND order_status = 'Delivered';
```

This returns delivered orders from Delhi.

Sorting Data

```
SELECT customer_name, order_date, price_per_unit  
FROM orders  
ORDER BY order_date DESC;
```

This sorts orders by latest first.



Updating Rows

To modify existing data in a table, we use the `UPDATE` statement. Always use `WHERE` to avoid updating all rows by mistake.

Updating a Single Row

```
UPDATE orders
SET order_status = 'Delivered'
WHERE order_id = 10;
```

This updates the status of the order with `order_id = 10`.

Updating Multiple Columns

```
UPDATE orders
SET discount_percent = 10,
    rating = 4
WHERE customer_name = 'Neha Verma';
```

This updates multiple columns for the matching row.

Updating Multiple Rows

```
UPDATE orders
SET order_status = 'Cancelled'
WHERE order_status = 'Pending';
```

This updates all pending orders to cancelled.

Updating Using a Condition

```
UPDATE orders
SET discount_percent = 20
WHERE category = 'Electronics' AND price_per_unit > 30000;
```

This applies a discount to expensive electronics.

Always Check Before Updating

```
SELECT *  
FROM orders  
WHERE order_status = 'Pending';
```

Run a `SELECT` first to confirm which rows will be updated.

Deleting Rows

To remove data from a table, we use the `DELETE` statement. Always use `WHERE` to avoid deleting all rows by mistake.

Deleting a Single Row

```
DELETE FROM orders  
WHERE order_id = 5;
```

This deletes the order with `order_id = 5`.

Deleting Multiple Rows

```
DELETE FROM orders  
WHERE order_status = 'Cancelled';
```

This deletes all cancelled orders.

Deleting Using a Condition

```
DELETE FROM orders  
WHERE order_date < '2025-01-10';
```

This deletes old orders placed before a specific date.

Always Check Before Deleting

```
SELECT *  
FROM orders  
WHERE order_status = 'Cancelled';
```

Run a `SELECT` first to confirm which rows will be deleted.

Deleting All Rows (Use Carefully)

```
DELETE FROM orders;
```

This removes all rows but keeps the table structure.

CodeWithHarry



In SQL, we use `ALTER` to **change a table structure** and `DROP` to **remove a table completely**.

Altering a Table

`ALTER TABLE` modifies the table structure **without deleting existing data**.

Adding a New Column

```
ALTER TABLE orders
ADD COLUMN delivery_partner VARCHAR(50);
```

Adds a new column to the `orders` table. All existing rows remain unchanged.

Modifying a Column

```
ALTER TABLE orders
MODIFY price_per_unit DECIMAL(12,2);
```

Changes the data type or size of a column. Data stays as long as it fits the new definition.

Renaming a Column

```
ALTER TABLE orders
RENAME COLUMN city TO customer_city;
```

Renames a column while keeping all data.

Dropping a Column

```
ALTER TABLE orders
DROP COLUMN rating;
```

Deletes **only the column**, not the table.

Deleting Table Data vs Table Structure

Delete All Rows but Keep Structure

```
DELETE FROM orders;
```

- Removes **all data**
- Table structure stays
- Can be rolled back in transactions

Dropping a Table

Delete Everything Including Structure

```
DROP TABLE orders;
```

- Deletes **table data**
- Deletes **table structure**
- Table no longer exists
- Cannot be rolled back

Drop Table Only If It Exists

```
DROP TABLE IF EXISTS orders;
```

Prevents errors if the table does not exist.

Quick Summary

- `ALTER TABLE` Changes structure, data stays
- `DELETE FROM table` Deletes data, structure stays
- `DROP TABLE` Deletes data and structure completely



A transaction is a group of SQL statements that are executed **as one unit**. Either **all changes are saved**, or **none of them are**.

Turning Auto Commit OFF

By default, MySQL saves every change immediately (auto commit ON). Let's turn it OFF using:

```
SET autocommit = 0;
```

This tells MySQL **not to save changes automatically**.

From now on, changes will be saved only when we explicitly commit them.

What Is Auto Commit

When `autocommit` is ON (default):

- Every `INSERT`, `UPDATE`, or `DELETE` is saved immediately
- You cannot undo changes

When `autocommit` is OFF:

- Changes are temporary
 - You can choose to save or undo them
-

Example Transaction on orders

Step 1: Update Data

```
UPDATE orders
SET order_status = 'Cancelled'
WHERE order_id = 3;
```

The change is **not permanent yet**.

Step 2: Check the Change

```
SELECT order_id, order_status
FROM orders
WHERE order_id = 3;
```

You will see the updated value.

Committing a Transaction

```
COMMIT;
```

- Saves all changes permanently
- Changes cannot be undone after this

Rolling Back a Transaction

```
ROLLBACK;
```

- Cancels all changes made after the last commit
- Restores the data to its previous state

Transaction Flow Summary

```
SET autocommit = 0  
UPDATE / INSERT / DELETE  
COMMIT → save changes  
ROLLBACK → undo changes
```

Important Notes

- Transactions work only with **transactional engines** like InnoDB
- `SELECT` statements do not need commit or rollback
- Always use transactions when working with critical data



Constraints are rules applied to table columns to **control what data can be stored**. They help maintain **data accuracy and integrity**.

Commonly Used Constraints

UNIQUE

Ensures that all values in a column are different. No duplicate values are allowed.

NOT NULL

Ensures that a column cannot store `NULL` values.

CHECK

Applies a condition that every value in the column must satisfy.

DEFAULT

Automatically assigns a value if no value is provided.

Creating Table with Constraints

Step 1: Create `employees` Table

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    email VARCHAR(150) UNIQUE,  
    name VARCHAR(100) NOT NULL,  
    age INT CHECK (age ≥ 18),  
    department VARCHAR(50) DEFAULT 'General',  
    salary DECIMAL(10,2) CHECK (salary > 0),  
    joining_date DATE DEFAULT (CURRENT_DATE)  
);
```

This table demonstrates multiple constraints together.

UNIQUE Constraint Example

```
INSERT INTO employees (email, name, age, salary)  
VALUES ('amit@company.com', 'Amit Sharma', 25, 45000);
```

Trying to insert the same email again will fail.

NOT NULL Constraint Example

```
INSERT INTO employees (email, age, salary)
VALUES ('neha@company.com', 24, 40000);
```

This will fail because `name` cannot be NULL.

CHECK Constraint Example

```
INSERT INTO employees (email, name, age, salary)
VALUES ('rahul@company.com', 'Rahul Khan', 16, 30000);
```

This will fail because age must be 18 or above.

DEFAULT Constraint Example

```
INSERT INTO employees (email, name, age, salary)
VALUES ('pooja@company.com', 'Pooja Nair', 26, 50000);
```

- `department` will be set to `General`
- `joining_date` will be set to today's date

Viewing the Data

```
SELECT * FROM employees;
```



A **PRIMARY KEY** uniquely identifies each row in a table. It **cannot be NULL** and **cannot have duplicate values**.

`AUTO_INCREMENT` automatically generates the next unique number for a column.

Example Table: employees

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    email VARCHAR(150) UNIQUE,  
    name VARCHAR(100) NOT NULL  
);
```

- `emp_id` uniquely identifies each employee
- MySQL automatically assigns values like 1, 2, 3, and so on

Inserting Data Without emp_id

```
INSERT INTO employees (email, name)  
VALUES ('amit@company.com', 'Amit Sharma');
```

MySQL automatically assigns `emp_id = 1`.

Viewing the Data

```
SELECT * FROM employees;
```

Trying to Add Another Primary Key (This Will Fail)

```
ALTER TABLE employees  
ADD PRIMARY KEY (email);
```

This query will **fail** because:

- A table can have **only one PRIMARY KEY**
- `emp_id` is already defined as the primary key

MySQL will throw an error saying the table already has a primary key.

Why Primary Key Is Important

- Ensures row uniqueness
- Makes searching faster
- Required for table relationships

CodeWithHarry



`ORDER BY` is used to **sort data**, and `LIMIT` is used to **control how many rows are returned**.

Sorting Data Using ORDER BY

Sort by Price (Low to High)

```
SELECT *  
FROM orders  
ORDER BY price_per_unit;
```

Sorts orders in ascending order by default.

Sort by Price (High to Low)

```
SELECT *  
FROM orders  
ORDER BY price_per_unit DESC;
```

Sorts orders from highest to lowest price.

Sort by Date

```
SELECT order_id, customer_name, order_date  
FROM orders  
ORDER BY order_date DESC;
```

Shows the latest orders first.

Sorting by Multiple Columns

```
SELECT *  
FROM orders  
ORDER BY city ASC, price_per_unit DESC;
```

First sorts by city, then by price within each city.



Limiting the Number of Rows

```
SELECT *  
FROM orders  
LIMIT 5;
```

Returns only the first 5 rows.

Using ORDER BY with LIMIT

```
SELECT customer_name, product, price_per_unit  
FROM orders  
ORDER BY price_per_unit DESC  
LIMIT 3;
```

Returns the top 3 most expensive orders.

OFFSET with LIMIT

```
SELECT *  
FROM orders  
ORDER BY order_date  
LIMIT 5 OFFSET 5;
```

Skips the first 5 rows and returns the next 5.

Key Points

- `ORDER BY` controls sorting
- `LIMIT` controls how many rows are shown
- Always use `ORDER BY` with `LIMIT` for predictable results



Functions are built-in operations that **perform calculations or transformations on data**. They are commonly used inside `SELECT` queries.

Aggregate Functions

These work on **multiple rows** and return a **single value**.

COUNT

```
SELECT COUNT(*)  
FROM orders;
```

Returns the total number of orders.

SUM

```
SELECT SUM(quantity * price_per_unit) AS total_revenue  
FROM orders;
```

Calculates total revenue.

AVG

```
SELECT AVG(price_per_unit)  
FROM orders;
```

Returns the average product price.

MIN and MAX

```
SELECT MIN(price_per_unit), MAX(price_per_unit)  
FROM orders;
```

Finds the cheapest and most expensive products.

Scalar Functions

These work on **each row individually**.

ROUND


```
SELECT customer_name, ROUND(price_per_unit, 0)
FROM orders;
```

Rounds the price to the nearest whole number.

UPPER and LOWER

```
SELECT UPPER(customer_name), LOWER(city)
FROM orders;
```

Converts text to uppercase or lowercase.

LENGTH

```
SELECT customer_name, LENGTH(customer_name)
FROM orders;
```

Returns the number of characters in a string.

Date Functions

CURRENT_DATE

```
SELECT CURRENT_DATE;
```

Returns today's date.

DATEDIFF

```
SELECT order_id, DATEDIFF(delivery_date, order_date) AS delivery_days
FROM orders;
```

Calculates delivery time in days.

Using Functions with WHERE

```
SELECT *
FROM orders
WHERE YEAR(order_date) = 2025;
```

Filters orders placed in 2025.

Key Points

- Aggregate functions summarize data
- Scalar functions modify individual values
- Functions are often combined with `WHERE`, `ORDER BY`, and `GROUP BY`

CodeWithHarry



Logical operators help us **filter data based on conditions**.

Using IN

IN is used to match a value against a **list of values**.

```
SELECT *  
FROM orders  
WHERE city IN ('Delhi', 'Mumbai', 'Bangalore');
```

This returns orders from only the listed cities.

NOT IN

```
SELECT *  
FROM orders  
WHERE payment_mode NOT IN ('Cash', 'UPI');
```

Returns orders where payment was not done using Cash or UPI.

Using BETWEEN and NOT BETWEEN

BETWEEN checks for values within a range and includes both ends.

```
SELECT *  
FROM orders  
WHERE price_per_unit BETWEEN 1000 AND 10000;
```

Returns orders with price between 1000 and 10000.

NOT BETWEEN

```
SELECT *  
FROM orders  
WHERE price_per_unit NOT BETWEEN 1000 AND 10000;
```

Returns orders outside the given price range.

Using LIKE with Wildcards

LIKE is used for **pattern matching** in text.

% Wildcard

Matches **any number of characters**.

```
SELECT *  
FROM orders  
WHERE customer_name LIKE 'A%';
```

Returns customers whose name starts with A.

```
SELECT *  
FROM orders  
WHERE product LIKE '%Table%';
```

Returns products containing the word "Table".

_ Wildcard

Matches **exactly one character**.

```
SELECT *  
FROM orders  
WHERE city LIKE 'D_lhi';
```

Matches `Delhi` but not `Dehli` or `Dilli`.

Combining Logical Conditions

```
SELECT *  
FROM orders  
WHERE category IN ('Electronics', 'Furniture')  
AND price_per_unit NOT BETWEEN 5000 AND 20000;
```

Filters data using multiple logical rules.



Key Points

- `IN` replaces multiple `OR` conditions
- `NOT BETWEEN` filters values outside a range
- `%` matches many characters
- `_` matches exactly one character

CodeWithHarry



A **FOREIGN KEY** connects two tables and ensures that the relationship between them is valid. It prevents orders from referencing sellers that do not exist.

Existing Table: orders

The `orders` table already exists and stores order details.

Now we want each order to be associated with a **seller**.

Step 1: Create sellers Table

```
CREATE TABLE sellers (  
    seller_id INT PRIMARY KEY AUTO_INCREMENT,  
    seller_name VARCHAR(100) UNIQUE NOT NULL,  
    city VARCHAR(50)  
);
```

This table stores unique seller information.

Step 2: Add Seller Column to orders

```
ALTER TABLE orders  
ADD COLUMN seller_id INT;
```

This column will store the seller for each order.

Step 3: Create Foreign Key Relationship

```
ALTER TABLE orders  
ADD CONSTRAINT fk_orders_seller  
FOREIGN KEY (seller_id)  
REFERENCES sellers(seller_id);
```

This ensures:

- Every `seller_id` in `orders` must exist in `sellers`
- Invalid seller references are not allowed

You can also create the foreign key while creating the `orders` table:

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_name VARCHAR(100),  
  city VARCHAR(50),  
  product VARCHAR(100),  
  category VARCHAR(50),  
  quantity INT,  
  price_per_unit DECIMAL(10,2),  
  discount_percent INT,  
  order_date DATE,  
  delivery_date DATE,  
  payment_mode VARCHAR(30),  
  order_status VARCHAR(30),  
  rating INT,  
  seller_id INT,  
  CONSTRAINT fk_orders_seller  
  FOREIGN KEY (seller_id)  
  REFERENCES sellers(seller_id)  
);
```

Example: Valid Data Insert

```
INSERT INTO sellers (seller_name, city)  
VALUES ('TechWorld', 'Delhi');
```

```
INSERT INTO orders (seller_id, product, quantity, price_per_unit)  
VALUES (1, 'Laptop', 1, 65000);
```

This works because seller `1` exists.

Example: Invalid Insert (Will Fail)

```
INSERT INTO orders (seller_id, product, quantity, price_per_unit)  
VALUES (999, 'Laptop', 1, 65000);
```

This fails because seller `999` does not exist.

Failed Data Deletion Due to Foreign Key

```
DELETE FROM sellers  
WHERE seller_id = 1;
```

This fails if there are orders linked to seller `1`.

Why Foreign Keys Matter

- Enforces data integrity
- Prevents invalid relationships
- Makes data consistent
- Essential for relational databases



When tables are related using a **foreign key**, `ON DELETE` defines **what happens to child rows** when a row in the parent table is deleted.

Here:

- `sellers` is the **parent table**
- `orders` is the **child table**
- `seller_id` connects them

Relationship Between Tables

- One seller can have many orders
- Each order belongs to one seller

Adding Foreign Key Without ON DELETE

```
ALTER TABLE orders
ADD CONSTRAINT fk_orders_seller
FOREIGN KEY (seller_id)
REFERENCES sellers(seller_id);
```

By default, MySQL uses `RESTRICT`.

ON DELETE CASCADE

Deletes all related orders when a seller is deleted.

```
ALTER TABLE orders
DROP FOREIGN KEY fk_orders_seller;

ALTER TABLE orders
ADD CONSTRAINT fk_orders_seller
FOREIGN KEY (seller_id)
REFERENCES sellers(seller_id)
ON DELETE CASCADE;
```

What Happens

```
DELETE FROM sellers
WHERE seller_id = 3;
```

- Seller with `seller_id = 3` is deleted
- All orders linked to seller 3 are deleted automatically

ON DELETE SET NULL

Keeps orders but removes the seller reference.

`seller_id` in `orders` must allow NULL for this to work.

```
ALTER TABLE orders
MODIFY seller_id INT NULL;
```

```
ALTER TABLE orders
DROP FOREIGN KEY fk_orders_seller;
```

```
ALTER TABLE orders
ADD CONSTRAINT fk_orders_seller
FOREIGN KEY (seller_id)
REFERENCES sellers(seller_id)
ON DELETE SET NULL;
```

What Happens

- Seller is deleted
- Orders remain
- `seller_id` in `orders` becomes `NULL`

ON DELETE RESTRICT (Default)

Prevents deletion if orders exist.

```
ALTER TABLE orders
DROP FOREIGN KEY fk_orders_seller;
```

```
ALTER TABLE orders
ADD CONSTRAINT fk_orders_seller
FOREIGN KEY (seller_id)
REFERENCES sellers(seller_id)
ON DELETE RESTRICT;
```

What Happens

```
DELETE FROM sellers
WHERE seller_id = 5;
```

- MySQL throws an error
- Seller is not deleted

Quick Comparison

- **CASCADE** Deletes orders automatically
- **SET NULL** Keeps orders, removes seller link
- **RESTRICT** Blocks seller deletion

Key Rule

Choose **ON DELETE** based on **business logic**:

Marketplace shutdown → **CASCADE**

- Seller leaves platform, orders kept → **SET NULL**

Prevent accidental deletion → **RESTRICT**



JOINS are used to **combine data from multiple tables** based on a related column.

Here, both tables are connected using `seller_id`.

Tables Involved

- `orders` Contains order details and `seller_id`
- `sellers` Contains seller information

INNER JOIN

Returns only rows where matching data exists in **both tables**.

```
SELECT
    o.order_id,
    o.product,
    o.city AS customer_city,
    s.seller_name
FROM orders o
INNER JOIN sellers s
ON o.seller_id = s.seller_id;
```

Only orders that have a valid seller are shown.

LEFT JOIN

Returns **all orders**, even if the seller is missing.

```
SELECT
    o.order_id,
    o.product,
    s.seller_name
FROM orders o
LEFT JOIN sellers s
ON o.seller_id = s.seller_id;
```

Orders without a seller will show `NULL` for seller details.

RIGHT JOIN

Returns **all sellers**, even if they have no orders.

```
SELECT
    s.seller_name,
    o.order_id,
    o.product
FROM orders o
RIGHT JOIN sellers s
ON o.seller_id = s.seller_id;
```

Sellers with no orders will show `NULL` for order columns.

JOIN with WHERE Condition

```
SELECT
    o.order_id,
    o.product,
    s.seller_name
FROM orders o
JOIN sellers s
ON o.seller_id = s.seller_id
WHERE o.order_status = 'Delivered';
```

Filters only delivered orders.

Key Points to Remember

- `INNER JOIN` returns matching rows only
- `LEFT JOIN` keeps all left table rows
- `RIGHT JOIN` keeps all right table rows
- JOINS use **foreign key relationships**



Use indexes **only when a table is read more than it is updated**. Indexes make `SELECT` queries faster but **slow down** `INSERT`, `UPDATE`, and `DELETE` operations.

What Is an Index

An index is a data structure that helps MySQL **find rows quickly** without scanning the entire table.

Think of it like an index page in a book.

Creating an Index

Index on a Frequently Searched Column

```
CREATE INDEX idx_orders_city
ON orders (city);
```

This speeds up queries that filter by city.

Using the Index

```
SELECT *
FROM orders
WHERE city = 'Delhi';
```

MySQL can use the index to locate rows faster.

Composite Index

An index can be created on multiple columns.

```
CREATE INDEX idx_orders_city_status
ON orders (city, order_status);
```

Useful when queries filter by both city and order status.

When NOT to Use Indexes

- Columns with very few unique values
- Tables with frequent updates
- Small tables



Indexes are not free, they trade **write performance** for **read speed**.

Removing an Index

```
DROP INDEX idx_orders_city ON orders;
```

Deletes the index but keeps the table data.

Key Points

- Indexes speed up `SELECT`
- Indexes slow down `INSERT`, `UPDATE`, `DELETE`
- Create indexes only on frequently searched columns



A **view** is a **saved SQL query** that behaves like a virtual table. It does not store data, it only stores the query.

Views are used to **simplify queries**, **hide complexity**, and **control data access**.

Creating a View

```
CREATE VIEW delivered_orders AS
SELECT order_id,
       customer_name,
       city,
       product,
       price_per_unit,
       order_date
FROM orders
WHERE order_status = 'Delivered';
```

This view shows only delivered orders.

Using a View

```
SELECT *
FROM delivered_orders;
```

You can query a view just like a table.

Why Use Views

- Reuse complex queries
- Improve readability
- Restrict access to sensitive columns
- Keep business logic in one place

Updating Data Through a View

```
UPDATE delivered_orders
SET price_per_unit = price_per_unit + 500
WHERE order_id = 1;
```

If the view is simple, updates affect the original table.

Dropping a View

```
DROP VIEW delivered_orders;
```

Deletes the view, not the table.

Key Difference

- View: Virtual, stores query only
- Table: Physical, stores actual data

CodeWithHarry



A **subquery** is a query written **inside another query**. It is used when one query depends on the result of another.

Subquery in WHERE

```
SELECT *
FROM orders
WHERE price_per_unit > (
    SELECT AVG(price_per_unit)
    FROM orders
);
```

This returns orders that are priced above the average price.

Subquery with IN

```
SELECT *
FROM orders
WHERE city IN (
    SELECT city
    FROM orders
    WHERE category = 'Electronics'
);
```

This returns orders from cities where electronics were sold.

Subquery in SELECT

```
SELECT order_id,
       customer_name,
       price_per_unit,
       (SELECT AVG(price_per_unit) FROM orders) AS avg_price
FROM orders;
```

Adds the average price as a column in every row.



Subquery with EXISTS

```
SELECT *  
FROM orders o  
WHERE EXISTS (  
    SELECT 1  
    FROM orders  
    WHERE city = o.city  
    AND category = 'Furniture'  
);
```

Returns orders from cities where furniture orders exist.

Key Points

- Subqueries run first
- Can be used in `WHERE`, `SELECT`, and `FROM`
- Useful when queries depend on other query results



`GROUP BY` is used to **group rows that have the same values** and perform calculations on each group.

It is commonly used with **aggregate functions**.

GROUP BY with COUNT

```
SELECT city, COUNT(*) AS total_orders
FROM orders
GROUP BY city;
```

Returns the number of orders from each city.

GROUP BY with SUM

```
SELECT category,
       SUM(quantity * price_per_unit) AS total_sales
FROM orders
GROUP BY category;
```

Calculates total sales for each category.

GROUP BY with AVG

```
SELECT city, AVG(price_per_unit) AS avg_price
FROM orders
GROUP BY city;
```

Returns the average price per city.

GROUP BY with Multiple Columns

```
SELECT city, order_status, COUNT(*) AS count
FROM orders
GROUP BY city, order_status;
```

Groups data by both city and order status.

GROUP BY Rules

- Every selected column must be either
- in `GROUP BY`, or
- inside an aggregate function
- `GROUP BY` reduces multiple rows into one row per group

GROUP BY with ORDER BY

```
SELECT category, COUNT(*) AS total_orders
FROM orders
GROUP BY category
ORDER BY total_orders DESC;
```

Sorts groups based on aggregated values.

HAVING Clause

`HAVING` is used to filter groups after aggregation.

```
SELECT city, COUNT(*) AS total_orders
FROM orders
GROUP BY city
HAVING COUNT(*) > 5;
```

Returns cities with more than 5 orders.



UNION is used to **combine results of multiple SELECT queries** into a single result set.

Rules to remember:

- Same number of columns
- Same order of columns
- Compatible data types

Existing Table: orders

We already have customer orders in the `orders` table.

Now we will create another table that stores **orders placed by employees**.

Creating employee_orders Table

```
CREATE TABLE employee_orders (  
    order_id INT,  
    name VARCHAR(100),  
    city VARCHAR(50),  
    product VARCHAR(100),  
    price_per_unit DECIMAL(10,2)  
);
```

This table has a similar structure so it can be combined with `orders`.

Inserting Data into employee_orders

```
INSERT INTO employee_orders  
VALUES  
(201, 'Ravi Kumar', 'Delhi', 'Laptop', 62000),  
(202, 'Sneha Patel', 'Mumbai', 'Headphones', 2300),  
(203, 'Ankit Verma', 'Pune', 'Office Chair', 11000),  
(204, 'Ananya Roy', 'Kolkata', 'Study Table', 15000);
```

Using UNION

```
SELECT order_id, customer_name AS name, city, product, price_per_unit
FROM orders
```

UNION

```
SELECT order_id, name, city, product, price_per_unit
FROM employee_orders;
```

This combines:

- Customer orders
- Employee orders into one result set.

Duplicate rows are removed automatically.

Using UNION ALL

```
SELECT order_id, customer_name AS name, city, product, price_per_unit
FROM orders
```

UNION ALL

```
SELECT order_id, name, city, product, price_per_unit
FROM employee_orders;
```

This keeps **all rows**, including duplicates.

Key Difference

- **UNION** Removes duplicates
- **UNION ALL** Keeps duplicates and is faster

When to Use UNION

- Combining similar data from multiple tables
- Creating combined reports
- Merging historical or temporary data



ROLLUP is used with GROUP BY to generate **summary rows** like subtotals and a grand total.

It is mainly used in **reporting and analytics**.

Basic ROLLUP Example

```
SELECT city,  
       category,  
       SUM(quantity * price_per_unit) AS total_sales  
FROM orders  
GROUP BY city, category WITH ROLLUP;
```

This query produces:

- Sales per city and category
- Subtotal for each city
- One grand total row at the end

How to Read ROLLUP Results

- (city, category) Normal grouped data
- (city, NULL) Total sales for that city
- (NULL, NULL) Grand total of all sales

ROLLUP with Single Column

```
SELECT category,  
       COUNT(*) AS total_orders  
FROM orders  
GROUP BY category WITH ROLLUP;
```

This gives:

- Order count per category
- Overall total orders

Identifying Total Rows

```
SELECT
    city,
    category,
    SUM(quantity * price_per_unit) AS total_sales
FROM orders
GROUP BY city, category WITH ROLLUP;
```

Rows containing `NULL` values represent **summary totals**, not missing data.

When to Use ROLLUP

- Business reports
- Sales summaries
- Dashboard totals
- Financial aggregation

Key Points

- `ROLLUP` adds subtotals automatically
- Order of columns in `GROUP BY` matters
- Supported natively in MySQL



A **stored procedure** is a **saved block of SQL statements** that runs as a single unit. It helps reuse logic, reduce repetition, and keep business logic inside the database.

Why We Change the Delimiter

By default, MySQL uses `;` to end a statement.

A stored procedure contains **multiple SQL statements**, each ending with `;`. So MySQL gets confused about **where the procedure ends**.

To fix this, we temporarily change the delimiter.

Changing the Delimiter

```
DELIMITER //
```

Now MySQL will treat `//` as the end of the procedure.

Creating a Stored Procedure

Example: Get Delivered Orders

```
CREATE PROCEDURE get_delivered_orders()  
BEGIN  
    SELECT *  
    FROM orders  
    WHERE order_status = 'Delivered';  
END //
```

The procedure ends with `//`, not `;`.

Resetting the Delimiter

```
DELIMITER ;
```

Always reset the delimiter after creating the procedure.

Calling a Stored Procedure

```
CALL get_delivered_orders();
```

This executes the stored procedure.

Stored Procedure with Parameter

```
DELIMITER //
```

```
CREATE PROCEDURE get_orders_by_city(IN city_name VARCHAR(50))  
BEGIN  
    SELECT *  
    FROM orders  
    WHERE city = city_name;  
END //
```

```
DELIMITER ;
```

Calling the Procedure with Parameter

```
CALL get_orders_by_city('Delhi');
```

Why Use Stored Procedures

- Reuse SQL logic
- Improve maintainability
- Reduce network traffic
- Centralize business rules

Key Points

- Stored procedures are reusable SQL blocks
- Delimiter is changed to avoid confusion
- Always reset the delimiter after use



A **trigger** is a piece of SQL that **automatically runs when a specific event happens** on a table.

Triggers are commonly used for:

- Logging changes
- Enforcing rules
- Maintaining audit history

Practical Scenario

Whenever an order is **cancelled**, we want to **store a record** of that cancellation.

This should happen automatically, without writing extra queries.

Step 1: Create a Log Table

```
CREATE TABLE order_cancellations (  
    log_id INT PRIMARY KEY AUTO_INCREMENT,  
    order_id INT,  
    cancelled_on DATETIME,  
    reason VARCHAR(100)  
);
```

This table stores cancellation history.

Step 2: Change the Delimiter

```
DELIMITER //
```

We change the delimiter because a trigger contains multiple SQL statements.



Step 3: Create the Trigger

```
CREATE TRIGGER trg_log_order_cancel
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    IF NEW.order_status = 'Cancelled'
    AND OLD.order_status <> 'Cancelled' THEN

        INSERT INTO order_cancellations
        (order_id, cancelled_on, reason)
        VALUES
        (NEW.order_id, NOW(), 'Order cancelled by user');

    END IF;
END //
```

Step 4: Reset the Delimiter

```
DELIMITER ;
```

How the Trigger Works

- Runs **after an update** on `orders`
- Checks if the order status changed to `Cancelled`
- Automatically inserts a log record

Testing the Trigger

```
UPDATE orders
SET order_status = 'Cancelled'
WHERE order_id = 2;
```

Now check the log table:

```
SELECT * FROM order_cancellations;
```

A cancellation record is automatically created.

Other Commonly Used Triggers

- BEFORE INSERT
- AFTER INSERT
- BEFORE UPDATE
- BEFORE DELETE
- AFTER DELETE

Why Triggers Are Powerful

- No manual logging needed
- Logic runs automatically
- Prevents missed business rules
- Keeps database consistent

Important Notes

- Triggers run automatically
- Cannot be called manually
- Use carefully, they add hidden logic



MySQL Workbench allows you to manage and modify database data **without writing SQL**, using its graphical interface. This is especially useful for beginners, quick fixes, and real-world admin tasks.

Editing Data from SELECT Query Results

How it works

1. Right-click a table
2. Click **Select Rows – Limit 1000**
3. Edit any cell directly in the result grid

Apply to Save Changes

- After editing a value, click **Apply**
- MySQL Workbench shows the generated `UPDATE` query
- Click **Apply** again to commit changes

Revert to Undo Changes

- If you made a mistake **before applying**, click **Revert**
- This restores the original values from the database

Note: Edits work reliably only if the table has a **Primary Key**.

Form Editor (Row-by-Row Editing)

The **Form Editor** lets you edit one row at a time in a structured layout.

How to open

- Click the **Form Editor icon** in the result grid toolbar

When to use

- Tables with many columns
- Long text fields
- Avoiding accidental edits

Safer than inline grid editing for beginners

Export Table Data as CSV

Steps

1. Right-click the table
2. Choose **Table Data Export Wizard**
3. Select **CSV**
4. Choose file location

[Click Next](#) → [Finish](#)

Use cases:

- Data sharing
- Backups
- Excel analysis

Import Data from CSV (GUI)

Steps

1. Right-click the table
2. Select **Table Data Import Wizard**
3. Choose the CSV file
4. Map columns carefully
5. Review preview
6. Click **Import**

Common mistakes:

- Wrong date format
- Column mismatch
- Header row not handled correctly

Key Takeaway

MySQL Workbench GUI is powerful for **learning, debugging, and admin work**, but understanding what happens **behind the scenes** is still important.

AI tools can help you **write SQL faster**, **understand queries**, and **explore data** more efficiently. They are assistants, not replacements for understanding SQL.

How AI Helps with SQL

AI can:

- Convert plain English to SQL
- Generate `SELECT`, `JOIN`, `GROUP BY`, and subqueries
- Explain complex queries
- Help debug errors

Example: English to SQL

Prompt to AI:

Show all delivered orders from Delhi sorted by latest order date

Generated SQL:

```
SELECT *  
FROM orders  
WHERE city = 'Delhi'  
      AND order_status = 'Delivered'  
ORDER BY order_date DESC;
```

Example: Analytics Query

Prompt to AI:

Find total sales per category

```
SELECT category,  
       SUM(quantity * price_per_unit) AS total_sales  
FROM orders  
GROUP BY category;
```

Example: Fixing a Query

Prompt to AI:

This query gives an error, fix it

AI can explain:

- Missing `GROUP BY`
- Wrong column names
- Logical mistakes

Best Way to Use AI

- Use AI to **speed up writing**
- Always **read and understand** the query
- Verify results with `SELECT`
- Never blindly trust generated SQL in production

What AI Cannot Replace

- Understanding table structure
- Knowing business logic
- Knowing when a query is correct
- Database design decisions

Key Takeaway

AI is a **SQL copilot**, not a shortcut to skip learning. The better your SQL basics, the better AI helps you.



HarryShop is a merchandise store for coders selling hoodies, mugs, stickers, notebooks, and accessories.

This project demonstrates:

- Database design
- Table relationships
- Foreign keys
- Realistic demo data
- Business-style SQL analysis

1. Create Database

```
CREATE DATABASE harryshop;  
USE harryshop;
```

2. Database Schema

Customers Table

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  email VARCHAR(150) UNIQUE,  
  city VARCHAR(50),  
  signup_date DATE  
);
```

Products Table

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY AUTO_INCREMENT,  
  product_name VARCHAR(100),  
  category VARCHAR(50),  
  price DECIMAL(10,2),  
  stock INT  
);
```

Orders Table


```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_id INT,  
  order_date DATE,  
  order_status VARCHAR(30),  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

Order Items Table

```
CREATE TABLE order_items (  
  order_item_id INT PRIMARY KEY AUTO_INCREMENT,  
  order_id INT,  
  product_id INT,  
  quantity INT,  
  FOREIGN KEY (order_id) REFERENCES orders(order_id),  
  FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

Payments Table

```
CREATE TABLE payments (  
  payment_id INT PRIMARY KEY AUTO_INCREMENT,  
  order_id INT,  
  payment_mode VARCHAR(30),  
  amount DECIMAL(10,2),  
  payment_date DATE,  
  FOREIGN KEY (order_id) REFERENCES orders(order_id)  
);
```

3. Insert Demo Data

Customers (20 rows)

```
INSERT INTO customers (name, email, city, signup_date) VALUES  
(  
'Amit Sharma','amit@gmail.com','Delhi','2025-01-01'),  
(  
'Neha Verma','neha@gmail.com','Mumbai','2025-01-02'),  
(  
'Rahul Khan','rahul@gmail.com','Bangalore','2025-01-03'),  
(  
'Pooja Nair','pooja@gmail.com','Chennai','2025-01-04'),
```

```
('Rohit Gupta','rohit@gmail.com','Delhi','2025-01-05'),
('Ananya Roy','ananya@gmail.com','Kolkata','2025-01-06'),
('Karan Mehta','karan@gmail.com','Ahmedabad','2025-01-07'),
('Simran Kaur','simran@gmail.com','Chandigarh','2025-01-08'),
('Mohit Jain','mohit@gmail.com','Jaipur','2025-01-09'),
('Sneha Patel','sneha@gmail.com','Surat','2025-01-10'),
('Vikram Singh','vikram@gmail.com','Lucknow','2025-01-11'),
('Alok Mishra','alok@gmail.com','Patna','2025-01-12'),
('Nidhi Agarwal','nidhi@gmail.com','Noida','2025-01-13'),
('Saurabh Verma','saurabh@gmail.com','Ghaziabad','2025-01-14'),
('Riya Sen','riya@gmail.com','Pune','2025-01-15'),
('Aditya Malhotra','aditya@gmail.com','Delhi','2025-01-16'),
('Kritika Shah','kritika@gmail.com','Mumbai','2025-01-17'),
('Yash Tiwari','yash@gmail.com','Kanpur','2025-01-18'),
('Mehul Joshi','mehul@gmail.com','Vadodara','2025-01-19'),
('Isha Kapoor','isha@gmail.com','Gurgaon','2025-01-20');
```

Products (20 rows)

```
INSERT INTO products (product_name, category, price, stock) VALUES
('Python Hoodie','Clothing',1999,50),
('Java Hoodie','Clothing',1899,40),
('Debugging Mug','Accessories',599,100),
('Code Like a Pro Mug','Accessories',649,80),
('DSA Notebook','Stationery',499,150),
('SQL Cheat Sheet','Stationery',299,200),
('Sticker Pack','Stationery',249,300),
('Algorithm T-Shirt','Clothing',1499,60),
('GitHub Cap','Accessories',799,70),
('Keyboard Mat','Accessories',999,90),
('Linux Hoodie','Clothing',2099,35),
('AI Nerd T-Shirt','Clothing',1599,55),
('Whiteboard Notebook','Stationery',699,120),
('Bug Hunter Mug','Accessories',549,100),
('Terminal Stickers','Stationery',199,250),
('Coder Bottle','Accessories',899,110),
('Late Night Hoodie','Clothing',2199,30),
('Python Socks','Accessories',399,140),
('DSA Flash Cards','Stationery',349,180),
('Clean Code Notebook','Stationery',599,130);
```

Orders (20 rows)

```
INSERT INTO orders (customer_id, order_date, order_status) VALUES
(1, '2025-02-01', 'Delivered'),
(2, '2025-02-02', 'Delivered'),
(3, '2025-02-03', 'Delivered'),
(4, '2025-02-04', 'Cancelled'),
(5, '2025-02-05', 'Delivered'),
(6, '2025-02-06', 'Pending'),
(7, '2025-02-07', 'Delivered'),
(8, '2025-02-08', 'Delivered'),
(9, '2025-02-09', 'Delivered'),
(10, '2025-02-10', 'Cancelled'),
(11, '2025-02-11', 'Delivered'),
(12, '2025-02-12', 'Delivered'),
(13, '2025-02-13', 'Pending'),
(14, '2025-02-14', 'Delivered'),
(15, '2025-02-15', 'Delivered'),
(16, '2025-02-16', 'Delivered'),
(17, '2025-02-17', 'Cancelled'),
(18, '2025-02-18', 'Delivered'),
(19, '2025-02-19', 'Delivered'),
(20, '2025-02-20', 'Delivered');
```

Order Items (20+ rows)

```
INSERT INTO order_items (order_id, product_id, quantity) VALUES
(1, 1, 1), (1, 7, 2),
(2, 3, 1), (2, 5, 1),
(3, 8, 1),
(4, 2, 1),
(5, 6, 3),
(6, 4, 1),
(7, 10, 1),
(8, 12, 2),
(9, 15, 3),
(10, 9, 1),
(11, 11, 1),
(12, 13, 1),
(13, 14, 2),
(14, 16, 1),
```

```
(15,17,1),
(16,18,2),
(17,19,1),
(18,20,1),
(19,1,1),
(20,5,2);
```

Payments (20 rows)

```
INSERT INTO payments (order_id, payment_mode, amount, payment_date) VALUES
(1, 'UPI', 2497, '2025-02-01'),
(2, 'Credit Card', 1098, '2025-02-02'),
(3, 'UPI', 1499, '2025-02-03'),
(5, 'Debit Card', 897, '2025-02-05'),
(7, 'UPI', 999, '2025-02-07'),
(8, 'Credit Card', 3198, '2025-02-08'),
(9, 'UPI', 747, '2025-02-09'),
(11, 'UPI', 2099, '2025-02-11'),
(12, 'Debit Card', 699, '2025-02-12'),
(14, 'UPI', 899, '2025-02-14'),
(15, 'Credit Card', 2199, '2025-02-15'),
(16, 'UPI', 798, '2025-02-16'),
(18, 'Debit Card', 599, '2025-02-18'),
(19, 'UPI', 1999, '2025-02-19'),
(20, 'Credit Card', 998, '2025-02-20');
```

4. Analysis Queries

Total Revenue

```
SELECT SUM(amount) AS total_revenue
FROM payments;
```

Revenue by Product

```
SELECT p.product_name,
       SUM(oi.quantity * p.price) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
```

```
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status = 'Delivered'
GROUP BY p.product_name
ORDER BY revenue DESC;
```

Top Customers by Spend

```
SELECT c.name,
       SUM(p.amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN payments p ON o.order_id = p.order_id
GROUP BY c.name
ORDER BY total_spent DESC;
```

Best Selling Products

```
SELECT p.product_name,
       SUM(oi.quantity) AS total_sold
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_name
ORDER BY total_sold DESC;
```

Cancelled Orders Count

```
SELECT COUNT(*) AS cancelled_orders
FROM orders
WHERE order_status = 'Cancelled';
```



Section 5: Advanced Python

Chapter 1: Iterables and How Python Loops Over Data

What Problem This Solves

When working with data, you rarely deal with single values.

You deal with collections of values.

Python handles all such collections using iterations.

Iterables in One Sentence

If you can loop over something using a `for` loop, it is an iterable.

```
data = [10, 20, 30]
```

```
for x in data:  
    print(x)
```

Lists, strings, dictionaries, and files are all iterables.

What the for Loop Does

A `for` loop takes values one by one from the data.

You do not control how the values are fetched. Python does it for you.

This is why the same loop works for many data types.

Iterating Over Common Data Structures

List:

```
for x in [1, 2, 3]:  
    print(x)
```

String:

```
for ch in "data":  
    print(ch)
```

Dictionary:

```
record = {"name": "Ali", "age": 20}
```

```
for key in record:  
    print(key)
```

Generators

Sometimes data is produced one value at a time instead of all at once.

This approach:

- Saves memory
- Is useful for large data
- Is common in data workflows



When working with data, you often:

- transform values
- filter values
- create new collections from old ones

List comprehensions are a clean and readable way to do this.

Basic List Comprehension

Instead of writing a loop:

```
numbers = [1, 2, 3, 4, 5]

squared = []
for x in numbers:
    squared.append(x * x)
```

You can write:

```
squared = [x * x for x in numbers]
```

Same logic, less noise.

Filtering Data

You can add conditions inside a comprehension.

```
numbers = [1, 2, 3, 4, 5, 6]

even_numbers = [x for x in numbers if x % 2 == 0]
```

This pattern is very common in data processing.

Transforming Text Data

```
names = [" Ali ", "Sara ", " JOHN"]

clean_names = [name.strip().lower() for name in names]
```

This is how raw data is cleaned before analysis.

Dictionary Comprehensions

You can create dictionaries the same way.

```
items = ["apple", "banana", "cherry"]
prices = [0.5, 0.3, 0.2]
price_dict = {item: price for item, price in zip(items, prices)}
```

Also you can filter items like this:

```
scores = {"math": 80, "science": 90, "english": 75}

passed = {k: v for k, v in scores.items() if v >= 80}
```

Useful when:

- filtering records
- remapping keys or values

Set Comprehensions

Set comprehensions remove duplicates automatically.

```
values = [1, 2, 2, 3, 3, 4]

unique_squares = {x * x for x in values}
```

This is helpful when you only care about unique results.

Nested Comprehensions

```
pairs = [(x, y) for x in [1, 2] for y in [3, 4]]
```

Use this sparingly. If it hurts readability, use a normal loop.

When NOT to Use Comprehensions

Avoid comprehensions when:

- logic becomes hard to read
- there are multiple side effects
- debugging becomes difficult

Readability matters more than clever code.

CodeWithHarry



Sometimes data is large. Sometimes data is produced over time. Sometimes storing everything at once is unnecessary.

Generators solve this by producing values one at a time.

The Core Idea

A generator does not store all values in memory. It gives you the next value only when you ask for it.

This makes generators memory efficient.

Generator Using yield

```
def get_numbers():  
    for i in range(5):  
        yield i
```

Using it:

```
for x in get_numbers():  
    print(x)
```

Values are generated one by one, not all at once.

Generator vs List

In Python, you can create generators using parentheses `()` instead of brackets `[]`. Parenthesis are not tuple comprehensions, they create generators. List version:

```
numbers = [i for i in range(5)]
```

Generator version:

```
numbers = (i for i in range(5))
```

Difference:

- List stores all values
- Generator produces values when needed

When Generators Are Useful

Generators are useful when:

- Data is large
- Data comes from files
- Data is processed step by step
- You only need one value at a time

Example:

```
def read_lines(filepath):  
    with open(filepath) as file:  
        for line in file:  
            yield line.strip()
```

next Function

You can get the next value from a generator using `next()`.

```
gen = (x * x for x in range(3))  
print(next(gen)) # 0  
print(next(gen)) # 1  
print(next(gen)) # 4
```

Important Limitation

Generators can be used only once.

```
gen = (x for x in range(3))  
  
list(gen)  
list(gen)
```

The second result will be empty.



Key Takeaway

- Generators produce data lazily
- They save memory
- They fit well with data workflows
- Use them when you do not need all data at once

CodeWithHarry



Many data bugs in Python are not syntax errors. They happen because data changes in places you did not expect.

This usually comes from mutability.

Mutable vs Immutable Objects

IMMUTABLE

Immutable objects **cannot be changed** after they are created.

- `int`
- `float`
- `str`
- `tuple`

MUTABLE

Mutable objects **can be modified** after they are created.

- `list`
- `dict`
- `set`

The Most Common Bug

```
a = [1, 2, 3]
b = a

b.append(4)

print(a)
```

Both `a` and `b` change.

Why:

- `a` and `b` point to the same data
- No copy was created

Copying Data Correctly

Shallow copy:

```
a = [1, 2, 3]
b = a.copy()
```

Now changes to `b` do not affect `a`.

For dictionaries:

```
record = {"name": "Ali", "marks": [80, 90]}  
copy_record = record.copy()
```

Nested data can still be shared.

Hidden Mutation Inside Functions

```
def add_item(items):  
    items.append(10)  
  
data = [1, 2, 3]  
add_item(data)  
  
print(data)
```

Functions can modify mutable data passed to them.

This is expected behavior, but often forgotten.



The Problem NumPy Solves

Python lists are flexible, but they are not designed for large numerical data.

SLOW

Operations become slow on large datasets.

MEMORY

Lists consume more memory than NumPy arrays.

LOOPS

Most calculations require explicit loops.

SOLUTION

NumPy provides a faster and more efficient way to work with numbers.

What NumPy Really Is

NUMERIC ARRAYS

Stores large collections of numbers efficiently.

FIXED TYPES

Uses fixed data types for better performance.

VECTORIZED

Performs operations on entire arrays at once.

BENEFIT

Less code, better readability, and faster execution.

Python List vs NumPy Array

Python list:

```
data = [1, 2, 3, 4]
result = []

for x in data:
    result.append(x * 2)
```

NumPy array:

```
import numpy as np

data = np.array([1, 2, 3, 4])
result = data * 2
```

Same result, less code, and much faster for large datasets.

Creating NumPy Arrays

```
import numpy as np

a = np.array([10, 20, 30])
b = np.zeros(5)
c = np.ones(3)
d = np.arange(1, 6)
```

ARRAY()

Creates an array from existing values.

ZEROS()

Creates an array filled with zeros.

ONES()

Creates an array filled with ones.

ARANGE()

Creates evenly spaced numbers within a range.

Vectorized Operations

NumPy allows operations on entire arrays:

```
a = np.array([1, 2, 3])

a + 10
a * 2
a > 1
```

No explicit loops are required. These operations are fast and efficient.

Basic Useful Operations

```
a.sum()
a.mean()
a.max()
a.min()
```

SUM()

Returns the total of all values.

MEAN()

Calculates the average value.

MAX()

Returns the largest value.

MIN()

Returns the smallest value.

Two and Three Dimensional Arrays

```
matrix = np.array([[1, 2], [3, 4]])  
tensor = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

2D ARRAY

Represents data in rows and columns, similar to a table or matrix.

3D ARRAY

Represents multiple matrices stacked together, often called a tensor.



The Problem Pandas Solves

Real-world data is often large, messy, and stored in files. Managing it with only Python lists and dictionaries quickly becomes difficult.

TABULAR

Most data is organized in rows and columns.

MESSY

Missing or inconsistent values are common.

FILES

Data is usually stored in CSV, Excel, or databases.

SOLUTION

Pandas makes working with structured data simple and efficient.

What Pandas Really Is

Pandas provides a powerful data structure called a **DataFrame**.

TABLE

Looks similar to a spreadsheet.

ROWS & COLUMNS

Organizes data into rows and columns.

COLUMN NAMES

Keeps data aligned using column labels.

COLUMN OPERATIONS

Lets you work with entire columns at once.

Think of it as a spreadsheet controlled with Python code.

Creating a DataFrame

From a dictionary:

```
import pandas as pd

data = {
    "name": ["Ali", "Sara", "John"],
    "marks": [85, 90, 78]
}

df = pd.DataFrame(data)
```

KEYS

Each dictionary key becomes a column.

VALUES

Each list becomes the data for that column.

Looking at the Data

Before analyzing data, inspect it first.

```
df.head()  
df.info()  
df.describe()
```

HEAD()

Shows the first few rows.

INFO()

Displays columns, data types, and missing values.

DESCRIBE()

Generates summary statistics for numeric columns.

PURPOSE

Understand the dataset before performing analysis.

Selecting Data

Select a single column:

```
df["marks"]
```

Select multiple columns:

```
df[["name", "marks"]]
```

SINGLE COLUMN

Returns one column from the DataFrame.

MULTIPLE COLUMNS

Returns a new DataFrame containing selected columns.

Rows and Index

Every row in a DataFrame has an index. By default, it starts from 0.

DEFAULT INDEX

Automatically assigned as 0, 1, 2, ...

ALIGNMENT

Helps Pandas match rows during operations.

IDENTIFICATION

Makes accessing and updating rows easier.

BEGINNER TIP

You usually don't need to change the index initially.

Why Pandas Is Used for Data Work

LOAD DATA

Read CSV, Excel, JSON, and database files.

CLEAN DATA

Handle missing values and fix inconsistent data.

ANALYZE

Perform calculations on entire columns quickly.

ORGANIZE

Keep data readable, structured, and easy to explore.

This is why Pandas is the foundation of most data analysis workflows.



Raw data is rarely ready for analysis. Before asking questions, you must select the right data, filter what you need, and clean what is broken.

This step decides whether your results are trustworthy.

Selecting Columns

Select a single column:

```
df["price"]
```

Select multiple columns:

```
df[["price", "category", "rating"]]
```

Column selection is the most common operation in Pandas.

Selecting Rows with Conditions

Filter rows using conditions:

```
df[df["price"] > 500]
```

Multiple conditions:

```
df[(df["price"] > 500) & (df["rating"] >= 4)]
```

This is how questions become answers.

Handling Missing Values

Check missing data:

```
df.isna()  
df.isna().sum()
```

Remove missing values:

```
df.dropna()
```

Fill missing values:

```
df["rating"] = df["rating"].fillna(df["rating"].mean())
```

Always decide whether to remove or fill based on context.

Renaming Columns

Clean column names early:

```
df = df.rename(columns={"Product Name": "product_name"})
```

Clean column names make code readable and consistent.

Changing Data Types

Check data types:

```
df.dtypes
```

Convert types:

```
df["price"] = df["price"].astype(float)
```

Incorrect data types lead to incorrect analysis.

Removing Duplicates

```
df.drop_duplicates()
```

Duplicates silently distort results if not handled.

Basic String Cleaning

```
df["category"] = df["category"].str.lower().str.strip()
```

String cleaning is common in real-world datasets.

Commonly Used Pandas Functions

Purpose	Function
View first rows	<code>head()</code>
Dataset summary	<code>info()</code>
Statistical summary	<code>describe()</code>
Select column	<code>df["col"]</code>
Filter rows	<code>df[condition]</code>
Check missing values	<code>isna()</code>
Remove missing values	<code>dropna()</code>
Fill missing values	<code>fillna()</code>
Rename columns	<code>rename()</code>
Change data type	<code>astype()</code>
Remove duplicates	<code>drop_duplicates()</code>
String operations	<code>str.lower()</code> , <code>str.strip()</code>



Most real-world data does not start inside your code. It comes from files created by people, systems, or other tools.

Knowing how to correctly read files is the first step of any data workflow.

Reading a CSV File

CSV files are the most common data format.

```
import pandas as pd

df = pd.read_csv("data.csv")
```

Always inspect the data after loading:

```
df.head()
df.info()
```

Never assume the file was read correctly.

Common CSV Issues

Custom Separator

```
pd.read_csv("data.csv", sep=";")
```

Skipping Rows

```
pd.read_csv("data.csv", skiprows=2)
```

Selecting Specific Columns

```
pd.read_csv("data.csv", usecols=["name", "price"])
```

Reading Excel Files

Excel files often contain multiple sheets.

```
df = pd.read_excel("data.xlsx")
```

Read a specific sheet:

```
pd.read_excel("data.xlsx", sheet_name="Sheet1")
```

List all sheet names:

```
pd.ExcelFile("data.xlsx").sheet_names
```

Handling Missing or Broken Data While Reading

```
pd.read_csv("data.csv", na_values=["NA", "missing", "-"])
```

This helps clean data early instead of fixing it later.

Setting Column Data Types While Reading

```
pd.read_csv(  
    "data.csv",  
    dtype={"price": float, "quantity": int}  
)
```

This prevents data type problems early.

Reading Large Files Safely

```
pd.read_csv("data.csv", chunksize=10000)
```

This reads the file in smaller pieces instead of loading everything at once.



Data analysis is incomplete until results are saved or shared. Often, your final output is not a plot or a model, but a clean file that others can use.

Pandas makes exporting data simple and reliable.

Writing to a CSV File

Save a DataFrame as a CSV file:

```
df.to_csv("clean_data.csv", index=False)
```

`index=False` is commonly used to avoid writing row numbers.

Writing to an Excel File

Save a DataFrame to Excel:

```
df.to_excel("clean_data.xlsx", index=False)
```

This creates a single-sheet Excel file.

Writing Multiple Sheets to Excel

```
with pd.ExcelWriter("report.xlsx") as writer:  
    df_sales.to_excel(writer, sheet_name="Sales", index=False)  
    df_users.to_excel(writer, sheet_name="Users", index=False)
```

This is useful for reports and dashboards.

Appending Data to Existing Files

Append to a CSV file:

```
df.to_csv("log.csv", mode="a", header=False, index=False)
```

Use this carefully to avoid duplicate data.

Controlling Output Format

Change separator:

```
df.to_csv("data.csv", sep=";")
```

Control missing values:

```
df.to_csv("data.csv", na_rep="NA")
```

Saving Only What You Need

Always export:

- only required columns
- cleaned and validated data

Example:

```
df[["name", "price", "rating"]].to_csv("final_output.csv", index=False)
```



1. What is Matplotlib and Why It Exists

Matplotlib is a **low-level plotting library** in Python that gives you **full control** over visualizations.

Why Matplotlib matters:

- Foundation of almost every Python plotting library
- Works with NumPy arrays directly
- Maximum control over plots (labels, ticks, layout, styles)
- Industry standard for custom, publication-ready plots

Seaborn, pandas `.plot()`, etc. are all built **on top of Matplotlib**.

2. Installation and Import

```
pip install matplotlib
```

```
import matplotlib.pyplot as plt
import numpy as np
```

3. Basic Plot Anatomy

Every Matplotlib plot has:

- Figure → entire canvas
- Axes → actual plotting area
- Data → lines, bars, points

Basic example:

```
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]

plt.plot(x, y)
plt.show()
```

4. Most Important Plot Types

4.1 Line Plot

Used for trends over time or continuous data.



```
plt.plot(x, y)
plt.title("Line Plot Example")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

Useful parameters:

```
plt.plot(x, y, color="red", linewidth=2, linestyle="--", marker="o")
```

4.2 Scatter Plot

Used to show **relationship between two variables**.

```
plt.scatter(x, y)
plt.show()
```

Common options:

```
plt.scatter(x, y, s=100, c="green", alpha=0.7)
```

4.3 Bar Chart

Used for **categorical comparisons**.

```
categories = ["A", "B", "C"]
values = [20, 35, 30]

plt.bar(categories, values)
plt.show()
```

Horizontal bar:

```
plt.barh(categories, values)
```

4.4 Histogram

Used to show **distribution of data**.

```
data = np.random.randn(1000)
```

```
plt.hist(data, bins=30)  
plt.show()
```

Key parameters:

```
plt.hist(data, bins=30, edgecolor="black")
```

4.5 Box Plot

Used to detect **outliers and spread**.

```
plt.boxplot(data)  
plt.show()
```

4.6 Pie Chart

Only for **very small categories**.

```
sizes = [40, 35, 25]  
labels = ["A", "B", "C"]  
  
plt.pie(sizes, labels=labels, autopct="%1.1f%%")  
plt.show()
```

5. Titles, Labels, Legends

```
plt.plot(x, y, label="Sales")  
plt.title("Monthly Sales")  
plt.xlabel("Month")  
plt.ylabel("Revenue")  
plt.legend()  
plt.show()
```



6. Multiple Lines on One Plot

```
y2 = [12, 18, 22, 28]

plt.plot(x, y, label="Product A")
plt.plot(x, y2, label="Product B")
plt.legend()
plt.show()
```

7. Figure Size and DPI

```
plt.figure(figsize=(8, 4), dpi=100)
plt.plot(x, y)
plt.show()
```

8. Subplots - Multiple Charts Together

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].plot(x, y)
axes[0].set_title("Line Plot")

axes[1].bar(categories, values)
axes[1].set_title("Bar Chart")

plt.show()
```

9. Grid, Ticks, and Limits

Grid

```
plt.grid(True)
```

Axis Limits

```
plt.xlim(0, 5)
plt.ylim(0, 30)
```

Custom Ticks

```
plt.xticks([1, 2, 3, 4])  
plt.yticks([10, 20, 30])
```

10. Styles

```
plt.style.use("ggplot")
```

Other common styles:

- `seaborn-v0_8`
- `dark_background`
- `classic`



We can save our plots to image files like PNG for sharing or reports.

```
plt.savefig("plot.png", bbox_inches="tight")
```

We can also save in other formats like JPG, SVG, PDF, etc. For instance:

```
plt.savefig("plot.pdf")
```

Save before `plt.show()` .

Real Data Analytics Example

```
days = np.arange(1, 8)
sales = [120, 150, 170, 160, 180, 200, 220]

plt.figure(figsize=(6,4))
plt.plot(days, sales, marker="o")
plt.title("Weekly Sales Trend")
plt.xlabel("Day")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```

What this tells:

- Trend direction
- Growth pattern
- Business insight in one glance



Seaborn is a powerful Python library for data visualization built on top of Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

1. The Real Question

Why do we even need Seaborn?

We already have Matplotlib. So why another library?

Because **Matplotlib is hard for analysis**.

In real data analytics, our goal is not to draw charts. Our goal is to **understand data quickly**.

Seaborn exists to answer questions like:

- Is there a relationship between two columns?
- How does a number behave across categories?
- Are there outliers?
- Which features are correlated?

Seaborn helps you **see patterns in one line of code**.

2. Think Like an Analyst

Task	Matplotlib	Seaborn
Draw a chart	Good	Good
Explore data quickly	Hard	Very easy
Default visuals	Basic	Excellent
Statistical meaning	Manual	Built-in
Pandas friendly	Okay	Excellent

Rule of thumb

- Seaborn for **understanding**
- Matplotlib for **fine control**

3. Installation and Import

```
pip install seaborn
```



```
import seaborn as sns
import matplotlib.pyplot as plt
```

4. Seaborn Works on DataFrames (This Is Key)

Seaborn is designed for **tabular data**.

```
df = sns.load_dataset("tips")
df.head()
```

Instead of passing lists and arrays, you pass:

- column names
- the full DataFrame

This makes analysis much faster.

5. The Most Important Idea in Seaborn

Almost every Seaborn plot looks like this:

```
sns.plot_name(x="column1", y="column2", data=df)
```

You describe:

- what is on x
- what is on y
- where the data lives

That is it.

6. The 6 Seaborn Plots You Actually Need

You do not need to learn everything.

These 6 plots cover **almost all analytics use cases**.

6.1 Scatter Plot

Relationship between two numbers



```
sns.scatterplot(x="total_bill", y="tip", data=df)
plt.show()
```

Why we use it:

- Is there correlation?
- Do values increase together?
- Are there clusters?

Add category comparison:

```
sns.scatterplot(
    x="total_bill",
    y="tip",
    hue="sex",
    data=df
)
```

6.2 Line Plot

Trend with automatic averaging

```
sns.lineplot(x="size", y="tip", data=df)
plt.show()
```

Why Seaborn is special here:

- It automatically calculates the mean
- Shows trends, not raw noise

6.3 Count Plot

How many times something occurs

```
sns.countplot(x="day", data=df)
plt.show()
```

Why we use it:



- Distribution of categories
- Class imbalance
- Most common values

With split:

```
sns.countplot(x="day", hue="sex", data=df)
```

6.4 Bar Plot

Average value per category

```
sns.barplot(x="day", y="total_bill", data=df)  
plt.show()
```

Important:

- This shows **mean**, not sum
- Confidence interval is shown by default

This is why Seaborn is powerful for analysis.

6.5 Box Plot

Spread, median, and outliers

```
sns.boxplot(x="day", y="total_bill", data=df)  
plt.show()
```

Why analysts love it:

- See outliers instantly
- Compare distributions across groups

6.6 Heatmap

Relationships between numeric columns



```
corr = df.corr(numeric_only=True)

sns.heatmap(corr, annot=True)
plt.show()
```

Why this is critical:

- Feature selection
- Understanding dependencies
- Detecting redundancy

7. One Example That Explains Everything

```
sns.boxplot(
    x="day",
    y="tip",
    hue="sex",
    data=df
)
plt.show()
```

In one plot you see:

- Day-wise comparison
- Gender-wise comparison
- Spread and outliers

Doing this in Matplotlib would take much more code.

8. Styling Is Secondary (But Easy)

```
sns.set_theme(style="whitegrid")
```

This is not the main reason we use Seaborn. Insight comes first. Looks come later.

9. When Should You Use Seaborn First?

Use Seaborn when:



- You open a new dataset
- You are doing EDA
- You want quick insights
- You want statistically meaningful plots

Use Matplotlib when:

- You need full control
- You are exporting final visuals
- You are building dashboards

Final Summary

We use Seaborn because:

- It understands data, not just pixels
- It works directly with DataFrames
- It shows statistics automatically
- It helps us think like analysts

Seaborn is not about pretty charts. It is about understanding data fast.



1. Problem Statement

Background

You are working as a **Data Analyst for a real estate advisory firm** operating in Gurgaon. The firm spent significant time to collect data on residential properties across various localities in Gurgaon. The firm wants data-backed insights to guide **buyers, investors, and developers** in making informed real estate decisions.

You are provided with a dataset containing residential property listings across multiple sectors of Gurgaon.

The dataset includes pricing, area, BHK configuration, property type, builder details, and RERA approval status.

Dataset link:

<https://www.kaggle.com/datasets/nikhilmehrahr26/gurgaon-real-estate-dataset?resource=download>

If the dataset link is broken, we have also added the dataset for your download below.

Business Questions to Answer

Your analysis should answer the following business questions clearly.

1. Which is the **costliest flat** in the dataset?
 2. Which locality has the **highest average price**?
 3. Which locality has the **highest rate per square foot**?
 4. Do **ready-to-move** properties cost more than **under-construction** properties?
 5. Do **RERA-approved** properties command a price premium?
 6. How does **area (sqft)** impact property price?
 7. Which **BHK configuration** is the most expensive on average?
 8. Which **property type** (Apartment, Floor, Plot) is the costliest?
 9. Do certain **builders or companies** consistently price higher?
 10. Are larger homes always more expensive per square foot?
-

2. Solution Overview

We will solve this problem using **Exploratory Data Analysis (EDA)**.

EDA helps us understand patterns, relationships, and anomalies in the data before applying any advanced modeling.

- The solution is divided into logical steps:
1. Data loading and understanding
 2. Data cleaning and preparation
 3. Analysis to answer each business question
 4. Summary of insights

Each step below explains **what we are doing and why**.

3. Data Loading and Initial Understanding

Step 1: Import Required Libraries

We use Pandas for data manipulation and Matplotlib and Seaborn for visualization.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load the Dataset

```
df = pd.read_csv("gurgaon_real_estate.csv")
df.head()
```

Why this step matters:

- Allows us to inspect the structure of the dataset
 - Helps verify column names and data types
-

Step 3: Basic Dataset Overview

```
df.shape
df.info()
```

Why this step matters:

- Identifies number of rows and columns
 - Reveals missing values and incorrect data types
-

4. Data Cleaning and Preparation

Real-world datasets are messy. Cleaning ensures accurate analysis.

Step 4: Standardize Column Names


```
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
```

Why:

- Makes column names consistent and easier to use in code

Step 5: Convert Numeric Columns

```
df["price"] = (  
    df["price"].astype(str)  
    .str.replace(",", "")  
    .astype(int)  
)  
  
df["area"] = (  
    df["area"].astype(str)  
    .str.replace(",", "")  
    .astype(int)  
)  
  
df["rate_per_sqft"] = (  
    df["rate_per_sqft"].astype(str)  
    .str.replace(",", "")  
    .astype(int)  
)
```

Why:

- Numeric calculations are impossible if values are stored as strings

Step 6: Clean Categorical Columns

```
df["status"] = df["status"].str.strip().str.lower()  
df["rera_approval"] = df["rera_approval"].str.strip().str.lower()  
df["flat_type"] = df["flat_type"].str.strip().str.lower()
```

Why:

- Avoids duplicate categories caused by inconsistent formatting

Step 7: Remove Duplicates

```
df = df.drop_duplicates()
```

Why:

- Duplicate rows can skew averages and insights

5. Answering Business Questions with Analysis

Question 1: Which is the costliest flat?

```
df.loc[df["price"].idxmax()]
```

Why:

- Identifies the highest priced property directly

Question 2: Which locality has the highest average price?

```
df.groupby("locality")["price"].mean().sort_values(ascending=False)
```

Why:

- Grouping helps compare average prices across localities

Question 3: Which locality has the highest rate per square foot?

```
df.groupby("locality")["rate_per_sqft"].mean().sort_values(ascending=False)
```

Why:

- Rate per sqft highlights premium locations independent of property size

Question 4: Ready-to-move vs Under-construction pricing

```
df.groupby("status")["price"].median()
```

Why:

- Median is preferred over mean due to outliers in property prices

Question 5: Does RERA approval affect pricing?

```
df.groupby("rera_approval")["price"].median()
```

Why:

- Helps measure trust premium associated with regulatory approval

6. Deeper Insights

Question 6: How does area impact price?

```
sns.scatterplot(x="area", y="price", data=df)
plt.show()
```

Why:

- Visual inspection reveals correlation and pricing behavior

Question 7: Which BHK configuration is most expensive?

```
df.groupby("bhk_count")["price"].mean()
```

Why:

- Helps identify demand and pricing by family size

Question 8: Which property type is the costliest?

```
df.groupby("flat_type")["price"].mean()
```

Why:

- Compares Apartments, Floors, and Plots fairly

Question 9: Do certain builders price higher?

```
df.groupby("company_name")["price"].mean().sort_values(ascending=False)
```

Why:

- Captures brand and trust impact on pricing

Question 10: Are larger homes more expensive per sqft?

```
sns.scatterplot(x="area", y="rate_per_sqft", data=df)
plt.show()
```

Why:

- Helps evaluate economies of scale in real estate pricing

7. Final Insights Summary

From the analysis, we can conclude:

- Premium localities consistently show higher prices and rate per sqft
- Ready-to-move and RERA-approved properties tend to command higher prices
- Builder reputation significantly impacts property pricing
- Larger homes are not always more expensive per sqft
- Apartments generally have higher rate per sqft than plots and floors



Section 6: AI Tools for Data Analysts - Make Coders Jealous

SECTION 06

Chapter 1: What AI can and cannot do

Lets talk about what AI can and cannot do as of now

The Two Big Directions in AI Today

AI research in 2026 is broadly moving in two directions:

1. Incremental improvement of existing models

This includes scaling and refining Large Language Models (LLMs) such as GPT-4, GPT-4.5, and GPT-5.

2. Solving problems current models fundamentally cannot solve

These problems likely require entirely new architectures rather than larger versions of today's models.

Both directions are active, but they lead to very different outcomes.

What AI Can Do Well Today

1. Pattern Recognition at Scale

Modern AI systems excel at:

- Language generation and summarization
- Code completion and boilerplate generation
- Image, video, and audio synthesis
- Translation and paraphrasing
- Extracting patterns from large datasets

These systems operate by learning statistical correlations from massive datasets and predicting likely outputs.

2. Task Performance Within a Fixed Context

AI performs well when:

- The task is well-defined
- The environment is static
- The rules do not change
- The model is not required to adapt beyond its training

Examples:

- Playing chess after being trained only on chess
- Sorting objects in a controlled factory setting
- Answering questions using known information

What AI Cannot Do Well Today

Despite impressive outputs, LLMs have fundamental constraints.

1. No Understanding of Reality

LLMs:

- Do not understand time
- Do not understand space
- Do not model physical causality
- Do not predict real-world consequences

Example: If a coin lands on heads ten times, a human questions the coin.

An LLM only predicts the next token based on prior tokens, not the underlying physical process.

This limitation is well-documented and widely accepted in AI research.

2. Poor Performance on Rare or Uncertain Events

LLMs struggle when:

- Data is sparse
- Outcomes are unpredictable
- Reasoning requires causal inference rather than correlation

This is a direct consequence of next-token prediction.



What Is Agent Mode?

Agent Mode turns VS Code from a passive editor into an active collaborator. Instead of just suggesting code, the agent understands your workspace, plans multi-step changes, and executes them toward a goal.

Why It Matters

Modern projects are large and interconnected. One-shot AI prompts fail at:

- Multi-file changes
- Refactors
- Context-heavy debugging

Agent Mode exists to handle **end-to-end tasks**, not just snippets.

Core Ideas

- **Goal-driven:** You give intent, not instructions
- **Workspace-aware:** Understands folders, files, imports
- **Iterative:** Plans, acts, checks, improves
- **Action-capable:** Can edit files, create files, and run tasks

Agent Mode vs Chat AI

Chat AI	Agent Mode
Single response	Multi-step execution
Local context	Full workspace
Suggestions	Ownership of tasks

Best Use Cases

- Large refactors
- Debugging across files
- Feature scaffolding
- Codebase understanding

Limitations

- Needs clear goals
- Can make broad changes
- Always requires review

Big Picture

Agent Mode raises the abstraction level of development.

You focus on *what* needs to be done.

The agent helps execute the *how*.

CodeWithHarry



AI based terminals are a new category of tools that combine the power of AI coding assistants with the command-line interface. They allow developers to interact with their terminal using natural language prompts, making it easier to write, debug, and optimize code without needing to remember complex commands or syntax.

What Is Claude Code?

Claude Code is an AI-powered coding assistant developed by Anthropic, designed to help developers write, debug, and optimize code using natural language prompts.

What is Warp?

Warp is a modern terminal that integrates with Claude Code to provide an enhanced command-line experience, allowing users to interact with their terminal using AI assistance.

Warp Offer Update: Warp has recently announced that they are now offering free access for limited time under their free tier for all users. Please visit their website and sign up to take advantage of this offer while it lasts.



Quadratic AI (and similar "AI spreadsheet" tools) try to combine:

- the familiarity of a spreadsheet
- the power of SQL and databases
- and an AI chat layer that helps you transform and analyze data faster

For a data analyst, this sits between Excel/Sheets and a full BI tool.

What It Can Help With

1. Spreadsheet tasks

- Writing formulas from plain English (LOOKUPS, IFs, date logic)
- Cleaning messy columns (currency symbols, extra spaces, mixed case)
- Creating summaries (pivots / group-bys)
- Generating quick charts

2. SQL and database work

- Drafting SQL queries from a question
- Explaining what an existing query does
- Debugging joins, GROUP BY, and window functions
- Suggesting query improvements (always verify with your DB)

A Simple Workflow

1. Import your data (CSV/Excel export or a query result)
2. Ask the AI to create a "clean" version of the table (new columns, fixed types)
3. Ask questions and request reproducible steps (formula + explanation, or SQL)
4. Validate the output (spot-check rows, totals, and edge cases)
5. Export the cleaned dataset / summary for reporting

Example Prompts You Can Use

- "Convert `price` from 'INR 1,23,000' into a numeric column and handle blanks."
- "Create a summary table: total revenue by `region` and `month`."
- "Find outliers in `delivery_time_days` and show the top 20."
- "Write SQL to get the top 10 customers by revenue in the last 90 days."
- "Explain why this JOIN is duplicating rows and fix it."

Offer

I partnered with the Quadratic team to bring you a special deal: Use the code **HARRYPRO** when you sign up, and you'll get **\$20 off** your Pro subscription (which is equivalent to **1 month free**). Just apply the code during checkout, and you're all set.

CodeWithHarry



Manus AI focus on turning a prompt into a deliverable:

- Slides (pitch decks, reports)
- Websites (landing pages, portfolios)
- Simple apps (forms, dashboards, demos)
- Sometimes includes one-click deployment/hosting

For data analysts, this is mainly useful for packaging your work for other people.

Where It Helps a Data Analyst

- Turn an analysis into a clean slide deck quickly
- Build a one-page "executive summary" website for stakeholders
- Create a small interactive demo (filters, charts, KPI cards)
- Produce templates you can reuse across projects

A Good Prompt Template

Include: 1. Audience: "CEO / Marketing Lead / Hiring Manager" 2. Goal: "Explain insights and recommendations" 3. Inputs: "These KPIs, this chart, these bullet points" 4. Style: "Minimal, professional, company colors" 5. Output: "10-slide deck" or "single-page site with 5 sections" 6. Constraints: "No fake data; include placeholders for missing assets"

Example:

Create a 10-slide deck for a real estate investor. Use a neutral, professional style. Slides: Problem, Dataset, Key Findings, Chart Placeholders, Recommendations, Risks, Next Steps. Do not invent numbers.



Ollama focuses on one powerful idea: **running Large Language Models (LLMs) locally instead of the cloud.**

Traditionally, whenever we used AI tools like ChatGPT or Gemini, the model was running on **remote servers**. Your prompts were sent to the cloud, processed there, and results came back. Ollama flips this model.

What Is Ollama?

Ollama is a local runtime that lets you download and run open-source LLMs on your own system.

In simple terms:

- No cloud calls
- No API keys required
- No internet needed (after model download)
- Full control over data and prompts

It supports popular open models like:

- Llama
- Mistral
- Gemma
- Code-focused models

Once installed, running a model feels as simple as running a command.

Why This Matters

Until now, AI workflows usually looked like this:

You → Internet → Cloud Model → Internet → You

With Ollama, it becomes:



Key Benefits



- **Privacy:** Your data never leaves your machine
- **Low latency:** No network delay
- **Offline usage:** Works without internet
- **Cost control:** No per-token billing
- **Customization:** Easier fine-tuning and experimentation

For developers, analysts, and students, this unlocks a whole new way to experiment with AI safely.

Where Ollama Helps in Real Projects

For Developers

- Build AI-powered apps without exposing user data
- Prototype agents and tools locally
- Test prompts and chains before deploying to cloud models

For Data Analysts

- Analyze sensitive datasets locally
- Use LLMs for:
 - SQL query generation
 - Data explanations
 - Report drafting
 - Avoid compliance and data-sharing issues

For Learners

- Understand how LLMs actually run
 - Experiment with models hands-on
 - Learn AI without paying for APIs
-

Installation & Setup

Ollama is designed to be easy to install and use. You can get it running in minutes on Windows, macOS, or Linux.

1. Download the installer from <https://ollama.com>
 2. Install like a normal application
 3. Restart your terminal
-

Downloading Your First Model

We'll use **Gemma 3** as an example, which is a powerful open-source model. You can choose from many models, but Gemma 3 is a great starting point. Here you can find the list of available models:

<https://ollama.com/models>

We will open the terminal and Run the following command to download the model:

```
ollama pull gemma3
```

This downloads the Gemma 3 model to your machine (one-time).

Running the Model Locally

Once the model is downloaded, you can run it with:

```
ollama run gemma3
```

You are now running **Gemma 3 fully locally**. You can type prompts directly in the terminal and see responses immediately.

When You Should NOT Use Ollama

Ollama is powerful, but it's not magic.

Avoid it if:

- You need **very large models** and your hardware can't handle them
- You rely on **cutting-edge proprietary models**
- You need **massive scale** for thousands of users

In those cases, cloud APIs still make sense.

Conclusion

Ollama is a **huge shift** in how we think about AI usage. Instead of relying on the cloud, you can now run powerful LLMs right on your own machine. This opens up new possibilities for privacy, control, and experimentation.

If you care about privacy, control, learning, or building serious AI-powered tools locally, Ollama is absolutely worth exploring.

Section 7: Advanced Excel for Data Analytics

Chapter 1: Advanced Excel for Data Analytics

In this section, we will study some of the advanced Excel techniques used in data analytics.

We already know how to use basic Excel functions and formulas, but there are many more powerful features in Excel that can help us analyze data more effectively. Some of these features include:

- **Pivot Tables:** Pivot tables allow you to summarize and analyze large datasets quickly. You can easily group, filter, and aggregate data to find insights.
- **VLOOKUP and HLOOKUP:** These functions help you search for specific values in a table and return corresponding data. They are useful for merging datasets or finding specific information.

& many more!

CodeWithHarry



VLOOKUP stands for **Vertical Lookup**. It searches for a value in the **first column** of a table and returns a value from a column to the **right** (same row).

In data analytics, VLOOKUP is commonly used to:

- match IDs (ProductID, EmployeeID, CustomerID) across tables
- bring in attributes like category, price, region, manager, etc.
- enrich raw datasets before building pivots/charts

Syntax

```
=VLOOKUP(lookup_value, table_array, col_index_num, [range_lookup])
```

- **lookup_value**: the value you want to find (e.g., `P1003`)
- **table_array**: the lookup table range (the lookup column must be the **first** column of this range)
- **col_index_num**: which column to return (1 = first column of `table_array`)
- **range_lookup**:
 - `FALSE` = exact match (most common)
 - `TRUE` (or omitted) = approximate match (requires sorted lookup column)

Example 1: Exact match (ProductID -> Price)

Step 1: Create a `Products` table

Paste this into Excel (start in cell `A1` on a sheet named `Products`):

ProductID	ProductName	Category	Price
P1001	Notebook	Stationery	50
P1002	Pen	Stationery	10
P1003	USB Cable	Electronics	199
P1004	Mouse	Electronics	499
P1005	Water Bottle	Lifestyle	299

Step 2: Create an `Orders` table

Paste this into another sheet named `Orders` (start in cell `A1`):

OrderID	ProductID	Qty
O-001	P1003	2
O-002	P1001	5
O-003	P1004	1
O-004	P1002	10
O-005	P9999	1

Step 3: Use VLOOKUP to fetch the Price

In `Orders!D1` type `Price`, then in `Orders!D2`:

```
=VLOOKUP(B2, Products!$A$2:$D$6, 4, FALSE)
```

Now add a `Total` column:

- In `Orders!E1` type `Total`
- In `Orders!E2`:

```
=C2*D2
```

Handle "not found" cleanly

To show a friendly message instead of `#N/A`:

```
=IFNA(VLOOKUP(B2, Products!$A$2:$D$6, 4, FALSE), "Product not found")
```

If your Excel doesn't have `IFNA`, you can use:

```
=IFERROR(VLOOKUP(B2, Products!$A$2:$D$6, 4, FALSE), "Product not found")
```

Example 2: Approximate match (Score -> Grade)

Approximate match is useful for *slabs* (grades, tax brackets, discount tiers).

Important rules:

- the lookup column **must be sorted ascending**
- use `TRUE` (or omit the last argument)

Paste this table into Excel (start in `A1`):

MinScore	Grade
0	F
50	D
60	C
70	B
85	A

If a student's score is in `D2`, then:

```
=VLOOKUP(D2, $A$2:$B$6, 2, TRUE)
```

If `D2 = 73`, the result is **B** (because 70 is the nearest lower boundary).

Example 3: Make VLOOKUP safer with MATCH (dynamic column number)

Hardcoding `col_index_num` breaks when someone inserts/reorders columns.

If you want to return the column whose header is written in `H1` (like `Price` or `Category`):

```
=VLOOKUP($B2, Products!$A$1:$D$6, MATCH($H$1, Products!$A$1:$D$1, 0), FALSE)
```

Common mistakes (and quick fixes)

- Getting wrong results? Make sure you used `FALSE` for exact matches.
- Formula changes when you copy it? Lock the table range using `$` (press **F4**).
- Lookup not matching even though it "looks" the same? Check for extra spaces and data type issues (number stored as text).
- Need to look left? VLOOKUP can't. Use **XLOOKUP** (recommended) or `INDEX` + `MATCH`.

HLOOKUP stands for **Horizontal Lookup**. It searches for a value in the **top row** of a table and returns a value from a row **below** (same column).

HLOOKUP is less common than VLOOKUP because most datasets in Excel are stored vertically (headers in the first row, data down rows). But it's still useful when your table is organized across columns (months across the top, KPIs in rows, etc.).

Syntax

```
=HLOOKUP(lookup_value, table_array, row_index_num, [range_lookup])
```

- **lookup_value**: the value to find in the first row (example: `Mar`)
- **table_array**: the table range (the lookup row must be the **first** row of this range)
- **row_index_num**: which row to return (1 = first row of `table_array`)
- **range_lookup**:
 - `FALSE` = exact match
 - `TRUE` (or omitted) = approximate match (first row must be sorted)

Example 1: Get a KPI for a given month (exact match)

Paste this into Excel (start in cell `A1`):

Metric	Jan	Feb	Mar	Apr
Revenue	12000	15000	11000	18000
Units	320	410	295	500
Profit	2500	3400	2100	4200

Fetch revenue for March

```
=HLOOKUP("Mar", $A$1:$E$4, 2, FALSE)
```

Explanation:

- It finds **Mar** in the first row (`A1:E1`)
- Then returns the value from the **2nd row** of the range (Revenue row)

Make it dynamic (Month + Metric)

If `G1` has the month (e.g., `Apr`) and `G2` has the metric (e.g., `Profit`), then:


```
=HLOOKUP($G$1, $A$1:$E$4, MATCH($G$2, $A$1:$A$4, 0), FALSE)
```

`MATCH($G$2, $A$1:$A$4, 0)` returns the correct row number inside the table (Revenue = 2, Units = 3, Profit = 4).

Example 2: Approximate match (slabs across the top)

Approximate match is useful when the top row contains **breakpoints** (tiers/slabs).

Important rules:

- the first row must be sorted ascending
- use `TRUE` (or omit the last argument)

Paste this table into Excel (start in `A1`):

0	10000	25000	50000	100000
0%	2%	3%	4%	5%

If monthly sales are in `D2`, then the commission rate is:

```
=HLOOKUP(D2, $A$1:$E$2, 2, TRUE)
```

If `D2 = 42000`, the result is **3%** (nearest lower breakpoint is 25000).

Quick notes

- If your data is vertical, prefer **VLOOKUP** (older) or **XLOOKUP** (modern).
- XLOOKUP can also do "horizontal lookups" by using a row as the lookup/return arrays.

XLOOKUP is the modern replacement for **VLOOKUP** and **HLOOKUP**.

Why data analysts like XLOOKUP:

- **Exact match by default** (no more "forgot to type FALSE")
- Can look **left or right** (no column-index limitation)
- Built-in **if_not_found** message
- Can return **multiple columns** at once (spills automatically)
- Can search from **last to first** (useful for "latest value" problems)

Note: XLOOKUP is available in Microsoft 365 and Excel 2021+. If you don't have it, use VLOOKUP/HLOOKUP or INDEX + MATCH.

Syntax

```
=XLOOKUP(lookup_value, lookup_array, return_array,  
[if_not_found], [match_mode], [search_mode])
```

- **lookup_value**: the value to find
- **lookup_array**: where to search (a single row/column)
- **return_array**: what to return (must be the same size as lookup_array)
- **if_not_found** (optional): what to show if there's no match
- **match_mode** (optional):
 - 0 exact match (default)
 - -1 exact or next smaller
 - 1 exact or next larger
 - 2 wildcard match (* and ?)
- **search_mode** (optional):
 - 1 first-to-last (default)
 - -1 last-to-first (great for "latest record")

Sample data (Products)

Paste this into Excel (start in cell A1 on a sheet named Products):



ProductID	ProductName	Category	Price
P1001	Notebook	Stationery	50
P1002	Pen	Stationery	10
P1003	USB Cable	Electronics	199
P1004	Mouse	Electronics	499
P1005	Water Bottle	Lifestyle	299

Example 1: Basic exact match (ProductID -> Price)

If contains a ProductID:

```
=XLOOKUP(E2, Products!$A$2:$A$6, Products!$D$2:$D$6, "Not found")
```

Example 2: Return multiple columns (spill)

If contains a ProductID, this returns **ProductName, Category, Price** in one shot:

```
=XLOOKUP(E2, Products!$A$2:$A$6, Products!$B$2:$D$6, "Not found")
```

Example 3: Lookup to the left (ProductName -> ProductID)

VLOOKUP can't easily "look left", but XLOOKUP can.

If has the ProductName:

```
=XLOOKUP(G2, Products!$B$2:$B$6, Products!$A$2:$A$6, "Not found")
```

Example 4: Approximate match for slabs (Score -> Grade)

Paste this into Excel (start in):

MinScore	Grade
0	F
50	D
60	C
70	B
85	A

If the score is in `D2`, then:

```
=XLOOKUP(D2, $A$2:$A$6, $B$2:$B$6, "Invalid", -1)
```

`-1` means: exact match, or the **next smaller** boundary (so the slab table should be sorted ascending).

Example 5: Wildcard / partial text match

If the user types `usb` in `J2` and you want to find the matching ProductID:

```
=XLOOKUP("*"&J2&"*", Products!$B$2:$B$6, Products!$A$2:$A$6, "Not found", 2)
```

`match_mode = 2` enables wildcard matching.

Example 6: Get the latest value (last-to-first search)

This is very common in analytics: "give me the most recent price/status for this ID".

Paste this table into Excel (start in `A1`):

Date	ProductID	Price
2025-01-05	P1003	199
2025-02-10	P1003	179
2025-03-20	P1003	189
2025-01-15	P1004	499
2025-03-05	P1004	449

If your ProductID is in `F2`, the last occurrence (from bottom) is:

```
=XLOOKUP(F2, $B$2:$B$6, $C$2:$C$6, "Not found", 0, -1)
```

Tip: If you sort this table by **Date** (oldest → newest), the "last match" becomes the **latest** record.

Example 7: Two-way lookup (row + column)

Sometimes your data is a matrix (like "Product x Month"). XLOOKUP can handle this by nesting.

Paste this table into Excel (start in **A1**):

Product	Jan	Feb	Mar
P1001	120	150	180
P1002	90	70	95
P1003	40	55	60

If **F1** has the Month (e.g., **Feb**) and **F2** has the Product (e.g., **P1002**):

```
=XLOOKUP($F$1, $B$1:$D$1, XLOOKUP($F$2, $A$2:$A$4, $B$2:$D$4))
```

Text to Columns is used to split data from one column into multiple columns using a delimiter or fixed width.

Where to Find "Text to Columns" in Excel

Data tab → Data Tools → Text to Columns

You can also use the keyboard shortcut:

Alt + A + E

Two Modes of Text to Columns

It has two modes for splitting data:

1. Delimited

Splits data based on a separator such as:

- Comma
- Space
- Tab
- Semicolon
- Custom character

2. Fixed Width

Splits data based on character positions that you manually define.

Example 1: Splitting Full Name

Sample Data

Paste into Excel starting at :

FullName
Rohan Das
Aisha Khan
Arjun Mehta

COPY PASTE (TSV):

FullName
Rohan Das
Aisha Khan
Arjun Mehta

Steps

1. Select column **A**

Data → Text to Columns

2. Choose **Delimited**
3. Select **Space**
4. Click **Finish**

Result:

FirstName	LastName
Rohan	Das
Aisha	Khan
Arjun	Mehta

Example 2: Splitting CSV Data

Sample Data

RawData
P1001,Notebook,50
P1002,Pen,10
P1003,USB Cable,199

RawData P1001,Notebook,50 P1002,Pen,10 P1003,USB Cable,199

Steps



1. Select the column

Data → Text to Columns

2. Choose **Delimited**

3. Select **Comma**

4. Click **Finish**

Result:

ProductID	ProductName	Price
P1001	Notebook	50
P1002	Pen	10
P1003	USB Cable	199

Example 3: Fixed Width Split

Sample Data

Code
IND2025001
IND2025002
IND2025003

Code IND2025001 IND2025002 IND2025003

Assume:

- First 3 characters = Country
- Next 4 = Year
- Last 3 = Serial

Steps

1. Select the column

Data → Text to Columns

2. Choose **Fixed Width**

3. Add break lines after 3 and 7 characters

4. Click **Finish**

Result:

Country	Year	Serial
IND	2025	001
IND	2025	002
IND	2025	003



Remove Duplicates is used to eliminate repeated values from a dataset so that only unique records remain.

How to use "Remove Duplicates" Feature in Excel

Navigate to:

Data tab → Data Tools → Remove Duplicates

You can also use the keyboard shortcut:

Alt + A + M

How It Works

When you use **Remove Duplicates**, Excel:

- Checks selected columns
- Identifies duplicate rows
- Keeps the first occurrence
- Deletes all repeated entries

You can choose one or multiple columns to check for duplication.

Example 1: Removing Duplicate Names

Sample Data

Paste the below data into Excel starting at **A1** :

Name
Rohan Das
Aisha Khan
Rohan Das
Arjun Mehta
Aisha Khan

Name Rohan Das Aisha Khan Rohan Das Arjun Mehta Aisha Khan

Steps

Chapter 6: Removing Duplicates in Excel

SECTION 07

1. Select column **A**

Data → **Remove Duplicates**

2. Ensure **Name** is checked

3. Click **OK**

Result:

Name
Rohan Das
Aisha Khan
Arjun Mehta

Excel will show a message like:

2 duplicate values found and removed.
3 unique values remain.

Example 2: Remove Duplicate Rows

Sample Data

OrderID	Customer	Amount
1001	Rohan	500
1002	Aisha	300
1001	Rohan	500
1003	Arjun	700

OrderID Customer Amount 1001 Rohan 500 1002 Aisha 300 1001 Rohan 500 1003 Arjun 700

Steps

1. Select the entire table

Data → **Remove Duplicates**

2. Keep all columns checked

3. Click **OK**

Result:

OrderID	Customer	Amount
1001	Rohan	500
1002	Aisha	300
1003	Arjun	700

Here, the entire duplicated row was removed.

Example 3: Remove Duplicates Based on One Column Only

Sometimes you want to remove duplicates based on a single column but keep other data.

Sample Data

Email	Name	City
rohan@email.com	Rohan	Delhi
aisha@email.com	Aisha	Mumbai
rohan@email.com	Rohan D	Pune
arjun@email.com	Arjun	Jaipur

Email Name City rohan@email.com Rohan Delhi aisha@email.com Aisha Mumbai rohan@email.com Rohan D Pune arjun@email.com Arjun Jaipur

Goal: Keep only one entry per **Email**

Steps

1. Select the entire table

Data → Remove Duplicates

2. Uncheck all columns
3. Check only **Email**
4. Click **OK**

Result:

Email	Name	City
rohan@email.com	Rohan	Delhi
aisha@email.com	Aisha	Mumbai
arjun@email.com	Arjun	Jaipur

Excel keeps the **first occurrence** of each email and removes the rest.



Notes and Comments are used to add extra information to a cell without changing the actual data inside the cell.

How to Add Notes and Comments in Excel

You can also right-click on a cell and choose:

New Note
New Comment

Also, you can use the Review tab in the ribbon to add Notes and Comments.

Navigate to:

Review tab → Notes → New Note

For comments:

Review tab → Comments → New Comment

Note: Notes are available in all versions of Excel, while Comments are a newer feature designed for collaboration in Excel Online and Excel 365.

Example 1: Adding a Note to a Cell

Sample Data

Paste the below data into Excel starting at **A1** :

Name	Marks
Rohan	85
Aisha	92
Arjun	76

Name	Marks
Rohan	85
Aisha	92
Arjun	76

Steps

- 1. Select any cell.
- 2. Right-click on the cell and select **New Note**

[Or you can go to Review](#) → [New Note](#)

- 3. Type your Note.
- 4. Click outside the note box

Result:

- A small red indicator appears in the corner of the cell
- Hovering over the cell shows the note

Example 2: Adding a Comment to a Cell

Comments are used for discussion or collaboration.

Sample Data

Task	Status
Report	Pending
Presentation	Completed

Task	Status
Report	Pending
Presentation	Completed

Steps

- 1. Select any cell.
- 2. Right-click on the cell and select **New Comment**

[Or you can go to Review](#) → [New Comment](#)

- 3. Type your message
- 4. Click **Post** (if required)

Result:

- A comment icon appears in the cell
- The comment can be opened to view or reply

Freeze Panes is used to lock specific rows or columns so they remain visible while you scroll through large datasets.

How to Use "Freeze Panes" Feature in Excel

Navigate to:

View tab → Freeze Panes

You will see three options:

Freeze Panes
Freeze Top Row
Freeze First Column

How It Works

When you use **Freeze Panes**, Excel:

- Locks selected rows above the active cell
- Locks selected columns to the left of the active cell
- Keeps frozen sections visible while scrolling
- Applies freezing only to the current worksheet

Example 1: Freeze Top Row

This is commonly used to keep headers visible.

Sample Data

Paste the below data into Excel starting at **A1** :

Name	Department	Salary
Rohan	IT	50000
Aisha	HR	45000
Arjun	Sales	40000
Meera	Finance	55000

Name	Department	Salary
Rohan	IT	50000
Aisha	HR	45000
Arjun	Sales	40000
Meera	Finance	55000

Steps

[Go to View](#) → [Freeze Panes](#)

1. Click **Freeze Top Row**

Result:

- Row 1 remains visible while scrolling down
- Headers stay fixed at the top

Example 2: Freeze First Column

Used when the first column contains important identifiers.

Steps

[Go to View](#) → [Freeze Panes](#)

1. Click **Freeze First Column**

Result:

- Column A remains visible while scrolling horizontally
- The first column stays fixed on the left

Example 3: Freeze Multiple Rows and Columns

You can freeze both rows and columns together.

Sample Scenario

Suppose you want to freeze:

- Row 1 (Headers)
- Column A (Names)

Steps

1. Select cell **B2**

Go to View → **Freeze Panes**

2. Click **Freeze Panes** (first option)

Result:

- Row 1 is frozen
- Column A is frozen
- Everything above and to the left of **B2** remains visible while scrolling

How to Unfreeze Panes

To remove freezing:

View tab → Freeze Panes → Unfreeze Panes

After clicking **Unfreeze Panes**, all rows and columns will scroll normally again.



Find and Replace is used to quickly search for specific data in a worksheet and replace it with new data.

How to Use "Find and Replace" Feature in Excel

You can use the keyboard shortcuts:

Ctrl + F (Find)
Ctrl + H (Replace)

Or you can navigate to:

Home tab → Editing → Find & Select → Find

For replacing values:

Home tab → Editing → Find & Select → Replace

Example 1: Using Find

Sample Data

Paste the below data into Excel starting at A1 :

Name	Department
Rohan	IT
Aisha	HR
Arjun	IT
Meera	Sales

Name Department
Rohan IT
Aisha HR
Arjun IT
Meera Sales

Steps

1. Press **Ctrl + F**
2. Type
3. Click **Find Next**

Result:

- Excel highlights the first cell containing
- Clicking **Find Next** moves to the next match

Example 2: Using Replace

Our goal is to replace with

Steps

1. Press **Ctrl + H**
2. In **Find what**, type
3. In **Replace with**, type
4. Click **Replace**

Result:

- Only the currently selected match is replaced
- You can click **Replace** again for the next occurrence

Example 3: Replace All

If you want to replace all occurrences at once, we can use the **Replace All** option.

Steps

1. Press **Ctrl + H**
2. Type in **Find what**
3. Type in **Replace with**
4. Click **Replace All**

Result:

Excel will show a message like:

2 replacements made.

All matching entries in the selected range or worksheet are updated at once.

Example 4: Find and Replace by Color Code

We can also search and replace cells based on their formatting, such as background color.

Sample Scenario

Suppose some cells in the **Department** column are highlighted in **Yellow**, and you want to replace only those highlighted cells.

Steps to Find by Color:

1. Press **Ctrl + F**
2. Click **Options >>**
3. In **Find what**, leave it blank (if searching by color only)
4. Click the **Format** button next to Find
5. Choose **Choose Format From Cell** or you can directly select the Yellow color from the options
6. Click **Find All** to see all cells with that color

Result:

- Excel finds only cells with the selected color
- Replacement is applied only to those formatted cells

Steps to Replace by Color:

1. Press **Ctrl + H**
2. Click **Options >>**
3. In **Find what**, leave it blank
4. Click the **Format** button next to Find
5. Choose **Choose Format From Cell** or you can directly select the Yellow color from the options
6. In **Replace with**, type the new value (e.g.,)
7. Click **Replace All**

Result: * Only the cells with the Yellow background are replaced with the new value * Other cells remain unchanged



Advanced Sorting is used to organize data based on multiple columns, custom rules, or specific criteria beyond basic A–Z or Z–A sorting.

How to Use "Advanced Sorting" Feature in Excel

You can access sorting from:

Data tab → Sort

Or use:

Home tab → Editing → Sort & Filter → Custom Sort

Example 1: Sorting by Multiple Columns

Sample Data

Paste the below data into Excel starting at **A1** :

Name	Department	Salary
Rohan	IT	50000
Aisha	HR	45000
Arjun	IT	60000
Meera	Sales	40000
Kabir	IT	45000

Data tab → Sort

Goal

Sort by:

1. Department (A to Z)
2. Salary (Largest to Smallest)

Steps

1. Select the entire table
2. Go to **Data tab**
3. Click **Sort**
4. In the Sort dialog box:
5. Sort by:
6. Order:
7. Click **Add Level**
8. Then by:
9. Order:
10. Click **OK**

Result:

- Data is grouped by Department
- Within each Department, Salary is sorted from highest to lowest

Example 2: Sorting by Custom Order

Sometimes we need a custom sequence instead of alphabetical order.

Sample Scenario

Suppose Departments should appear in this order:

IT → HR → Sales

Steps

1. Select the table
 →
2. Sort by:
3. In Order dropdown, choose **Custom List**
4. Click **Custom List**
5. Enter the order:
7. Click **Add** 8. Click **OK** 9. Click **OK** again

Result:

- Departments are arranged in the specified custom order
- Data within each department remains grouped

Example 3: Sorting by Values Based on Conditions

You can sort numeric or text data in ascending or descending order.

Steps

1. Select the column (e.g., Salary)
2. Go to **Data tab**
3. Click:
4. **Smallest to Largest** (Ascending)
5. OR
6. **Largest to Smallest** (Descending)
7. Click Sort on the confirmation dialogue

Result:

- Data is sorted based on numerical value
- Entire rows move together to maintain data consistency



Subtotals are used to automatically calculate totals (like Sum, Count, Average, etc.) for grouped data in a worksheet.

How to Use "Subtotal" Feature in Excel

You can access Subtotal from:

Data tab → Outline group → Subtotal

Example 1: Applying Subtotals

Sample Data

Paste the below data into Excel starting at **A1** :

Department	Name	Salary
IT	Rohan	50000
IT	Arjun	60000
HR	Aisha	45000
HR	Kabir	40000
Sales	Meera	55000

```
Department  Name    Salary
IT  Rohan   50000
IT  Arjun   60000
HR  Aisha   45000
HR  Kabir   40000
Sales Meera    55000
```

Important Step Before Subtotal

Before applying Subtotal, sort the data by the column you want to group.

Steps to Sort:

1. Select the entire table
2. Go to **Data tab**
3. Click **Sort**
4. Sort by:
5. Click **OK**

Steps to Apply Subtotal:

1. Select the dataset
2. Go to **Data tab**
3. Click **Subtotal**
4. In the Subtotal dialog box:
5. At each change in:
6. Use function:
7. Add subtotal to:
8. Click **OK**

Result:

- Excel inserts subtotal rows after each Department
- A grand total appears at the bottom
- Outline levels appear on the left side for collapsing and expanding data

Example 2: Changing the Function to Count or Average

Instead of Sum, you can use other functions.

Steps:

→

1. Choose:
2. Use function: OR
3. Use function:
4. Click **OK**

Result:

- Excel calculates subtotal using the selected function
 - Subtotal rows update accordingly
-

Removing Subtotals

If you want to remove all subtotals:

Steps:

Go to Data tab → Subtotal

1. Click **Remove All**
-

Result:

- All subtotal rows are removed
- Worksheet returns to original normal format



Excel Navigation Tips

Before we move towards DSUM, let's quickly review some useful tips for navigating Excel efficiently.

1. View All Sheets

- You can **right click on the sheet navigation arrows** (bottom left corner of Excel)
- This will show a list of all sheets in the workbook
- You can directly jump to any sheet from there

2. Use the **fx** Button

- Click the **fx** button next to the formula bar
- This opens the **Function Arguments** dialog box
- It helps you fill in arguments step by step
- Useful when you are unsure about formula syntax

DSUM Function Explained

DSUM is used to calculate the sum of values in a database that meet specific criteria.

Syntax

```
=DSUM(database, field, criteria)
```

Arguments

1. database

- The entire table including headers
- Example: `A1:D100`
- The first row must contain column labels

2. field

- The column you want to sum
- You can write:
- The column label in quotes → `"Salary"`

OR the column number → `4`

3. criteria

- A separate range that contains conditions
- Must include the same headers as the database
- Defines which rows should be included in the sum

Example

If you want to sum the **Salary** of employees in **Marketing**:

```
=DSUM(A1:D100, "Salary", F1:F2)
```

Where:

A1:D100 → Full database

- "Salary" → Column to sum

F1:F2 → Criteria range containing:

- Header: Department
- Value: Marketing

DSUM will add only the Salary values where Department = Marketing.



DSUM with OR Criteria

In `DSUM`, each row in the criteria range represents an OR condition.

That means:

Same row → AND

Different rows → OR

Syntax

```
=DSUM(database, field, criteria)
```

Example Database

Department	Gender	Country	Salary
Marketing	Female	India	50000
Marketing	Male	USA	60000
Sales	Female	India	55000
Marketing	Female	USA	70000

Assume this table is in `A1:D5`.

Condition:

Sum Salary where Department = Marketing OR Department = Sales

Criteria Range (F1:F3)

Department
Marketing
Sales

Formula

```
=DSUM(A1:D5, "Salary", F1:F3)
```

This adds salaries for both Marketing and Sales.

Example 2: OR with Multiple Conditions

Condition:

(Department = Marketing AND Country = India) OR (Department = Marketing AND Country = USA)

Criteria Range

Department	Country
Marketing	India
Marketing	USA

Formula

```
=DSUM(A1:D5, "Salary", F1:G3)
```

This works because:

Row 1 → Marketing AND India

Row 2 → Marketing AND USA

Different rows → OR

Important Rule to Remember

Structure	Meaning
Same row	AND
Different rows	OR

DSUM with AND Criteria

In **DSUM**, when multiple conditions are written **in the same row of the criteria range**, Excel treats them as **AND conditions**.

That means:

Same row → **AND**

Different rows → **OR**

Syntax

```
=DSUM(database, field, criteria)
```

- **database** → Full table including headers
- **field** → Column to sum (label in quotes or column number)
- **criteria** → Range containing headers and conditions

Example Database

Department	Gender	Country	Salary
Marketing	Female	India	50000
Marketing	Male	USA	60000
Sales	Female	India	55000
Marketing	Female	USA	70000

Assume this is in **A1:D5**.

Condition:

Department = Marketing AND Gender = Female

Criteria Range (F1:G2)

Department	Gender
Marketing	Female

Both conditions are written in the **same row**, so Excel applies AND logic.

Formula

```
=DSUM(A1:D5,"SaLary",F1:G2)
```

This will sum salaries where:

- Department is Marketing
- AND Gender is Female

From the table, it will include:

- 50000 (Marketing, Female, India)
- 70000 (Marketing, Female, USA)

Total = 120000

Key Rule

Criteria Layout	Logic Applied
Same row	AND
Different rows	OR



These are **List Functions** in Excel. They work on structured tables and calculate results based on criteria.

All of them follow the same structure:

```
=DFUNCTION(database, field, criteria)
```

Where **DFUNCTION** can be:

- **DAVERAGE**
- **DCOUNT**
- **DMAX**
- **DMIN**

1. DAVERAGE

Calculates the **average** of values in a column that meet specific criteria.

Syntax

```
=DAVERAGE(database, field, criteria)
```

Example

```
=DAVERAGE(A1:D100, "Salary", F1:G2)
```

A1:D100 → Full table including headers

- "Salary" → Column to average

F1:G2 → Criteria range

Returns the average salary matching the criteria.

2. DCOUNT

Counts **numeric values** in a column that meet criteria.

Syntax

```
=DCOUNT(database, field, criteria)
```

Example

```
=DCOUNT(A1:D100,"Salary",F1:G2)
```

Counts how many numeric Salary values satisfy the criteria.

Important:

- Counts only numbers
- If the column contains text, it will not count those cells

3. DMAX

Returns the **maximum value** in a column that matches criteria.

Syntax

```
=DMAX(database, field, criteria)
```

Example

```
=DMAX(A1:D100,"Salary",F1:G2)
```

Returns the highest salary that meets the criteria.

4. DMIN

Returns the **minimum value** in a column that matches criteria.

Syntax

```
=DMIN(database, field, criteria)
```

Example

```
=DMIN(A1:D100,"Salary",F1:G2)
```

Returns the lowest salary that meets the criteria.

Understanding the Arguments

1. database



- The entire data table
- Must include headers in the first row
- Example: A1:D100

2. field

- The column to perform the calculation on
- Can be:
- Column label in quotes → "Salary"

Column number → 4

3. criteria

- A separate range with conditions
- Must include matching headers

Same row → AND condition

Different rows → OR condition

Summary Table

Function	Purpose
DAVERAGE	Average of matching records
DCOUNT	Count of numeric matching records
DMAX	Maximum value of matching records
DMIN	Minimum value of matching records

The `SUBTOTAL` function is used to perform calculations on filtered data. It can automatically ignore hidden rows depending on the function number you use.

Syntax

```
=SUBTOTAL(function_num, ref1, [ref2], ...)
```

Arguments

1. **function_num** A number that tells Excel which calculation to perform.
2. **ref1** The range of cells you want to calculate.
3. **[ref2]** Optional additional ranges.

Function Numbers

SUBTOTAL supports different operations using numbers.

Basic Set which includes Manually Hidden Rows

Function	Number
AVERAGE	1
COUNT	2
COUNTA	3
MAX	4
MIN	5
SUM	9

Ignore Manually Hidden Rows

If you want to ignore both:

- Filtered rows
- Manually hidden rows

Use these numbers instead:



Function	Number
AVERAGE	101
COUNT	102
COUNTA	103
MAX	104
MIN	105
SUM	109

Example

Sum of Visible Salaries

```
=SUBTOTAL(9, D2:D100)
```

If you apply a filter, this will sum only the visible rows.

Average of Visible Salaries (Ignoring Hidden Rows)

```
=SUBTOTAL(101, D2:D100)
```

This ignores:

- Filtered rows
- Manually hidden rows

SUBTOTAL is ideal when you want calculations to automatically adjust based on filters applied to your dataset.



Data Validation is used to control what type of data users can enter into a cell. It helps maintain data integrity and prevents errors by restricting input to specific types, ranges, or lists.

How to Open Data Validation

We can access it from:

Data tab → Data Tools → Data Validation

Creating a Basic Dropdown in Excel

Let's create a dropdown list for selecting a department.

Example

We want to create a dropdown for **Department** selection.

First, type the list anywhere in the sheet:

IT
HR
Sales
Marketing

Steps

1. Select the cell where you want the dropdown (for example, **A1**)

[Go to Data tab](#) → [Data Validation](#)

2. In the **Settings** tab:
3. Allow: **List**
4. Source: Select the cells where you typed the list
5. Click **OK**

Result

- A dropdown arrow appears in the selected cell
- Users can only choose from the given options

Creating Whole Number Min-Max Validation

Let's create a validation that allows only whole numbers between 0 and 100.

Steps

1. Select the cells (for example, B1:B10)

Go to Data tab → Data Validation

2. In **Settings**:

3. Allow: Whole Number

4. Data: Between

5. Minimum: 0

6. Maximum: 100

7. Click **OK**

Result

- Only numbers between 0 and 100 can be entered
- If a value outside this range is entered, Excel shows an error message



Input Message and **Error Alert** are features in Data Validation that guide users while entering data and show warnings when invalid data is entered.

How to Open Data Validation

We can access it from:

Data tab → Data Tools → Data Validation

Input Message

Input Message displays a small message when a user selects a cell.

Steps

1. Select the validated cell

[Go to Data tab](#) → [Data Validation](#)

2. Open the **Input Message** tab
3. Enter:
4. Title
5. Input Message
6. Click **OK**

Result

- A message appears when the cell is selected
- It guides the user on what to enter

Error Alert

Error Alert shows a warning message when invalid data is entered.

Steps



1. Select the validated cell

Go to Data tab → Data Validation

2. Open the **Error Alert** tab
3. Enter:
4. Title
5. Error Message
6. Click **OK**

Result

- Excel shows an error message if invalid data is entered
- It prevents or warns against incorrect input



You can make a `DSUM` formula dynamic by linking it to a **Data Validation drop-down**.

When the selected value changes, the result updates automatically.

Example Data

Country	Salary
India	50000
USA	60000
India	55000
Canada	70000

Assume this table is in `A1:B5`.

Step 1: Create Criteria

In `D1:D2`:

Country
India

Step 2: Apply Data Validation

Select cell `D2`.

Go to:

Data → Data Validation

- **Allow:** List
- **Source:** Select the Country column (`A2:A5`)

Now `D2` has a drop-down list of countries.

Step 3: Use DSUM Formula

Now in 'D3' enter the formula:

```
=DSUM(A1:B5,"Salary",D1:D2)
```


If you select:

India	→	Total = 105000	USA	→	Total = 60000	Canada	→	Total = 70000
-------	---	----------------	-----	---	---------------	--------	---	---------------

When the drop-down value changes, the formula result changes automatically. This is how you can create dynamic formulas using Data Validation in Excel.



Importing data from text files allows you to bring large datasets into Excel in a structured and reliable way using Excel's **Get Data** feature.

As AI tools often generate datasets in text format, it's important to know how to import these files into Excel for analysis and manipulation.

How to Import a Text File into Excel

You can import a text file using the **Get Data** option.

Navigate to:

Data → Get Data → From File → From Text/CSV

Steps

1. Go to the **Data** tab in Excel.
2. Click **Get Data**.
3. Select **From File**.
4. Click **From Text/CSV**.
5. Choose the `.txt` file from your computer.
6. Click **Import**.

Excel will read the text file and load the data into the worksheet.

Why Import Data from Text Files

Many AI tools generate datasets in **text format**.

Since AI systems work best with text, large datasets may come as `.txt` files instead of structured spreadsheet files. Importing the text file into Excel allows you to work with that data properly inside Excel.

Why Not Use Text to Columns

Many users try to split text data using the **Text to Columns** feature, but it has several limitations.

- It can be **error prone**, especially when the dataset is large.
- It does **not work well with large datasets**.
- It does **not provide formatting while importing the data**.
- It does **not allow data transformation during the import process**.

Using the **Get Data → From Text/CSV** option is a more reliable method for bringing text datasets into Excel.

When exporting data from Microsoft Excel, several file formats are available. Each format has different capabilities, limitations, and compatibility considerations. Understanding these differences is important when sharing data, integrating with other systems, or preserving formatting and formulas.

1. XLS (Excel 97–2003 Workbook)

Overview

`.xls` is the legacy Excel file format used in versions of Excel before 2007. It is based on a **binary** format.

Key Features

- Supports **multiple worksheets**
- Preserves **formulas, formatting, charts, and pivot tables**
- Compatible with **older versions of Excel**
- Can contain **macros**

Limitations

- Maximum **65,536 rows** and **256 columns** per sheet
- Larger file sizes compared to modern formats
- Limited compatibility with modern tools and systems
- Considered **deprecated** for new workflows

When to Use

- When compatibility with **very old Excel versions (pre-2007)** is required.

2. XLSX (Excel Workbook)

Overview

`.xlsx` is the **default Excel format** introduced in Excel 2007. It uses the **Office Open XML** standard.

Key Features

- Supports **multiple worksheets**
- Preserves **formulas, formatting, tables, charts, and pivot tables**
- Supports up to **1,048,576 rows** and **16,384 columns**
- More **efficient storage** due to ZIP-based compression
- Compatible with most modern spreadsheet tools (Excel, Google Sheets, LibreOffice)

Limitations

- **Does not support macros**
- Slightly heavier to parse programmatically than CSV
- Some older systems cannot read it without conversion

When to Use

- For **most Excel workflows**
- When you need **formatting, formulas, charts, or multiple sheets**

3. CSV (Comma-Separated Values)

Overview

`.csv` is a **plain text format** used to store tabular data where each row is on a new line and values are separated by commas (or other delimiters).

Key Features

- Extremely **simple and lightweight**
- Widely supported across programming languages and software
- Ideal for **data exchange and data pipelines**
- Easily readable in text editors

Limitations

- **Only one sheet** (no multiple worksheets)
- **No formatting** (fonts, colors, borders are lost)
- **No formulas**
- **No charts or pivot tables**
- **No data types** (everything is treated as text)
- Special characters, commas, or quotes may require **escaping**
- Encoding issues may occur if not saved as **UTF-8**

Example

```
Name, Age, City
Alice, 25, Delhi
Bob, 30, Mumbai
Charlie, 28, Bangalore
```

When to Use

- Data transfer between **systems or programming languages**
- Importing data into **databases**
- Machine learning and data processing pipelines

Quick Comparison

Feature	XLS	XLSX	CSV
Multiple Sheets	Yes	Yes	No
Formatting	Yes	Yes	No
Formulas	Yes	Yes	No
Charts / Pivot Tables	Yes	Yes	No
Macros	Yes	No	No
File Type	Binary	XML-based	Plain Text
Row Limit	65,536	1,048,576	No strict format limit
File Size	Larger	Smaller	Smallest

Practical Rule of Thumb

- Use **XLSX** for normal Excel work.
- Use **CSV** when sharing data with software, databases, or code.
- Use **XLS** only when compatibility with **very old Excel versions** is necessary.



Pivot Tables are used to quickly summarize large datasets and generate reports without writing multiple formulas. They allow you to analyze data by selecting fields and organizing them into rows, columns, and values.

Why Pivot Tables Are Used

When working with large datasets, it can be difficult to manually calculate summaries.

For example, you may want to answer questions like:

- What is the **total salary for each country**?
- How many employees belong to a **specific category**?
- What is the **average value of a field**?

You could write formulas such as **DSUM**, but if the question changes, the formulas also need to be changed. Pivot Tables allow you to reorganize fields and generate different summaries instantly.

How to Create a Pivot Table

Navigate to:

Insert → PivotTable

Excel will automatically detect the data range and you will see two options:

New Worksheet
Existing Worksheet

Select **New Worksheet** and click **OK**. Excel will create an empty pivot table and open the **PivotTable Fields** panel.

Pivot Table Layout

The PivotTable Fields panel contains four areas:

Filters
Columns
Rows
Values

You build a pivot table by dragging fields into these sections.

- **Rows** organize data vertically
- **Columns** organize data horizontally
- **Values** perform calculations such as totals or averages
- **Filters** allow filtering of the entire report

Example: Total Salary by Country

Let's try to understand how to use pivot tables with an example.

Paste the following dataset into Excel starting at `A1`.

First Name	Last Name	Age	Gender	Country	Salary
Rohan	Sharma	25	Male	India	50000
Aisha	Khan	28	Female	India	60000
Liam	Brown	31	Male	Canada	65000
Emma	Wilson	29	Female	UK	70000
Noah	Taylor	26	Male	Australia	55000

```
First Name  Last Name  Age  Gender  Country  Salary
Rohan      Sharma    25   Male    India    50000
Aisha      Khan      28   Female  India    60000
Liam       Brown     31   Male    Canada   65000
Emma       Wilson    29   Female  UK       70000
Noah       Taylor    26   Male    Australia 55000
```

Steps

1. Select any cell inside the dataset

`Go to Insert` → `PivotTable`

2. Choose **New Worksheet** and click **OK**
3. Drag **Country** into the **Rows** section
4. Drag **Salary** into the **Values** section

Result

Excel automatically calculates the **Sum of Salary** for each country.

Example output:

Country	Sum of Salary
Australia	55000
Canada	65000
India	110000
UK	70000

Changing the Calculation Type

The calculation used in the **Values** area can be changed.

Right click the value field → Value Field Settings

You can choose different calculations such as:

- **Sum**
- **Average**
- **Max**
- **Count**

For example:

- **Sum** calculates the total salary
- **Average** calculates the average salary
- **Max** shows the highest salary
- **Count** counts how many records exist in that field

Removing Fields

You can easily remove fields from the pivot table if you no longer need them.

To remove a field from the pivot table:

- Drag it out of the field area

or * Uncheck it from the PivotTable Fields panel

The pivot table will update immediately.



Columns and Filters in Pivot Tables help organize and control how your summarized data is displayed.

They allow you to present the same dataset in different ways depending on the analysis you want to perform.

Creating a Pivot Table

First, create a pivot table from your dataset.

Navigate to:

Insert → PivotTable

Excel will detect the dataset automatically.

Choose:

New Worksheet

Click **OK** to create the pivot table.

Opening the Field List

If the field list is not visible:

PivotTable Analyze → Field List

This will open the **PivotTable Fields panel**, where you can manage all fields used in the pivot table.

Default Behavior of Values

When fields are placed in the **Values** area, Excel automatically chooses the calculation type.

- **Numeric fields** are summarized using **Sum**
- **Text fields** are summarized using **Count**

For example:

- If **Salary** is added to Values, Excel calculates the **Sum of Salary**
- If a text field is added, Excel counts how many records exist

Changing the Calculation

You can change how a value is calculated.

Steps

1. Right click the value inside the pivot table
2. Click **Value Field Settings**
3. Select the calculation you want

Common options include:

Sum
Count
Average
Max
Min

Using the Columns Field

The **Columns** area places a field across the top of the pivot table. Instead of grouping data vertically, it splits the results into separate columns. This helps compare categories side by side.

For example, if **Gender** is placed in Columns, the pivot table creates separate columns for **Male** and **Female** values.

Using the Filters Field

The **Filters** area allows you to filter the entire pivot table using a selected field. When a field is placed in **Filters**, a dropdown appears above the pivot table. You can choose a specific category and the pivot table will update to show only that selection.

Pivot tables are flexible and can be arranged in many ways. How you place fields in **Rows**, **Columns**, **Values**, and **Filters** depends on the question you want the data to answer.

The structure of the pivot table determines the **story the data tells**.

Renaming Value Fields

Sometimes the default field names are not very readable.

For example, Excel may display:

Sum of Salary

You can rename this to something more descriptive, like:

Total Salary

Steps

1. Right click the value field
2. Click **Value Field Settings**
3. Edit the **Custom Name** field
4. Click **OK**

This allows you to give clearer labels to the values shown in the pivot table.



Date fields in Pivot Tables can be grouped to analyze data by different time periods such as days, months, or years.

This helps summarize date-based data more effectively.

Creating a Pivot Table

First create a pivot table from your dataset.

Navigate to:

Insert → PivotTable

Choose where you want the pivot table to appear and click **OK**.

Grouping Dates in a Pivot Table

When a dataset contains a **date column**, Pivot Tables allow you to group those dates into different time intervals.

To group dates:

PivotTable Analyze → Group Field

Excel will open the **Grouping** window.

Here you can choose how the dates should be grouped.

Available Grouping Options

Dates in a pivot table can be grouped into different time units such as:

Days
Months
Years

Selecting one or more of these options allows Excel to summarize the data based on those time periods.

For example, instead of showing every individual date, the pivot table can show totals for each **month** or **year**.

How Grouping Helps

Grouping dates makes it easier to analyze trends over time.

Instead of working with many individual dates, the pivot table summarizes the data into larger time periods like **months or years**, making the report easier to read.



Pivot Tables allow you to format values so the data is easier to read and understand.

Formatting can be applied directly from the **Value Field Settings**.

Opening Value Field Settings

To format values in a Pivot Table:

1. Right click any value inside the pivot table
2. Click **Value Field Settings**

This opens the settings window for that value field.

Applying Number Formatting

Inside the **Value Field Settings** window, you can apply number formatting.

Navigate to:

Value Field Settings → Number Format

This opens the standard **Format Cells** window.

Using Custom Date Format

To apply a custom format:

Number Format → Custom

Enter the format:

d mmm yy

This format displays dates in the following style:

5 Jan 24
12 Mar 25
8 Aug 23

This makes the date values more compact and easier to read inside the pivot table.

Pivot Tables allow you to change how values are calculated and displayed. Instead of only showing totals, you can modify the calculation to show counts, averages, or percentages.

Opening Value Field Settings

To modify how a value is calculated:

1. Right click any value inside the pivot table
2. Click **Value Field Settings**

This opens the settings window for the selected field.

Changing the Calculation

Inside **Value Field Settings**, you can choose how Excel should calculate the values.

Common calculation options include:

Sum
Count
Average
Max
Min

Select the calculation you want and click **OK**.

The pivot table will update immediately with the new calculation.

Showing Values as Percentage of Row Total

You can also display values as a percentage instead of raw numbers.

Steps

1. Drag the same field into the **Values** area again
2. Right click the new value in the pivot table
3. Click **Value Field Settings**
4. Go to **Show Values As**
5. Select:

% of Row Total

Click **OK**.

Example

Suppose the pivot table shows:

Sum of Salary

If you change the calculation to **Count**, the pivot table will show how many records exist instead of the total salary.

If you use **% of Row Total**, the pivot table will display each value as a percentage of the row total instead of showing the raw numbers.



Developer Mode in Excel unlocks advanced tools used for automation, macros, VBA programming, and form controls. It adds the **Developer tab** to the ribbon, allowing users to create macros, access the Visual Basic Editor, and build interactive Excel solutions.

How to Enable Developer Mode

Navigate to:

File → Options → Customize Ribbon → Developer (check the box) → OK

Steps to Enable Developer Mode

1. Open **Microsoft Excel**.
2. Click **File** in the top-left corner.
3. Select **Options** at the bottom of the menu.
4. In the Excel Options window, click **Customize Ribbon**.
5. In the right-side list of **Main Tabs**, find **Developer**.
6. Check the **Developer** checkbox.
7. Click **OK**.

Result:

- The **Developer tab** appears in the Excel ribbon.
- It appears alongside tabs like **Home, Insert, Page Layout, Formulas, Data, and View**.

Tools Available in the Developer Tab

The **Developer tab** contains tools used for automation and programming inside Excel.

Main options include:

Visual Basic
Macros
Record Macro
Insert (Form Controls & ActiveX Controls)
Add-ins

These tools allow you to write code, automate repetitive tasks, and build interactive spreadsheets.

Opening the VBA Editor

Navigate to:

Developer → Visual Basic

This opens the **VBA Editor**, where you can write programs that control Excel.

Creating a Simple Macro

A **macro** is a small program written in **VBA (Visual Basic for Applications)** that automates tasks in Excel.

Steps

Go to Developer → Visual Basic

1. In the VBA window click **Insert**
2. Select **Module**
3. A new module window will appear
4. Paste the following code

Macro Code

```
Sub HelloWorld()  
    MsgBox "Hello from Harry"  
End Sub
```

Running the Macro

Navigate to:

Developer → Macros → Select Macro → Run

Result:

- Excel shows a popup message
- The message displayed is **Hello from Harry**

Writing Data to a Cell with Macro

```
Sub WriteToCell()  
    Range("A1").Value = "Harry"  
    MsgBox "Done! Hello from Harry"  
End Sub
```

What Happens When It Runs

- Excel writes **Harry** into cell **A1**
- A popup message appears confirming the action

This demonstrates how VBA can directly control worksheet cells.

Generating Sample Kirana Store Data

Macros can also generate sample datasets automatically. This is useful when practicing **sorting**, **filtering**, **PivotTables**, or **charts**.

Steps

[Go to Developer](#)[Visual Basic](#)[Click Insert](#)[Module](#)

1. Paste the following macro
2. Run the macro

Macro Code

```
Sub GenerateKiranaSampleData()  
  
    Range("A1").Value = "Product Name"  
    Range("B1").Value = "Category"  
    Range("C1").Value = "Brand"  
    Range("D1").Value = "Quantity"  
    Range("E1").Value = "Cost Price"  
    Range("F1").Value = "Selling Price"  
    Range("G1").Value = "Supplier"  
    Range("H1").Value = "Purchase Date"  
  
    Range("A2").Resize(9, 8).Value = Array( _  
        Array("Salt", "Snacks", "Surf Excel", _  
            17, 199.98, 262.13, _  
            "Om Trading", "09-Aug-2025"), _
```

```
Array("Biscuits","Beverages","Fortune", _  
      29,59.23,68.25, _  
      "Agarwal Distributors","25-Jun-2025"), _  
  
Array("Toor Dal","Snacks","Amul", _  
      18,187.16,255.11, _  
      "Agarwal Distributors","19-Aug-2025"), _  
  
Array("Wheat Flour","Cleaning","Surf Excel", _  
      40,28.42,41.92, _  
      "Agarwal Distributors","21-May-2025"), _  
  
Array("Milk Powder","Pulses","Fortune", _  
      19,193.25,237.10, _  
      "Gupta Wholesalers","13-May-2025"), _  
  
Array("Poha","Personal Care","Britannia", _  
      23,157.54,183.26, _  
      "Gupta Wholesalers","28-Dec-2025"), _  
  
Array("Toor Dal","Beverages","Aashirvaad", _  
      41,162.78,201.06, _  
      "Agarwal Distributors","25-Dec-2025"), _  
  
Array("Sugar","Beverages","Patanjali", _  
      54,116.11,162.52, _  
      "Agarwal Distributors","08-Nov-2025"), _  
  
Array("Noodles","Beverages","Aashirvaad", _  
      17,90.28,128.25, _  
      "Om Trading","03-Jun-2025") _  
)  
  
End Sub
```

Result

When the macro runs, Excel automatically fills the worksheet with structured kirana store inventory data.

Generated Data

Product Name	Category	Brand	Quantity	Cost Price	Selling Price	Supplier	Purchase Date
Salt	Snacks	Surf Excel	17	199.98	262.13	Om Trading	09-Aug-2025
Biscuits	Beverages	Fortune	29	59.23	68.25	Agarwal Distributors	25-Jun-2025
Toor Dal	Snacks	Amul	18	187.16	255.11	Agarwal Distributors	19-Aug-2025
Wheat Flour	Cleaning	Surf Excel	40	28.42	41.92	Agarwal Distributors	21-May-2025
Milk Powder	Pulses	Fortune	19	193.25	237.10	Gupta Wholesalers	13-May-2025
Poha	Personal Care	Britannia	23	157.54	183.26	Gupta Wholesalers	28-Dec-2025
Toor Dal	Beverages	Aashirvaad	41	162.78	201.06	Agarwal Distributors	25-Dec-2025
Sugar	Beverages	Patanjali	54	116.11	162.52	Agarwal Distributors	08-Nov-2025
Noodles	Beverages	Aashirvaad	17	90.28	128.25	Om Trading	03-Jun-2025

Recording a Macro in Excel allows you to capture a sequence of actions and replay them automatically. It is useful for repetitive tasks such as formatting data, applying formulas, or cleaning datasets. Excel records each step you perform and converts it into a macro that can be run later.

Where to Find the Option

Developer → Code Group → Record Macro

If the **Developer** tab is not visible, enable it first:

File → Options → Customize Ribbon → Developer (check the box) → OK

Example:

Suppose you receive raw sales data and you always perform the same formatting steps:

Product	Sales	Profit
Laptop	45000	6000
Mouse	8000	1200
Keyboard	12000	2500

Every time you want to:

- Make the header bold
- Apply currency format to Sales and Profit
- Auto fit column width

Instead of repeating these steps manually, you can record a macro.

Steps to Record a Macro

1. Open **Excel** and go to the **Developer** tab.
2. Click **Record Macro**.
3. Enter a name for the macro, for example `FormatSalesData`.
4. Choose where to store the macro (usually **This Workbook**).
5. Click **OK** to start recording.
6. Perform the actions you want Excel to remember.
7. Select the header row and apply **Bold**.
8. Select Sales and Profit columns and apply **Currency format**.
9. Auto fit column widths.
10. Go back to the **Developer** tab and click **Stop Recording**.

Running the Recorded Macro

To apply the same formatting later:

Developer → Macros → Select Macro → Run

Excel will instantly repeat the recorded actions on the selected data. This saves time and ensures consistency across your spreadsheets.



VBA (Visual Basic for Applications) is a programming language built into Excel that lets you automate tasks.

Instead of doing repetitive work manually, you can write code once and let Excel do it for you.

Opening the VBA Editor

1. Open Excel
2. Press `ALT + F11` or

`Go to Developer tab` → `Visual Basic` `Click Insert` → `Module`

This opens the coding environment where you write and store macros.

Your First VBA Code

Let's start with a simple example that shows a message box:

```
Sub HelloWorld()  
    MsgBox "Hello, World!"  
End Sub
```

`Sub` defines a macro (procedure). `MsgBox` displays a popup message.

When you run this macro, Excel shows a dialog box with the text.

MsgBox with Options

```
Sub ShowMessage()  
    MsgBox "Task Completed!", vbInformation, "Status"  
End Sub
```

Here you enhance the message box:

- `vbInformation` adds an info icon
- `"Status"` becomes the title of the popup

Useful for user-friendly notifications after a task completes.

Taking User Input

```
Sub AskName()  
    Dim name As String  
    name = InputBox("Enter your name:")  
  
    MsgBox "Hello " & name  
End Sub
```

`InputBox` lets the user type a value. The input is stored in a variable and then used in the message.

This is the simplest way to make your macros interactive.

Variables

```
Dim age As Integer  
age = 25  
  
MsgBox "Your age is " & age
```

Variables store data temporarily while your code runs.

- `Dim` declares a variable
- `Integer` stores whole numbers
- `String` stores text

Using variables makes your code flexible and reusable.

If Else

```
Sub CheckAge()  
    Dim age As Integer  
    age = InputBox("Enter your age:")  
  
    If age >= 18 Then  
        MsgBox "You are an adult"  
    Else  
        MsgBox "You are under 18"  
    End If  
End Sub
```

If Else is used for decision-making.

- Condition is checked (`age ≥ 18`)
- If true → first block runs
- If false → second block runs

This is how you control logic in VBA.

Loops

For Loop

```
Sub CountNumbers()  
    Dim i As Integer  
  
    For i = 1 To 5  
        MsgBox i  
    Next i  
End Sub
```

A For loop repeats code a fixed number of times.

- Starts at 1
- Ends at 5
- Runs 5 times

Useful when you know exactly how many repetitions you need.

For Loop with Step

```
Sub SkipNumbers()  
    Dim i As Integer  
  
    For i = 1 To 10 Step 2  
        MsgBox i  
    Next i  
End Sub
```

Step 2 increases the value by 2 each time.

So instead of 1,2,3..., it goes 1,3,5,7,9.

Useful when working with intervals or skipping data.

Loop Through Excel Cells

```
Sub FillCells()  
    Dim i As Integer  
  
    For i = 1 To 5  
        Cells(i, 1).Value = i  
    Next i  
End Sub
```

This interacts directly with Excel.

- `Cells(i, 1)` means row *i*, column 1 (Column A)
- It fills A1 to A5 with numbers 1 to 5

This is where VBA becomes powerful for automation.

While Loop

```
Sub WhileExample()  
    Dim i As Integer  
    i = 1  
  
    Do While i <= 5  
        MsgBox i  
        i = i + 1  
    Loop  
End Sub
```

A `Do While` loop runs **as long as a condition is true**.

- Starts with `i = 1`

Keeps running until `i` → 5

Use this when the number of repetitions depends on a condition, not a fixed count.

Running Macro

To run a macro: 1. Press `ALT + F8` 2. Select macro 3. Click Run

This opens the macro list and lets you execute any VBA procedure you've written.

Buttons are a powerful way to make your Excel sheets interactive. They allow users to trigger VBA code with a single click, making it easier to perform tasks without navigating through the Developer tab or running macros manually.

Where to Find the Option

You can find the option to insert buttons in Excel under the Developer tab:

Developer → Insert → Form Controls

- You will see multiple options like:
- Button
- Combo Box
- Check Box
- Option Button
- These controls can be used to build interactive Excel dashboards

Creating a Button

Step 1: Insert Button

Developer → Insert → Button (Form Control)

- Click on **Button**
- Drag on the sheet to place it

Step 2: Assign Macro

- After placing the button, Excel will ask to assign a macro
- Click **New**
- This opens the VBA editor

VBA Code Example

Let's create a simple example.

```
Sub Button1_Click()  
    MsgBox "5 Green Cars"  
End Sub
```


This code shows a message box when the button is clicked.

Renaming the Button

- Right click on the button
- Click **Edit Text**
- Change the name

Example:

Find Green Cars

Customizing the Button

Right click → Format Control

- You can:
- Change font
- Resize button
- Adjust alignment

This helps make your Excel UI cleaner and more user-friendly.



Shapes and form controls help you design clean and interactive Excel dashboards. Proper formatting makes your UI look professional and easier to use.

Enabling/ Disabling Developer Tab

If you want to use shapes and form controls, you need to enable the Developer tab:

You can enable it by following these steps:

Right Click on Ribbon → Customize the Ribbon → Check "Developer" → OK

Inserting Shapes / Controls

Go to the Developer tab to insert shapes and form controls:

Developer → Insert → Form Controls

- You can insert:
- Button
- Option Button (Radio Button)
- Check Box

Place them anywhere on the sheet.

Aligning Multiple Shapes

You can align multiple shapes to make your dashboard look neat and organized.:

Step 1: Select Multiple Shapes

- Hold **CTRL**
- Click on multiple buttons or option buttons

Step 2: Align Shapes

Shape Format → Align

- Click:
- **Align Top** (to align horizontally)
- **Align Left** (to align vertically)

This ensures all controls are perfectly aligned.

Removing Gridlines for Clean UI

You can remove gridlines to give your dashboard a cleaner look:

View → Uncheck Gridlines

This removes background lines and gives a cleaner appearance.

Additional Shape Formatting Tips

Right click on shape → Format Control / Format Shape

You can:

- Change fill color
- Remove outline
- Adjust size
- Change font style

You can also combine aligned shapes + no gridlines to create a professional dashboard interface that feels like a real application inside Excel.



Form controls in Excel allow you to collect input from users in a structured and controlled way. These controls are available inside the Developer tab and can be linked to cells for storing values.

Where to Find Form Controls

Developer → Insert → Form Controls

You will see options like:

- Button
- Combo Box
- Check Box
- Spin Button
- List Box
- Option Button

Combo Box

A combo box allows users to select a value from a dropdown list.

Developer → Insert → Combo Box

Setup

Right click → Format Control

- Set:
- **Input Range** → Select list of items
- **Cell Link** → Select a cell

Key Behavior

- Dropdown shows actual values (e.g. cities)
- Linked cell stores position number:

First item → 1

Second item → 2

Third item → 3

This is useful for compact data storage.

Check Box

Used for yes/no or true/false input.

Developer → Insert → Check Box

Setup

Right click → Format Control

- Set **Cell Link**

Output

- Checked → TRUE
- Unchecked → FALSE

Spin Button

Used to increase or decrease numeric values.

Developer → Insert → Spin Button

Setup

Right click → Format Control

Set:

- Minimum Value
- Maximum Value
- Incremental Change
- Current Value
- Cell Link

The linked cell updates as you click the arrows.

List Box

Displays a list of options directly on the sheet.

Developer → Insert → List Box

Setup

Right click → Format Control

- Input Range → Select list
- Cell Link → Select a cell

Behavior

- Shows all options at once
- Linked cell stores selected index (1, 2, 3...)

Option Button/ Radio Button

Used when only one option should be selected.

Developer → Insert → Option Button

Setup

Right click → Format Control

- Set **Cell Link**

Behavior

- Selected option returns a number in linked cell
- Only one option can be selected at a time

Button

You can use buttons to trigger actions, usually VBA macros.

Developer → Insert → Button

- Used to trigger actions (usually VBA macros)

Can be renamed by right click → Edit Text

Using these form controls, you can create interactive dashboards and forms in Excel without needing complex VBA code.



Power Query is a powerful tool in Excel that allows you to connect, combine, and refine data from various sources. It provides an intuitive interface for data transformation and cleaning without needing to write complex formulas or VBA code.

Why Use Power Query?

Power Query simplifies the process of data preparation, making it easier for users to work with large datasets and perform complex transformations without needing advanced technical skills. It also helps maintain data integrity and consistency by allowing you to automate repetitive tasks.

The main idea of Power Query is automation of repetitive data cleaning tasks. Instead of spending time manually preparing data every day, you define the steps once and let Excel handle the rest.

Problem with Manual Data Cleaning

Manually cleaning and preparing data can be time-consuming and error-prone, especially when dealing with large datasets. It often involves repetitive tasks such as filtering, sorting, merging, and transforming data, which can lead to inconsistencies and mistakes. Power Query addresses these issues by automating the data preparation process, allowing you to define the steps once and apply them to new data with ease.

Getting Started with Power Query

1. Go to the **Data** tab in Excel.
 2. Click on **Get Data** to see the available data sources.
 3. Select the desired data source and follow the prompts to connect.
 4. Use the Power Query Editor to apply transformations such as filtering, sorting, merging, and more.
 5. Once you are satisfied with the transformations, click **Close & Load** to load the data back into Excel.
-



1. Introduction

In this video, we'll understand the complete Power Query workflow using just four steps: Connect, Transform, Combine, and Load.

This is all you need to clean and automate your data workflow in Excel.

2. Connect

Start by bringing your data into Power Query.

[Go to Data](#) → [Get Data](#)

- Select your source:
- Excel
- CSV
- Folder
- Database

This creates a connection to your data, not a copy.

3. Transform

Now clean and shape your data inside Power Query.

- Remove unnecessary columns
- Filter rows
- Change data types
- Rename columns

Every action is saved in **Applied Steps**

These steps will automatically run again when data updates.

4. Combine

Now bring multiple datasets together.

Two main ways:

- **Append:** Stack similar tables
- **Merge:** Join related tables using a common column

This lets you work with multiple files or tables as one.

5. Load

Finally, send your processed data to Excel.

- Click **Close & Load**
- Choose where to load:
 - Worksheet
 - Data Model

Your data is now ready for analysis.

Basically, the entire workflow can be summarized in these four steps:

1. Connect → Import data

Transform



Clean data

Combine



Merge data

Load



Use data

Master these four steps, and you can handle any data task in Excel.



Let's see how Power Query helps you clean data once and then refresh it automatically when the source updates.

We'll use a simple sales dataset to understand this.

Importing Data into Power Query

Start by bringing your dataset into Power Query.

- Go to **Data** → **Get Data** → **From File** → **From Excel Workbook**
- Select your file and choose the sheet
- Click **Transform Data**

Example Data (Before Cleaning)

Row	Name	Product	Sales	Date
1	(blank)	(blank)	(blank)	(blank)
2	(blank)	(blank)	(blank)	(blank)
3	Name	Product	Sales	Date
4	Rohan	Laptop	55000	01-01-2026
5	Amit	Phone	20000	02-01-2026
6	Neha	Tablet	15000	03-01-2026

This data is messy and not ready for analysis.

Understanding Applied Steps

Once inside Power Query, you'll see **Applied Steps** on the right.

- **Source** → File selected
- **Navigation** → Sheet selected

Promoted Headers → May or may not be correct

- **Changed Type** → Auto-detected types

Every transformation you apply will be added here.

Cleaning the Data

Now, let's fix the dataset step by step.

- Remove the top 2 rows so actual data comes up
- Promote the first row as headers
- Change data types:

Sales

→

Whole Number

Date

→

Date

Example Data (After Cleaning)

Name	Product	Sales	Date
Rohan	Laptop	55000	01-01-2026
Amit	Phone	20000	02-01-2026
Neha	Tablet	15000	03-01-2026

Each of these actions is saved as a step.

Loading Data into Excel

- Click **Close & Load**
- Data will be loaded into a worksheet

Now you have a clean and structured table.

Refreshing Data

Now assume next week you get updated data in the same format:

Row	Name	Product	Sales	Date
1	(blank)	(blank)	(blank)	(blank)
2	(blank)	(blank)	(blank)	(blank)
3	Name	Product	Sales	Date
4	Rahul	Laptop	60000	08-01-2026
5	Priya	Phone	25000	09-01-2026

Instead of repeating all steps:

- Click **Refresh**

Power Query automatically:

- Removes top rows
- Promotes headers
- Fixes data types

And updates your clean table.

CodeWithHarry



Custom columns in Power Query let you create new columns using formulas so you can process and analyze data dynamically instead of manually editing it. This is useful for calculations, transformations, and deriving insights from your data.

Importing Data into Power Query

Start by loading your dataset into Power Query.

Data → Get Data → From File → From Excel Workbook

Select your file and click **Transform Data**.

Example Data

ID	Name	Joining Date	Salary
1	Rohan	2022-01-10	30000
2	Amit	2023-06-15	25000
3	Neha	2021-09-01	40000

Understanding Applied Steps

When the data loads, Power Query automatically creates steps like:

- Source
- Navigation
- Promoted Headers
- Changed Type

Each step is actual code running in the background.

Creating a Custom Column

Now let's try to calculate how many days ago each employee joined.

[Go to Add Column](#) → [Custom Column](#)

- Set column name: Days Since Joining

Enter this formula:



```
DateTime.LocalNow() - [Joining Date]
```

Click **OK**.

Fixing the Error

You will see an error because the data types don't match. `DateTime` and `Date` cannot be directly subtracted.

- `DateTime.LocalNow()` returns `DateTime`
- `[Joining Date]` is a `Date`

So we need to convert the `DateTime` to `Date` first.

```
Date.From(DateTime.LocalNow()) - [Joining Date]
```

Now both values are in the same format.

Understanding Applied Step

A new step appears in Applied Steps:

- **Added Custom Column**

This stores your formula and runs automatically every time data refreshes.

Result After Custom Column

ID	Name	Joining Date	Days Since Joining
1	Rohan	2022-01-10	1100+
2	Amit	2023-06-15	600+
3	Neha	2021-09-01	1300+

Editing the Custom Column

You can modify the formula anytime.

- Click the gear icon next to **Added Custom Column**
- Update the formula as needed

Changes apply instantly.

Refreshing Data with Custom Logic

If new data comes in the same format:

ID	Name	Joining Date
4	Rahul	2020-05-20
5	Sneha	2024-01-01

Click **Refresh**.

Power Query recalculates the custom column automatically.

Updated Output

ID	Name	Joining Date	Days Since Joining
4	Rahul	2020-05-20	1800+
5	Sneha	2024-01-01	400+



Conditional columns let you create new columns using logic. You define conditions, and Power Query assigns values based on those conditions. This is commonly used for categorization like Age groups, Sales performance, etc.

Importing Data into Power Query

Let's start by loading your dataset.

Data → Get Data → From File → From Excel Workbook

Select your file and click **Transform Data**.

Example Data

Name	Age
Rohan	17
Amit	22
Neha	15
Priya	30

Creating a Conditional Column

Now classify people based on Age.

Go to Add Column → Conditional Column

- Configure like this:
- New column name: Category

If Age < 25 → Kid

Else if Age ≥ 35 → Senior Citizen

Else if Age ≥ 25 → Adult

Else → Error

Click **OK**.

This matches what you see in the editor and generates logic like:



```
if [Age] < 25 then "Kid"  
else if [Age] ≥ 35 then "Senior Citizen"  
else if [Age] ≥ 25 then "Adult"  
else "Error"
```

Understanding Applied Steps

A new step is added to the **Applied Steps** pane:

- **Added Conditional Column**

This step stores your full logic and runs automatically during refresh.

Result After Applying Condition

Name	Age	Category
Rohan	17	Kid
Amit	22	Kid
Neha	15	Kid
Priya	30	Adult

Editing the Condition

You can update the logic anytime.

- Click the gear icon next to **Added Conditional Column**
- There you can modify conditions or add new ones.

And changes will reflect instantly in the table.

Refreshing Data with Conditions

Now assume new data comes in the same format:

Name	Age
Rahul	40
Sneha	27

Click **Refresh**.

Chapter 37: Conditional Columns in Power Query

SECTION 07

Power Query automatically applies the same logic.

Updated Output

Name	Age	Category
Rahul	40	Senior Citizen
Sneha	27	Adult



Merging and appending are used to combine multiple datasets in Power Query. Merging joins data side by side using a common column, while appending stacks data one below another.

We'll use two datasets to understand both concepts.

Importing Multiple Queries

First, load both datasets into Power Query.

Data → Get Data → From File → From Excel Workbook

- Import the first file → click **Transform Data**
- Repeat for the second file

Now you'll see two queries in Power Query, for example:

- Sample_Data
- Sample_Data_2

Understanding the Scenario

Assume:

- First dataset contains first 30 records
- Second dataset contains last records

Now we want to combine them.

Merging Queries (Joining Data)

Merge is used when you want to combine columns based on a common field.

[Go to Home](#) → [Merge Queries](#)

- Select:
- First table: Sample_Data
- Second table: Sample_Data_2
- Select a common column (like Name or ID) in both tables
- Choose Join Type
- Click **OK**

Power Query creates a new merged column.

Now:

- Click expand icon
- Select columns you want to bring from second table

Result (Merge)

Name	Department	Salary	Extra Info

Data is joined horizontally based on matching values.

Appending Queries

Append is used when both datasets have the same structure and you want to combine rows.

[Go to Home](#) → [Append Queries](#)

- Select:
- Two Tables
- Choose Sample_Data and Sample_Data_2
- Click **OK**

Result

Name	Department	Salary
Data from first file		
Data from second file		

All rows are stacked into one table.

Understanding Applied Steps

Power Query adds steps like:

- **Merged Queries**
- **Appended Queries**

These steps are saved and will run automatically during refresh.

When to Use What

Use **Merge** when:

- You need to join related data
- You have a common key like ID or Name

Use **Append** when:

- Data structure is same
- You want to combine rows

Refreshing Combined Data

If both source files update:

- Click **Refresh**

Power Query will:

- Re-run merge or append
- Update the final dataset automatically

This way, you can combine multiple data sources without repeating the process every time.



In Power Query, every action you perform is saved as a step. The key idea is simple: you don't redo work, you just edit the step. This makes it easy to adjust your data cleaning process without starting from scratch. You can change values, add new steps, or even use AI to generate steps for you. The flow will update automatically based on your edits.

Example

Let's say you have a messy dataset with extra rows at the top and bottom, and you want to clean it up.

Row	Name	Amount
1	(blank)	(blank)
2	(blank)	(blank)
3	Name	Amount
4	Rohan	5000
5	Amit	3000
6	*	*

Load this into Power Query:

Data → Get Data → From File → From Excel Workbook

Click **Transform Data**.

Working with Steps

Start cleaning the data by removing unwanted rows:

- Remove top rows so headers come up
- Use **First Row as Headers**
- Remove unwanted bottom rows

Each action appears in **Applied Steps** on the right.

Now instead of repeating the process, just edit the step. If you removed 2 rows but now need 3, open that step and change the value. Power Query will update everything automatically.

Using AI to Help

You can also generate steps using AI. For example, if you want to remove the top 2 rows, ask something like: "Remove top 2 rows from my data". The AI will generate the formula for you. Just copy it and add it using the **fx** button.

Sometimes the code may not work directly. In that case you can edit the generated code to fit your needs. This way, you can leverage AI to speed up your workflow while still having full control over the steps.

Editing in Practice

While working, you might run into errors after changing or deleting a step. This usually happens because one step depends on another. For example, if headers are removed, later steps using column names can break.

The fix is simple. Reapply the missing step or adjust the order, and the flow will work again. You can also tweak values anytime, like changing how many rows to remove, without redoing anything.

Once your steps are set, click **Close & Load**. When new data comes, just hit **Refresh** and all steps, including AI-generated ones, will run automatically.



Microsoft Copilot is an AI assistant built into Excel that helps you work faster. You can ask questions in plain English to generate formulas, analyze data, create charts, summarize information, and automate common tasks.

Opening Copilot

1. Open your workbook in Excel.
2. Go to the **Home** tab.
3. Click the **Copilot** button in the ribbon.
4. The Copilot pane will open on the right.
5. Type your request and press **Enter**.

Copilot works best when your data is organized in a table with clear column headings.

What Can Copilot Do?

FORMULAS

Generate Excel formulas from simple English instructions.

CHARTS

Create charts and visualizations from your data.

ANALYSIS

Identify trends, summaries, top values, and important insights.

AUTOMATION

Format tables, sort data, filter records, and perform repetitive tasks.

Example Prompts

Formula

Calculate total sales.

Chart

Create a bar chart.

Pivot Table

Create a PivotTable.

Summary

Summarize this dataset.

Insights

Find sales trends.

Formatting

Format this table.

Top Values

Show top 5 products.

Average

Calculate average sales.

Tips for Better Results

TIP 1

Write clear and specific prompts instead of vague requests.

TIP 2

Review Copilot's suggestions before applying them.

TIP 3

Ask follow-up questions to refine the results.

TIP 4

Use Copilot to learn new Excel formulas and features.

CodeWithHarry



PowerPivot is an Excel add-in for analyzing large datasets. It lets you build data models, create relationships between multiple tables, and perform advanced calculations using **DAX (Data Analysis Expressions)**.

Why Use PowerPivot?

LARGE DATA

Analyze millions of rows without slowing down your workbook.

RELATIONSHIPS

Connect multiple tables without using VLOOKUP or manual copying.

DAX

Create advanced calculations, measures, and custom formulas.

REPORTS

Build interactive PivotTables and dashboards from connected data.

PowerPivot is commonly used by business analysts and data professionals to analyze large datasets and create advanced reports.

Enabling PowerPivot

1. Open Excel and click the **File** tab.
2. Select **Options** → **Add-ins**.
3. Choose **COM Add-ins** from the **Manage** drop-down and click **Go**.
4. Enable **Microsoft PowerPivot for Excel**.
5. Click **OK**.
6. The **PowerPivot** tab will appear on the Excel ribbon.

What Can You Do with PowerPivot?

Import

Load data from multiple sources.

Model

Create relationships between tables.

DAX

Write advanced calculations.

Measures

Create reusable calculations.

PivotTables

Analyze connected data.

Reports

Build interactive dashboards.

Performance

Handle large datasets efficiently.

Insights

Discover trends and patterns.

A data model in Excel lets you connect multiple tables using relationships so you can analyze them together without merging everything into one sheet. Instead of building one large dataset, you keep tables separate and link them using common columns like IDs.

This keeps data organized, reduces duplication, and makes analysis faster.

Example Structure

Let's say you have three tables:

- **Employees** Contains employee details and a Project ID (PID)
- **Projects** Contains project details and a Text Tag ID (TID)
- **Text Tag** Contains technology or tag details

These tables are connected as follows:

Employees → Projects using PID

Projects → Text Tag using TID

Why Use IDs for Relationships?

IDs act as stable connectors between tables.

Instead of repeating texts in multiple places, you store it once and refer to it using an ID like T001. This improves consistency, avoids typos, and keeps relationships reliable.

Where to Create Relationships in PowerPivot

PowerPivot Tab → Manage → Diagram View



Creating the Data Model

1. Ensure each table has a unique identifier (PID, TID).
2. Go to the **PowerPivot** tab in Excel.
3. Click **Manage**.
4. Switch to **Diagram View**.
5. Create relationships by dragging:

Employees[PID] → Projects[PID] Projects[TID] → Text Tag[TID]

Once connected, Excel treats these tables as a single data model.

Example Data

Employees Table

EmpID	Name	PID
E001	Rohan	P001
E002	Aman	P002
E003	Neha	P001
E004	Priya	P003

Projects Table

PID	Project Name	TID
P001	Website Development	T001
P002	Mobile App	T002
P003	Data Dashboard	T003

Text Tag Table

TID	Technology
T001	Python + HTML
T002	Flutter
T003	Power BI

How the Relationship Works

Each table links step by step:

```
Rohan → PID P001  
P001 → Website Development → T001  
T001 → Python + HTML
```

This chaining allows Excel to fetch related data across tables without merging them.

What You Can Do with This

You can now create a PivotTable using fields from all three tables. For example, you can directly see:

```
Employee → Project → Technology
```

All of this works because the relationships connect the tables behind the scenes. This is the power of data modeling in PowerPivot!



In the previous lecture, we built a **data model using multiple tables** and connected them using relationships.

Employees → Projects → Tech Stack

Each table holds different pieces of information, but they are linked using IDs like PID and TID.

The Problem

Even though the tables are connected, the **Employees table still looks incomplete**.

It only contains:

- Employee Name
- Project ID (PID)

But we cannot directly see:

- Project Name
- Deadline
- Technology used

The Goal

We want to **pull related data into the Employees table** without merging or duplicating data.

This keeps the model clean while still giving us full visibility.

Using RELATED to Fetch Data

Open the **Employees table in PowerPivot** and create a new column.

To get the project name:

```
=RELATED(Projects[Project Name])
```

PowerPivot uses the relationship between Employees and Projects (via PID) to return the correct value.

Fetching More Fields

You can bring more columns from the Projects table the same way.

For example, to get the deadline:

```
=RELATED(Projects[DeadLine])
```

Now each employee row shows which project they are working on and its deadline.

Chaining Relationships

Now suppose you want the **Technology**, which is stored in another table.

The relationship path is:

```
Employees → Projects → Tech Stack
```

Even though Employees is not directly connected to Tech Stack, PowerPivot follows the chain automatically.

```
=RELATED('Tech Stack'[Technology])
```

What Happens Internally

For each employee, PowerPivot follows the linked IDs:

```
E001 → PID P001  
P001 → TID T001  
T001 → Python + HTML
```

It traces the relationship path and returns the correct value without manual joins.

Result in Employees Table

EmpID	Name	PID	Project Name	Deadline	Technology
E001	Rohan	P001	Website Development	10-Jun	Python + HTML
E002	Aman	P002	Mobile App	25-Jun	Flutter

Now the Employees table contains all the important details, while the data model remains structured and efficient.

In the previous lectures, we saw how powerful PowerPivot is for handling large datasets, creating relationships, and performing advanced analysis.

But like any tool, it also has some important limitations that you should be aware of.

1. Not Available on Mac

PowerPivot is **not supported in Excel for Mac**.

It relies on a Windows-based data model engine, which is not available on macOS.

This means:

- You can only use PowerPivot on Windows
- Mac users cannot access this feature at all

2. High RAM Requirement

PowerPivot is memory-intensive. Even with small datasets, you may notice:

- Slow performance
- Lag while creating columns
- Delay in calculations

When working with large datasets (thousands or millions of rows), this can become worse and may even cause Excel to crash.

3. Performance Issues with Large Data

Although PowerPivot is designed for big data, in practice:

- It can become slow when data grows
- Calculations like `RELATED()` may take noticeable time
- System performance heavily affects speed

So performance depends a lot on your system configuration.

4. Dependency on Excel

PowerPivot is not a standalone tool. It works **inside Excel**, which means:



- You need Excel to use it
- It is part of a paid ecosystem
- You don't get the flexibility of tools like SQL or dedicated data platforms

5. Slow Data Refresh

Refreshing data in PowerPivot can be slow, especially when:

- Data size increases
- Multiple tables are connected
- Complex relationships exist

Even simple datasets can refresh slowly, and larger ones take more time. PowerPivot works best for moderate data inside Excel, but for bigger or more complex needs, tools like SQL are better.



Section 8: Git & GitHub for Data Analysts

Chapter 1: Introduction to Git and GitHub

Git is a version control system that tracks changes in your files, and GitHub is a platform that lets you store and manage those Git repositories online. Together, they help you manage files, track progress, and collaborate without losing work.

Why Git and GitHub Matter

Most people think Git is only for coding. That's incorrect.

Git is simply a **file tracking system**. If you work with files, datasets, reports, dashboards, or projects, it applies to you.

Think of it like a smarter version of this:

```
final.xlsx  
final_v2.xlsx  
final_final.xlsx  
final_final_last.xlsx
```

Git replaces this chaos with structured version tracking.

What is Git?

Git is a tool that:

- Tracks changes in files over time
- Lets you go back to previous versions
- Shows exactly what changed and when
- Allows multiple versions without duplication

Instead of creating new files, Git stores changes internally.

What is GitHub?

GitHub is a cloud platform where:

- Your Git projects (repositories) are stored
- You can access your work from anywhere
- You can collaborate with others
- You can showcase your projects publicly

Git works locally. GitHub adds storage, sharing, and collaboration.

Basic Workflow

Let's break down the workflow in simple terms:

Working Folder → Git → GitHub

1. You create or modify files
2. Git tracks changes
3. You save a version (commit)
4. You upload it to GitHub

This way, you have a history of your work and can share it easily.

Where It Fits in Data Analytics

You can use Git and GitHub for:

- Storing datasets
- Managing Jupyter notebooks
- Tracking SQL queries
- Versioning dashboards
- Sharing projects with recruiters

Git is not about coding. It is about **control over your work**. Once you start using it, you stop worrying about losing files or messing things up, because every version is safely tracked.



Before you can start using Git and GitHub, you need to set up the tools on your system and understand how they work together. This includes installing Git and choosing how you want to interact with it, either through commands or a graphical interface.

Git vs GitHub (Quick Clarity)

Git is a **software installed on your computer** that helps you track changes in files.

GitHub is a **cloud service** where your Git projects are stored and shared online.

In simple terms:

- Git = works on your computer
- GitHub = works on the internet

This distinction is essential before moving forward.

Installing Git

To use Git, you must install it on your system. Here's how:

1. Go to the official [Git website](#)
2. Download the version for your operating system (Windows x64 if you're on Windows)
3. Run the setup file
4. Click "Next" through the installation process
5. Finish installation

Once installed, Git is ready to use on your system.

Using Git After Installation

After installation, Git provides a terminal interface where you can run commands like:

```
git init
git status
```

This gives you full control over version tracking, but it requires learning commands.

GitHub Account Setup

Before using GitHub, you need an account.

- Go to [GitHub website](#)
- Sign up using your email
- Create a username and password
- Log in

The process is similar to signing up on any website.

What is GitHub Desktop?

GitHub Desktop is a **graphical application** that allows you to use Git without writing commands.

It is designed for people who:

- Don't want to use terminal commands
- Prefer clicking instead of typing
- Are non-coders or beginners

Why Use GitHub Desktop

Git can feel intimidating at first because it works through commands.

GitHub Desktop solves this by providing:

- Buttons for commits, push, pull
- Visual file changes
- Easy repository management

Even someone with no coding background can use Git through this interface.

Installing GitHub Desktop

1. Download [GitHub Desktop](#)
2. Install the application
3. Sign in with your GitHub account
4. Enter your name and email
5. Start using repositories

Command Line vs GUI

You have two ways to use Git:

- Command Line (Git)
- More control
- Preferred by developers
- GUI (GitHub Desktop)
- Easier to use
- No commands required

Choose based on your comfort level.

Git was originally built for developers, but it is not limited to them. Even if you don't write code, you can still use Git through tools like GitHub Desktop to manage files, track changes, and collaborate effectively.



Before working with Git and GitHub, it is important to configure Git on the system and understand how a repository is created and managed. This includes setting up identity, initializing a project, and knowing what the basic commands actually do.

Checking Git Installation

After installing Git, the first step is to verify that it is working correctly.

Open the project folder, right click, and select "Open in Terminal". Then run:

```
git --version
```

If Git is installed properly, it will return the installed version number. This confirms Git is ready to use.

Configuring Git

Git requires identity details so it can track who is making changes in a project.

Run the following commands:

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

These commands set your name and email globally, meaning they will be used for all repositories on your system.

To confirm the setup:

```
git config --list
```

This displays all current Git settings, including your configured name and email.

Creating Your First Repository

A repository is a folder where Git tracks all changes made to files.

Create or open a project folder in terminal, then run:

```
git init
```

This command initializes the folder as a Git repository. It creates a hidden `.git` folder that stores all version history and tracking information.

You don't have to remember these Git commands, as GitHub Desktop will be used in the long run.

What Happens After Initialization

Once a repository is created:

- Git starts monitoring file changes
- Versions of the project can be saved using commits
- The project can later be connected to GitHub for backup and sharing

At this stage, everything is still stored locally on the system.

Understanding Basic Git Commands

Once a repository is initialized, a typical workflow looks like this:

```
git add .  
git commit -m "message"  
git push
```

Here is what each command does:

- `git add .` stages all changes in the project, meaning Git prepares those files to be saved
- `git commit -m "message"` creates a snapshot of the changes with a message describing what was done
- `git push` uploads those changes to GitHub, making them available online and backed up.

These commands form the core of how Git tracks and saves work.

Using GitHub Desktop

Instead of typing commands manually, GitHub Desktop can be used to perform the same actions through a graphical interface.

Git commands can feel technical and hard to remember at first, so GitHub Desktop simplifies the workflow by providing a clean and visual way to manage everything.

With GitHub Desktop, it becomes possible to:

- Add a local folder as a repository
- See file changes visually
- Write commit messages easily
- Push code to GitHub with a button

This makes version control easier to understand and especially useful for beginners or anyone who prefers a GUI over the terminal.

CodeWithHarry



Before using Git effectively, it is important to understand how changes move through different stages. Git is not just a set of commands, it is a system with a clear flow of how your code is tracked, saved, and shared.

The Four Core Areas in Git

Git works across four main areas:

- **Working Directory** This is where you create and edit files. Any changes you make in your project exist here first.
- **Staging Area** This is a temporary space where you prepare changes before saving them. Only staged changes will be committed.
- **Local Repository** This is where Git permanently stores your project history on your system.
- **Remote Repository** This is the online version of your project (for example, GitHub), used for backup and collaboration.

The Basic Workflow

A typical Git workflow follows this sequence:

```
git add .  
git commit -m "message"  
git push
```

This represents how changes move step by step through the system.

Step 1: Working Directory → Staging Area

When you modify files, Git does not automatically track those changes.

To include them, you run:

```
git add .
```

This command moves changes from the working directory to the staging area.

What this means:

- You are selecting which changes should be saved
- Nothing is saved yet, only prepared
- You can stage specific files instead of everything

Step 2: Staging Area → Local Repository

Once changes are staged, they can be permanently saved using:

```
git commit -m "Your message"
```

This creates a snapshot of your project.

What happens here:

- Git records the current state of staged files
- A commit message describes what changed
- The snapshot is stored in the local repository

This is your project history.

Step 3: Local Repository → Remote Repository

To share your work or back it up online, you use:

```
git push
```

This uploads your commits to the remote repository.

What this does:

- Sends your local commits to GitHub
 - Makes your changes available to others
 - Keeps your code safe online
-

Getting Changes from Remote

If someone else has pushed updates, you bring them into your project using:

```
git pull
```

This does two things:

- Downloads changes from the remote repository
 - Updates your local repository and working directory
-

Switching Between Versions

Git allows moving between different commits or branches using:

```
git checkout
```

This command:

- Changes your working directory to match a specific commit or branch
- Lets you explore different versions of your project



When working alone on a project, tools like Git and GitHub Desktop make it easier to manage your code without relying on the command line. This approach is especially useful for beginners who want a visual way to track changes and manage repositories.

Creating a New Repository

The first step is to create a repository on your local system.

In GitHub Desktop, you can:

- Click on "Create a New Repository"
- Choose a project name (for example, *backend project*)
- Select the folder where the project will be stored
- Initialize the repository

This sets up Git tracking for your project.

Working on Your Project

Once the repository is created, you can start adding files and writing code in that folder.

As you make changes:

- GitHub Desktop automatically detects modified files
 - You can see all changes in a visual interface
 - You decide what changes to save
-

Saving Changes

After making changes, you need to save them using a commit.

In GitHub Desktop:

- Write a short message describing your changes
- Click on "Commit to main"

This creates a snapshot of your project at that point.

Why This Workflow Matters

Even when working alone, this process helps you:



- Keep a history of your work
- Avoid losing progress
- Fix mistakes by going back to earlier versions
- Stay organized as your project grows

CodeWithHarry



Working with GitHub is about staying in sync with others. Push and pull are what keep everyone's code aligned.

Collaborating on a Repository

Collaboration begins when you are added as a collaborator on a repository.

Once you have access, you and other developers can work on the same project at the same time. Git keeps track of changes so everyone can safely contribute without overwriting each other's work.

Cloning the Repository

Before you can start working, you need a local copy of the project.

Using GitHub Desktop:

- Copy the repository URL from GitHub
- Open GitHub Desktop
- Click "Clone a repository from the internet"
- Paste the URL and choose a folder
- Clone the repository

Using terminal:

```
git clone <repository-url>
```

This downloads the project to your system so you can edit it locally.

Pulling Changes

If someone else has pushed updates, your local project may be outdated.

Using GitHub Desktop:

- Click "Fetch origin"
- Then click "Pull"

Using terminal:

```
git pull origin main
```

This updates your local files with the latest changes from GitHub.

Pushing Changes

After making changes, you need to share them with others.

Using GitHub Desktop:

- Write a commit message
- Click "Commit to main"
- Click "Push origin"

Using terminal:

```
git add .  
git commit -m "your message"  
git push origin main
```

This uploads your changes to GitHub so others can access them.

Pushing and pulling are essential for teamwork. They ensure everyone is working with the latest code and that contributions are shared effectively.



Git keeps a full history of your project, so you can view past versions and undo changes when needed.

Viewing Commit History

Each repository shows a list of commits.

Click on the commit count to see all changes made over time. Every commit is a snapshot of your project at that moment.

Accessing a Specific Commit

You can open any past version of your project. This is useful for checking how things looked before a change or for debugging.

- Go to the commits section
- Click a commit
- Select "Browse repository at this point"

This shows how your project looked at that exact time.

Reverting a Commit

Let's say you made a change that caused issues. You can revert it to fix the problem.

In GitHub Desktop:

- Right click the commit
- Click "Revert changes in commit"

This creates a new commit that cancels the previous change.

How Reverting Works

Git does not delete history. Instead, it adds a new commit that reverses the old one, keeping everything traceable. This way, you can always see what happened and why, which is crucial for learning and collaboration.



Branching lets you create a separate line of work without touching the main project. This is essential for developing new features, fixing bugs, or experimenting without risking the stability of the main codebase.

What is Branching

A branch is a copy of your project where you can make changes independently. The default branch is usually `main`. Any new branch starts from it but works separately. You can have multiple branches for different features or fixes, and they won't affect each other until you decide to merge them back together.

Why Branching is Needed

You should not make experimental or incomplete changes directly in the main branch.

Branching helps you:

- Keep the main project stable
- Work on new features safely
- Test changes without breaking existing code

Creating a Branch

On GitHub Desktop:

- Click on "Current Branch"
- Select "New Branch"
- Give it a name and create

If you prefer the terminal, you can create and switch to a new branch with:

```
git branch <branch-name>
git checkout <branch-name>
```

Or in one command:

```
git checkout -b <branch-name>
```

Working on a Branch

Once you switch to a branch:

- Make changes to your files
- Commit those changes

These changes stay only in that branch.

Merging the Branch

Merging brings your changes from the feature branch back into the main branch. After your work is complete, you merge it back into `main`.

On GitHub Desktop: * Switch to the main branch * Click on "Branch" in the menu * Select "Merge into Current Branch" * Choose the branch you want to merge from and confirm

If you prefer the terminal, you can merge branches with:

```
git checkout main
git merge <branch-name>
```

This brings your changes into the main project.

Branching keeps your workflow clean and prevents breaking the main code while you build new features.



A pull request is how you propose changes from one branch into another, usually from a feature or dev branch into the `main` branch. It allows others to review your work before it becomes part of the main project.

A pull request compares two branches and shows the differences between them. It highlights what was added, removed, or modified.

Instead of directly merging your branch into `main`, you first create a pull request to ask for approval.

Why Pull Requests are Needed

You should not merge changes directly into the main branch, especially in collaborative projects.

Pull requests help you:

- Review code before merging
- Catch mistakes early
- Keep the main branch stable

They add a safety layer between development and production code.

Creating a Pull Request

On GitHub:

- Push your branch to GitHub
- Open the repository
- Click on "Compare & pull request"
- Review the changes shown
- Add a title and description
- Click on "Create pull request"

GitHub will show all the commits and file changes included in your branch.

Reviewing and Merging

Once the pull request is created:

- Others can review your code
- Suggestions or changes can be requested

If everything looks good:



- Click on “Merge pull request”
- Confirm the merge

Your changes are now added to the `main` branch.

Pull requests are essential for maintaining code quality and ensuring that all changes are reviewed before they become part of the main project. They facilitate collaboration and help catch issues early in the development process.

CodeWithHarry



The `.gitignore` file tells Git which files or folders should not be tracked. This is useful when you have files in your project that you don't want to push to GitHub.

What is .gitignore

A `.gitignore` file is a special file where you list the names of files or folders that Git should ignore.

Even if those files exist in your project folder, Git will not track changes in them, and they won't be pushed to GitHub.

Why .gitignore is Needed

Sometimes your project contains files that should not be tracked.

For example:

- Large files like videos or setup files
- Temporary or generated files
- Files only needed for local use

Tracking such files makes the repository heavy and slows down push and pull operations.

Creating a .gitignore File

You can create a file named `.gitignore` in your project directory.

Inside it, you simply write the names of files you want Git to ignore.

Example:

```
setup.exe
```

This tells Git to ignore the `setup.exe` file completely.

How It Works

Once you add a file name inside `.gitignore` :

- Git stops tracking that file
- It will not appear in commits
- It will not be pushed to GitHub

In tools like GitHub Desktop, ignored files may appear faded or not show up at all.

The file still exists in your folder, but Git behaves as if it doesn't exist. You control what gets tracked and what stays local using `.gitignore`.

CodeWithHarry



Git is not just for developers. For a data analyst, it is a system to track experiments, manage datasets, and keep analysis reproducible. Using Git effectively can save you from losing work, help you collaborate, and keep your projects organized.

How Data Analysts Should Use Git

Data analysis is an iterative process. You start with raw data, clean it, run analysis, and generate insights. During this process, multiple files keep changing.

During this process, Git helps you:

- Track every step of your analysis
- Keep raw and processed data organized
- Maintain different versions of notebooks and scripts
- Collaborate without overwriting each other's work

Instead of saving files like `final_v2_last_final.ipynb`, Git keeps everything structured.

Best Practices for Data Analysts

Keep Raw Data Untouched

Always store raw data separately and never modify it directly. Treat it as a source of truth.

Use a structure like:

- `data/raw/`
- `data/processed/`

Use Meaningful Commits

Avoid vague messages like "update" or "changes".

Write clear commit messages:

- "Cleaned missing values in sales dataset"
- "Added feature engineering for customer segmentation"

This makes your work understandable later.

Commit Small and Often

Do not wait until the end of the day to commit everything.

Commit after:

- Cleaning data
- Writing a function
- Completing a meaningful step

This makes it easy to track and revert changes.

Avoid Large Files in Git

Git is not designed for large datasets. It can slow down your repository and cause issues.

Do not upload:

- Huge CSV files
- Raw datasets in GBs

Instead:

- Use `.gitignore` to exclude them
 - Store them externally (cloud, database)
-

Use Branches for Experiments

Data analysis involves a lot of experimentation. When trying new models or approaches, create a separate branch. This way, if something goes wrong, your main work remains safe.

You can create branches for:

- Trying new models
- Testing different approaches
- Running alternative analyses

If something fails, your main work remains safe.

Keep Notebooks Clean

Jupyter Notebooks can get messy. Limit the number of output cells and clear unnecessary outputs before committing. This keeps your version history clean and focused on code changes.

Before committing:

- Clear unnecessary outputs
- Remove debugging cells
- Keep code readable

This avoids noise in version history.

Document Your Work

Always include a README file. A good README explains:

- Project goal
- Dataset details
- Steps to reproduce the analysis
- Tools used

This makes your project understandable to others and to your future self.

Using Git as a data analyst is about clarity and control. It helps you manage experiments, avoid losing work, and keep your analysis reproducible.



Section 9: Web Development for Data Analysts (UI, Dashboards & Visualization)

Chapter 1: Introduction to Web Development for Data Analysts

Let's now learn how web development can help you present and share your data analysis work effectively.

As a data analyst, your job is not just to analyze data but also to communicate insights. Web development gives you the ability to build simple dashboards, interactive reports, and share your work with others. We will later see how you can use Chat.js - yes AI powered as already promised towards the beginning of this course!

Why Web Development for Data Analysts

Most analysts rely on tools like Excel, Power BI, or Python. These are powerful, but they have limitations:

- Sharing is not always smooth
- Customization is limited
- Interactivity can be restricted

With basic web development skills, you can:

- Create your own dashboards (Yes you can use AI)
- Display data in a clean and interactive way
- Share your work using a simple link - We will learn deployment later
- Build a portfolio to showcase your projects (Yes again this can be done using AI)

What You Will Learn in This Section

You will not become a full stack pro developer here. The goal is to learn just enough web development to support your data analysis work.

We will cover:

- HTML to structure content
- CSS to style your pages
- JavaScript to make things interactive
- Using AI to 100x all these skills
- Deploying your project online

Before anything, I want you to understand how websites work. This is something I have been teaching in every web development class I have ever taken (You know it if you watch my videos)

How Websites Work

A typical website works using a simple idea called the request and response cycle.

Whenever you open a website, a conversation happens between your browser and a server.

Let us understand this step by step.

What Happens When You Open a Website

When you type a URL in your browser, for example:

<https://codewithharry.com>

This is what happens behind the scenes:

1. Your browser sends a request to a server
2. The server processes that request
3. The server sends back a response
4. Your browser displays the result

This process is called the request and response cycle.

Request and Response Cycle

- Request
A request is sent by your browser asking for some data or a webpage.
- Response
A response is what the server sends back. This can include HTML, CSS, JavaScript, or data in JSON format.

You can think of it like ordering food:

- You place an order
- The kitchen prepares it
- The food is served to you

Role of the Browser

Browsers like Chrome, Edge and Firefox act as the client.

Their job is to:

- Send requests to servers
- Receive responses
- Convert code into a visual webpage

What Chrome Actually Does

Let us break down what happens inside Chrome:



1. You enter a URL in the address bar
2. Chrome sends a request to the server using the internet
3. The server returns files like:
4. HTML for structure
5. CSS for styling
6. JavaScript for interactivity
7. Chrome reads the HTML and builds the structure of the page
8. Chrome applies CSS to style the page
9. Chrome runs JavaScript to make the page interactive
10. Finally, you see the fully loaded website



In this lecture, you will learn how to create the structure of a webpage using HTML.

Think of HTML as the skeleton of a website. It defines what appears on the page and how content is organized.

As a data analyst, HTML helps you present dashboards, reports, tables, and insights clearly.

What is HTML

HTML stands for HyperText Markup Language.

It is used to structure content using tags.

A browser like Chrome reads HTML and converts it into a visual page.

Examples of things you can create using HTML:

- Headings
- Paragraphs
- Tables for data
- Links to reports
- Images and charts

How a Website Works (Quick Intuition)

When you type a URL in your browser:

1. Your browser sends a request to a server
2. The server sends back HTML
3. The browser renders that HTML into a webpage

So HTML is what the browser understands and displays.

Basic Structure of an HTML Page

Every HTML page follows this structure:




```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>This is my first webpage</p>
</body>
</html>
```

Understanding the Structure

- `<!DOCTYPE html>` tells the browser this is an HTML5 document
- `<html>` is the root of the page
- `<head>` contains metadata (not visible on the page)
- `<body>` contains everything the user sees

Important additions:

- `<meta charset="UTF-8">` ensures text displays correctly
- `<meta name="viewport">` makes your page responsive on mobile

Most Important HTML Tags

Headings

```
<h1>Main Heading</h1>
<h2>Sub Heading</h2>
<h3>Smaller Heading</h3>
```

Use only one `<h1>` per page for best structure.

Paragraphs and Text


```
<p>This is a paragraph</p>
<strong>Bold Text</strong>
<em>Italic Text</em>
<br>
```

- `<strong>` highlights important text
- `<em>` emphasizes text
- `<br>` adds a line break

Links

```
<a href="https://example.com" target="_blank">Visit Website</a>
```

- `href` is the destination
- `target="_blank"` opens in a new tab

Images

```

```

- `alt` is important for accessibility and SEO
- Always describe your image properly

Tables for Data Analysts

Tables are one of the most important tools for you.



```
<table border="1">
  <thead>
    <tr>
      <th>Name</th>
      <th>Score</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Rahul</td>
      <td>85</td>
    </tr>
    <tr>
      <td>Anita</td>
      <td>92</td>
    </tr>
  </tbody>
</table>
```

Key Tags

- `<table>` defines the table
- `<thead>` is the header section
- `<tbody>` contains the data
- `<tr>` is a row
- `<th>` is a header cell
- `<td>` is a data cell

Lists

Unordered List

```
<ul>
  <li>Python</li>
  <li>SQL</li>
  <li>Excel</li>
</ul>
```

Ordered List

```
<ol>
  <li>Collect Data</li>
  <li>Clean Data</li>
  <li>Analyze Data</li>
</ol>
```

Forms

Very useful when building tools or dashboards.

```
<form>
  <label>Name:</label>
  <input type="text" placeholder="Enter your name">

  <label>Email:</label>
  <input type="email">

  <button type="submit">Submit</button>
</form>
```

Semantic HTML

These tags improve readability and structure:

```
<header>Website Header</header>
<nav>Navigation Links</nav>
<main>Main Content</main>
<section>Section</section>
<footer>Footer</footer>
```

Think of them as meaningful containers instead of just using `<div>` everywhere.

Div and Span

```
<div>
  <p>This is inside a div</p>
</div>

<span>This is inline text</span>
```

- `<div>` is a block container
- `<span>` is an inline container

You will use these a lot with CSS later.

CodeWithHarry



In this lecture, we will learn how to make your webpages clean, readable, and professional using CSS.

HTML gives structure. CSS makes it usable.

As a data analyst, your goal is simple: present information clearly so people can understand it fast.

What is CSS

CSS stands for Cascading Style Sheets.

It is used to control:

- Colors
- Fonts
- Spacing
- Layout

Without CSS, your page looks like raw data. With CSS, it looks like a dashboard.

Three Ways to Use CSS

There are three main ways to apply CSS to your HTML:

1. Inline CSS

Applied directly inside an HTML tag.

```
<p style="color: blue;">This is a paragraph</p>
```

- Quick to use
- Not recommended for large projects

2. Internal CSS

Written inside a `<style>` tag in the `<head>` section.

```
<head>
  <style>
    p {
      color: blue;
    }
  </style>
</head>
```

- Useful for small projects
- Keeps styles in one place

3. External CSS

Written in a separate `.css` file and linked to HTML.

```
<link rel="stylesheet" href="style.css">
```

```
p {  
  color: blue;  
}
```

- Clean and scalable
- Reusable across multiple pages
- Recommended for almost all projects

The Right Way to Use CSS

You can write CSS inside HTML, but a better approach is to use a separate file.

Step 1: Create a CSS file

Create a file called `style.css`

Step 2: Link it to HTML

```
<link rel="stylesheet" href="style.css">
```

Your HTML file now uses styles from `style.css`

Basic CSS Syntax

```
selector {  
  property: value;  
}
```

Example:



```
h1 {  
  color: blue;  
}
```

- `h1` is the selector
- `color` is the property
- `blue` is the value

Styling Text

```
body {  
  font-family: Arial, sans-serif;  
  font-size: 16px;  
  line-height: 1.6;  
}  
  
h1 {  
  color: #1a1a1a;  
  font-size: 28px;  
}  
  
p {  
  color: #444;  
}
```

This alone makes a huge difference in readability.

Colors and Backgrounds

Background Color

```
body {  
  background-color: #f5f7fa;  
}
```

Background Image




```
body {  
  background-image: url("bg.jpg");  
  background-size: cover;  
  background-repeat: no-repeat;  
}
```

Use background images carefully. Keep text readable.

Container Layout

Wrap your content inside a container to keep it centered and clean.

```
.container {  
  max-width: 900px;  
  margin: 40px auto;  
  background: #ffffff;  
  padding: 20px;  
  border-radius: 8px;  
}
```

This is one of the most important patterns you will reuse.



Styling Tables

```
table {  
  width: 100%;  
  border-collapse: collapse;  
  margin-top: 20px;  
}  
  
th, td {  
  padding: 10px;  
  border: 1px solid #ddd;  
  text-align: left;  
}  
  
th {  
  background-color: #f0f0f0;  
  font-weight: bold;  
}  
  
tr:nth-child(even) {  
  background-color: #fafafa;  
}
```

This makes your data instantly more readable.

Spacing

```
h1, p {  
  margin-bottom: 10px;  
}  
  
.section {  
  margin-top: 20px;  
}
```

Key Idea

- Margin controls space outside
- Padding controls space inside

Buttons

```
button {  
  background-color: #007bff;  
  color: white;  
  border: none;  
  padding: 10px 15px;  
  border-radius: 5px;  
  cursor: pointer;  
}  
  
button:hover {  
  background-color: #0056b3;  
}
```

Simple Layout with Flexbox

You do not need to master flexbox, just this:

```
.row {  
  display: flex;  
  gap: 20px;  
}  
  
.card {  
  flex: 1;  
  background: white;  
  padding: 15px;  
  border-radius: 6px;  
}
```

This helps you place items side by side.

Full Example You Can Copy

HTML



```
<!DOCTYPE html>
<html>
<head>
  <title>Data Dashboard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
  <h1>Sales Report</h1>

  <p>Monthly performance overview</p>

  <table>
    <tr>
      <th>Month</th>
      <th>Revenue</th>
    </tr>
    <tr>
      <td>Jan</td>
      <td>5000</td>
    </tr>
    <tr>
      <td>Feb</td>
      <td>7000</td>
    </tr>
  </table>

  <button>Download Report</button>
</div>

</body>
</html>
```

CSS



```
body {  
  font-family: Arial, sans-serif;  
  background-color: #f5f7fa;  
}  
  
.container {  
  max-width: 800px;  
  margin: 40px auto;  
  background: white;  
  padding: 20px;  
  border-radius: 8px;  
}  
  
h1 {  
  color: #222;  
}  
  
table {  
  width: 100%;  
  border-collapse: collapse;  
  margin-top: 20px;  
}  
  
th, td {  
  padding: 10px;  
  border: 1px solid #ddd;  
}  
  
th {  
  background-color: #f0f0f0;  
}  
  
button {  
  margin-top: 20px;  
  padding: 10px;  
  background: #007bff;  
  color: white;  
  border: none;  
}
```

In this lecture, you will learn how to make your webpages interactive using JavaScript.

HTML gives structure. CSS makes it look good. JavaScript makes it useful.

As a data analyst, this is where you start building real tools like dashboards and data viewers.

What is JavaScript

JavaScript is a programming language that runs inside your browser.

It allows you to:

- Update content dynamically
- Handle user actions
- Fetch real data from APIs
- Build interactive reports and dashboards

Adding JavaScript to Your Page

You can write JavaScript inside a `<script>` tag or use a separate file.

Internal JavaScript

```
<script>
  console.log("Hello World");
</script>
```

External JavaScript (Better)

```
<script src="script.js"></script>
```

Variables (Storing Data)

```
let name = "Harry";
let age = 25;
let isAnalyst = true;
```

Use `let` for values that can change.

Functions (Reusable Logic)

```
function greet(name) {  
  console.log("Hello " + name);  
}  
  
greet("Harry");
```

Working with the Webpage (DOM)

JavaScript can change what is shown on the page.

```
<h1 id="title">Hello</h1>  
  
<script>  
  let heading = document.getElementById("title");  
  heading.innerText = "Hello Data Analysts";  
</script>
```

Handling User Actions

```
<button id="btn">Click Me</button>  
  
<script>  
  let btn = document.getElementById("btn");  
  
  btn.addEventListener("click", function () {  
    alert("Button clicked");  
  });  
</script>
```

This is better than inline `onclick`.

Arrays and Objects (Very Important)

This is how real data looks in JavaScript.




```
let scores = [85, 90, 78];

let student = {
  name: "Rahul",
  score: 85
};
```

Looping Through Data

```
let data = [10, 20, 30];

data.forEach(function(item) {
  console.log(item);
});
```

Cleaner than traditional loops.

Displaying Data on the Page

```
<ul id="list"></ul>

<script>
  let items = ["Python", "SQL", "Excel"];
  let list = document.getElementById("list");

  items.forEach(function(item) {
    let li = document.createElement("li");
    li.innerText = item;
    list.appendChild(li);
  });
</script>
```

Fetching Data from an API

This is where things become powerful.



```
<div id="data"></div>

<script>
  fetch("https://jsonplaceholder.typicode.com/users")
    .then(function(response) {
      return response.json();
    })
    .then(function(users) {
      let container = document.getElementById("data");

      users.forEach(function(user) {
        let p = document.createElement("p");
        p.innerText = user.name;
        container.appendChild(p);
      });
    })
    .catch(function(error) {
      console.log("Error:", error);
    });
</script>
```

You just fetched real data and displayed it.

Cleaner Fetch with async/await

```
async function loadData() {
  try {
    let response = await fetch("https://jsonplaceholder.typicode.com/users");
    let users = await response.json();

    console.log(users);
  } catch (error) {
    console.log(error);
  }
}

loadData();
```

Chart.js is a powerful JavaScript library for creating beautiful and responsive charts. This lecture will guide you through the basics of using Chart.js to create various types of charts. It is heavily maintained and has a large community, making it a great choice for both beginners and experienced developers.

1. Basic Working Setup

This is your **base template**. Every chart will work inside this.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Chart.js Tutorial</title>
  <style>
    body {
      font-family: Arial;
      padding: 20px;
    }
    .chart-container {
      width: 600px;
      margin: auto;
    }
  </style>
</head>
<body>

  <h2>My First Chart</h2>

  <div class="chart-container">
    <canvas id="myChart"></canvas>
  </div>

  <!-- Chart.js CDN -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

  <script>
    // JS will go here
  </script>

</body>
</html>
```

2. Bar Chart

Replace the `<script>` part with this:

```
const ctx = document.getElementById('myChart');

new Chart(ctx, {
  type: 'bar',
  data: {
    labels: ['Jan', 'Feb', 'Mar', 'Apr'],
    datasets: [{
      label: 'Sales',
      data: [12, 19, 3, 5],
      backgroundColor: ['red', 'blue', 'green', 'orange']
    }]
  }
});
```

3. Line Chart

```
const ctx = document.getElementById('myChart');

new Chart(ctx, {
  type: 'line',
  data: {
    labels: ['Jan', 'Feb', 'Mar', 'Apr'],
    datasets: [{
      label: 'Revenue',
      data: [10, 20, 15, 25],
      borderColor: 'blue',
      fill: false
    }]
  }
});
```



4. Pie Chart

```
const ctx = document.getElementById('myChart');

new Chart(ctx, {
  type: 'pie',
  data: {
    labels: ['Red', 'Blue', 'Yellow'],
    datasets: [{
      data: [10, 20, 30],
      backgroundColor: ['red', 'blue', 'yellow']
    }]
  }
});
```

5. Doughnut Chart

```
const ctx = document.getElementById('myChart');

new Chart(ctx, {
  type: 'doughnut',
  data: {
    labels: ['A', 'B', 'C'],
    datasets: [{
      data: [30, 40, 30],
      backgroundColor: ['purple', 'cyan', 'orange']
    }]
  }
});
```



6. Radar Chart

```
const ctx = document.getElementById('myChart');

new Chart(ctx, {
  type: 'radar',
  data: {
    labels: ['Speed', 'Strength', 'Skill'],
    datasets: [{
      label: 'Player',
      data: [80, 90, 70],
      backgroundColor: 'rgba(0, 0, 255, 0.2)',
      borderColor: 'blue'
    }]
  }
});
```

If you want to force the scale to start at zero, you can add this to the options:

```
new Chart(ctx, {
  options: {
    scales: {
      r: {
        beginAtZero: true,
        min: 0,
        max: 100
      }
    }
  },
  type: 'radar',
  data: {
    labels: ['Speed', 'Strength', 'Skill'],
    datasets: [{
      label: 'Player',
      data: [80, 90, 70],
      backgroundColor: 'rgba(0, 0, 255, 0.2)',
      borderColor: 'blue'
    }]
  }
});
```

7. Customization

Add Title

```
options: {  
  plugins: {  
    title: {  
      display: true,  
      text: 'My Chart Title'  
    }  
  }  
}
```

Add Legend Position

```
options: {  
  plugins: {  
    legend: {  
      position: 'bottom'  
    }  
  }  
}
```

Add Border

```
borderColor: 'black',  
borderWidth: 2
```

8. Update Data Dynamically

```
const chart = new Chart(ctx, {...});  
  
chart.data.datasets[0].data = [5, 10, 15, 20];  
chart.update();
```



Scenario

You are a data analyst at a company called TechNova. Your manager has given you monthly performance data and asked you to visualize it using Chart.js.

Given Data

```
const months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'];  
  
const revenue = [50000, 60000, 55000, 70000, 75000, 80000];  
  
const expenses = [30000, 35000, 32000, 40000, 42000, 45000];  
  
const profit = [20000, 25000, 23000, 30000, 33000, 35000];
```

Tasks

1. Create a Line Chart

- Show Revenue, Expenses, and Profit
- Each should be a separate dataset
- X-axis: Months

2. Create a Bar Chart

- Show only Revenue
- Each bar should have a different color

3. Create a Pie Chart

- Show Profit distribution for June
- Categories:
 - Reinvestment: 40%
 - Salaries: 35%
 - Misc: 25%

4. Bonus

- Add a title to at least one chart
- Move the legend to bottom

Expected Learning

By solving this, you will learn:

- Handling multiple datasets
- Choosing the right chart type
- Customizing charts
- Thinking like a data analyst

Hint

- Use `type: 'line'` for trends
- Use `datasets: []` for multiple lines
- Use `plugins.title` for title

Challenge

Use AI to create a dashboard with 3 charts on one page



In this lecture, we will explore how AI can be integrated into your web development workflow to improve productivity, speed, and overall code quality. Instead of replacing your role as a developer, AI acts as a powerful assistant that helps you build faster and smarter.

Why Use AI in Web Development?

AI tools can assist in generating code, improving UI design, debugging issues, and even suggesting better approaches to implementation. When used correctly, they reduce repetitive work and allow you to focus on core logic and problem-solving.

How AI Fits Into Your Workflow

A practical approach is to keep an AI assistant open alongside your development environment. You can provide clear instructions and guide the AI to make specific changes without affecting critical parts of your application.

For example, you can instruct the AI to:

- Improve the frontend design
- Add responsiveness
- Keep your data and chart logic unchanged

This ensures that you maintain control while still benefiting from AI-generated improvements.

Writing Effective Prompts

The quality of AI output depends heavily on how you communicate your requirements. Instead of vague instructions, use precise prompts such as:

- "Redesign the UI but do not modify existing data logic"
- "Add a responsive navbar without changing chart configurations"

Clear instructions lead to better and safer results.

Iterating Faster with AI

AI allows you to make rapid changes without manually editing every detail. You can:

- Adjust styles like colors, spacing, and layout instantly
- Add new components such as headers or navigation bars
- Enhance charts by adding legends or improving readability

If the result is not perfect, you can refine your prompt and regenerate the output quickly.

Leveraging Large Context Understanding

Modern AI tools can process large chunks of code at once. This allows you to:

- Share entire files or sections of your project
- Make broader improvements in a single step
- Maintain consistency across your application

Best Practices

1. Always review AI-generated code before using it
2. Ensure that your data and logic remain unchanged
3. Use AI for enhancement, not blind automation
4. Iterate and refine prompts for better results

AI can significantly enhance your web development workflow by reducing effort and increasing speed. When combined with your understanding of the code, it becomes a powerful tool that helps you build better applications more efficiently.



In this lecture, we will learn how to deploy our interactive charts and dashboards to Netlify, a popular platform for hosting static websites. This will allow us to share our work with others and make it accessible online.

What is Netlify?

Netlify is a cloud-based platform that provides hosting and serverless backend services for web applications and static websites. It offers features like continuous deployment, custom domains, and SSL certificates, making it an excellent choice for deploying our Chart.js projects.

Steps to Deploy to Netlify

- 1. Create a Netlify Account:** If you don't have one already, sign up for a free account at [Netlify](https://netlify.com).
- 2. Prepare Your Project:** Ensure your project is organized and includes all necessary files (HTML, CSS, JS, and any assets).
- 3. Create a New Site:** In your Netlify dashboard, click on "New site from Git" if you are using a Git repository, or "Deploy manually" to upload your files directly.
- 4. Connect Your Repository:** If you chose to deploy from Git, connect your GitHub, GitLab, or Bitbucket account and select the repository containing your project.
- 5. Configure Build Settings:** If your project requires a build step (e.g., using a static site generator), configure the build command and publish directory.
- 6. Deploy Your Site:** Click on "Deploy site" and wait for Netlify to build and deploy your project. Once deployed, you will receive a unique URL where your site is accessible.
- 7. Custom Domain (Optional):** You can set up a custom domain for your site through Netlify's domain management features.

Conclusion

Deploying your projects to Netlify is a great way to share your work with others and make it accessible online. With Netlify's user-friendly interface and powerful features, you can easily host your interactive charts and dashboards without worrying about server management.



Section 10: Probability & Statistics for Data Analytics

Chapter 1: Introduction to Probability

1. What is Probability?

Probability is a measure of the likelihood that a particular event will occur. It ranges from 0 (impossible) to 1 (certain). Probability is used to model uncertainty with data and make informed decisions based on that uncertainty.

Examples:

- Tossing a coin
- Rolling a die
- Predicting customer churn

2. Role of Probability in Data Analysis

Probability forms the foundation of statistical inference, machine learning, and decision-making under uncertainty. Data analysts use it to:

- Estimate unknown values from sample data
- Evaluate risks and uncertainties
- Make predictions using probabilistic models

3. Types of Probability

Theoretical Probability

Based on known possible outcomes.

- Example: Probability of getting heads in a fair coin = 0.5

Empirical (Experimental) Probability

Based on observed data.

- Example: Probability that a user clicks an ad based on 10,000 past impressions.

4. Key Concepts

- **Sample Space (S):** All possible outcomes of an experiment.
- **Event (E):** A subset of the sample space; one or more outcomes.

Example:

For rolling a die:

- Sample space: {1, 2, 3, 4, 5, 6}
- Event (even number): {2, 4, 6}

5. Real-World Business Scenarios

- Predicting customer conversion rate from a marketing campaign
- Estimating product failure rate in manufacturing
- Calculating insurance risks
- Detecting fraud in financial transactions

Calculating Probability

Formula:

$P(E) = \text{Number of favorable outcomes} / \text{Total number of outcomes}$

Where:

- $P(E)$ is the probability of event E
- Number of favorable outcomes is the count of outcomes that satisfy event E
- Total number of outcomes is the count of all possible outcomes in the sample space

Example:

If you roll a die, the probability of rolling a 3 is:

$P(\text{rolling a 3}) = \text{Number of favorable outcomes (1)} / \text{Total number of outcomes (6)} = 1/6 \approx 0.1667$

Quick Quiz

1. What is the probability of rolling a 5 on a fair six-sided die?
2. Two fair six-sided dice are rolled. What is the probability that at least one of the dice shows a 4?

Homework / Practice

1. List three real-world problems in Data Science where probability is involved.
2. Identify the sample space and possible events for each.
3. Try calculating a simple empirical probability using past data (if available).

Types of Experiments

1. **Deterministic** – Always produces the same outcome when repeated under identical conditions.
2. **Probabilistic / Random** – Outcome is uncertain and may vary even if the experiment is repeated under identical conditions.

Sample Space

The sample space is the set of all possible outcomes of a probabilistic experiment.

Examples:

- Rolling a Die $\rightarrow \{ 1, 2, 3, 4, 5, 6 \}$
- Tossing 2 Coins $\rightarrow \{ (HH), (HT), (TH), (TT) \}$

For Coins

- Number of possible outcomes = 2^n
(where n is the number of coins)

For Dice

- Number of possible outcomes = 6^n
(where n is the number of dice)

For Cards

A standard deck contains **52 cards**:

- **26 Red Cards**

13 ♥ Hearts $\rightarrow$ A, 2–10, J, Q, K

13 ♦ Diamonds $\rightarrow$ A, 2–10, J, Q, K

- **26 Black Cards**

13 ♠ Spades $\rightarrow$ A, 2–10, J, Q, K

13 ♣ Clubs $\rightarrow$ A, 2–10, J, Q, K

Conclusion

Understanding the basic types of experiments and the concept of sample space is essential for solving probability problems. Knowing the number of possible outcomes for coins, dice, and cards helps in calculating probabilities accurately. Mastery of these fundamentals forms the foundation for more advanced topics in probability and statistics.

Questions

1. You roll a fair six-sided die. What is the probability of getting an even number?
2. Two coins are tossed. Find the sample space associated with this random experiment.
3. Two six-sided dice are rolled. What is the probability that the sum is 7?
4. A bag contains 3 red, 2 blue, and 5 green balls. What is the probability of randomly selecting a red ball?
5. A card is drawn at random from a deck. Find the probability of:
 - Getting a heart
 - Getting a face card



1. Important Definitions

In probability, understanding the basic definitions is crucial for grasping more complex concepts.

- **Experiment:** An action or process that leads to one or more outcomes (e.g., rolling a die).
Deterministic experiments have predictable outcomes, while probabilistic or random experiments have uncertain outcomes.
- **Outcome:** A possible result of an experiment (e.g., rolling a 3 on a die).
- **Probability (P):** A measure of the likelihood that an event will occur, ranging from 0 (impossible) to 1 (certain).
- **$P(A \cup B)$:** The probability of event A or event B occurring.
- **$P(A \cap B)$:** The probability of both events A and B occurring.
- **Sample Space (S):** The set of all possible outcomes of an experiment.
- **Event (E):** A subset of the sample space.
- **Mutually Exclusive Events:** Two events that cannot occur at the same time (e.g., rolling a 2 and rolling a 5 on a die). $P(A \cap B) = 0$
- **Independent Events:** Two events where the occurrence of one does not affect the other.
 $P(A \cap B) = P(A) \cdot P(B)$
- **Certain Event:** An event that is guaranteed to happen, with a probability of 1.
- **Impossible Event:** An event that cannot happen, with a probability of 0.
- **Exhaustive Events:** A set of events that cover the entire sample space, meaning at least one of them must occur.

Example:

Tossing a die:

- Sample space: $S = \{1, 2, 3, 4, 5, 6\}$
- Event (E): Rolling an even number = $\{2, 4, 6\}$

2. The Complement Rule

The complement of an event A is the event that A does not occur.

- Notation: A^c
- Rule: $P(A^c) = 1 - P(A)$

Example:

If the probability of rain today is 0.3, the probability it won't rain is: - $P(\text{No Rain}) = 1 - 0.3 = 0.7$

3. The Addition Rule

Used to calculate the probability of the union of two events.

For general events:

$$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$$

For mutually exclusive events:

$$P(A \text{ or } B) = P(A) + P(B)$$

Example:

- $P(A) = 0.4$, $P(B) = 0.5$, $P(A \text{ and } B) = 0.2$
 - $P(A \text{ or } B) = 0.4 + 0.5 - 0.2 = 0.7$
-

4. The Multiplication Rule

Used to find the probability that two events occur together.

For Independent events:

$$P(A \text{ and } B) = P(A) \times P(B)$$

Example: A event A is rolling a 3 on a first throw of a die, and event B is rolling an even number on second throw.

- $P(A) = 1/6$ (rolling a 3)
- $P(B) = 3/6$ (rolling a 2, 4, or 6)
- $P(A \text{ and } B) = P(A) \times P(B) = (1/6) \times (3/6) = 1/12$

For Dependent events:

$$P(A \text{ and } B) = P(A) \times P(B | A)$$

Example: A bag contains 5 red and 3 blue balls. Two balls are drawn **without replacement**. Let:

- Event A be drawing a red ball first.
- Event B be drawing a red ball second.
- $P(A) = 5/8$ (5 red balls out of 8)
- After removing one red ball, there are 4 red left out of 7 total: $P(B/A) = 4/7$
- $P(A \text{ and } B) = (5/8) \times (4/7) = 20/56 = 5/14$

So the probability of drawing two red balls **without replacement** is $\frac{5}{14}$

5. Independent vs Dependent Events - More examples

- **Independent:** The outcome of one event does not affect the other.
- Example: Tossing two coins.
- **Dependent:** One event affects the probability of the other.
- Example: Drawing two cards without replacement.

6. Mutually Exclusive Events

Two events are **mutually exclusive** if they cannot happen at the same time.

Example:

- Event A: Rolling a 2
- Event B: Rolling a 5
- These are mutually exclusive because a die can't show both at once.

Summary

This lesson covered key probability rules and concepts that are foundational for more advanced topics like distributions and statistical inference.

- Complement Rule: $P(A^c) = 1 - P(A)$
- Addition Rule for unions
- Multiplication Rule for intersections
- Understanding independence and exclusivity

Homework / Practice

1. A card is drawn from a standard deck. What is the probability of drawing a red card or a queen?
2. If two dice are rolled, what is the probability that both show even numbers?
3. Think of an example from your daily life where two events are dependent.

1. A box contains 4 red, 3 green, and 2 blue balls. One ball is picked at random. What is the probability that the ball is **not green**?
2. Two coins are tossed. What is the probability of getting **at least one head**?
3. A number is chosen at random from the set $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.
What is the probability that the number is a **multiple of 3 or 5**?
4. A bag contains 6 white and 4 black balls. Two balls are drawn **with replacement**.
What is the probability that **both are white**?
5. A student knows the answer to 70% of the questions on a test.
If one question is selected at random, what is the probability that the student **doesn't know** the answer?



What is Conditional Probability?

Conditional probability is the probability of an event **A** occurring, given that another event **B** has already occurred. It helps us update our understanding based on new information.

We denote it as:

$$P(A/B) = P(A \text{ and } B) / P(B)$$

This is read as: "The probability of A given B."

Law of Total Probability

The **Law of Total Probability** helps you find the probability of an event **A** by breaking it down based on other known events **B1**, **B2**, ..., **Bn**.

Formula:

$$P(A) = P(A/B1) * P(B1) + P(A/B2) * P(B2) + \dots + P(A/Bn) * P(Bn)$$

Conditions to Use This Law:

1. Partition of the Sample Space

The events **B1**, **B2**, ..., **Bn** must:

2. Be mutually exclusive (no overlap):

No two events can happen at the same time.

3. Be collectively exhaustive:

Together, they cover the entire sample space.

4. Known Probabilities

You must know:

5. $P(B_i)$ for each event in the partition.

6. $P(A/B_i)$, the conditional probability of A given B_i .

7. $P(B_i) \rightarrow 0$

The probability of each B_i must be greater than 0, since you can't condition on an impossible event.

Real-life Analogy

Suppose you have a box containing:



Chapter 6: Conditional Probability

SECTION 10

- 5 red balls
- 3 blue balls
- 2 green balls

Now, you're told that the ball picked is **not green**. What is the probability that it's red?

Let:

- A = ball is red
- B = ball is not green

Now, out of the 8 balls that are not green (5 red + 3 blue), 5 are red.

So: $P(\text{Red} / \text{Not Green}) = 5 / 8$

Formula Breakdown

If we know:

- $P(A \text{ and } B)$: The probability that both A and B happen
- $P(B)$: The probability that B happens

Then: $P(A / B) = P(A \text{ and } B) / P(B)$

This means: among all the outcomes where B occurs, how many also include A?

Example Problem

A class has 60% boys and 40% girls. 30% of the boys and 10% of the girls play football. If a randomly selected student is a football player, what is the probability that the student is a boy?

Let:

B = student is a boy → $P(B) = 0.6$

G = student is a girl → $P(G) = 0.4$

- F = student plays football
- $P(F / B) = 0.3$
- $P(F / G) = 0.1$

Now calculate the total probability that a student plays football:

$$P(F) = P(F / B) * P(B) + P(F / G) * P(G)$$

$$P(F) = 0.3 * 0.6 + 0.1 * 0.4 = 0.18 + 0.04 = 0.22$$

Now use conditional probability to find $P(B / F)$:

$$P(B / F) = (P(F / B) * P(B)) / P(F)$$

$$P(B / F) = 0.18 / 0.22 \approx 0.818$$

So, there's about an 81.8% chance that a football player is a boy.

Common Mistakes to Avoid

- Confusing $P(A / B)$ with $P(B / A)$
- Forgetting to divide by $P(B)$
- Assuming $P(A / B)$ is the same as $P(B / A)$, they are not



Bayes' Theorem is written as:

$$P(A / B) = [P(B / A) * P(A)] / P(B)$$

where:

$$P(A / B)$$

is the probability of event A occurring given that event B has occurred.

$$P(B / A)$$

is the probability of event B occurring given that event A has occurred.

$$P(A)$$

is the prior probability of event A.

This can also be interpreted as: $P(A / B) = [P(B / A) * P(A)] / [P(B / A) * P(A) + P(B / \text{No A}) * P(\text{No A})]$

It calculates the probability of event A given that event B has occurred, using the known conditional probability of B given A.

Proof of Bayes' Theorem

We start with the definition of conditional probability:

1. $P(A / B) = P(A \text{ and } B) / P(B)$

2. $P(B / A) = P(A \text{ and } B) / P(A)$

From equation (2), we can write:

$$P(A \text{ and } B) = P(B / A) * P(A)$$

Now substitute this into equation (1):

$$P(A / B) = [P(B / A) * P(A)] / P(B)$$

This is Bayes' Theorem.

Example: Medical Test Problem

Suppose a disease affects 1% of the population. A test for the disease has the following characteristics:

- If a person **has** the disease, the test is **positive** 99% of the time $\rightarrow P(\text{Positive} / \text{Disease}) = 0.99$
- If a person **does not have** the disease, the test is **positive** 5% of the time $\rightarrow P(\text{Positive} / \text{No Disease}) = 0.05$

The chance of having the disease $\rightarrow P(\text{Disease}) = 0.01$

- The chance of not having the disease $\rightarrow P(\text{No Disease}) = 0.99$

A person takes the test and gets a **positive** result. What is the probability they actually have the disease?

Let:

- A = Disease
- B = Positive test

We apply Bayes' Theorem:

$$P(\text{Disease} / \text{Positive}) = [P(\text{Positive} / \text{Disease}) * P(\text{Disease})] / P(\text{Positive})$$

First, calculate $P(\text{Positive})$:

$$P(\text{Positive}) = P(\text{Positive} / \text{Disease}) * P(\text{Disease}) + P(\text{Positive} / \text{No Disease}) * P(\text{No Disease})$$

$$P(\text{Positive}) = 0.99 * 0.01 + 0.05 * 0.99$$

$$P(\text{Positive}) = 0.0099 + 0.0495 = 0.0594$$

Now calculate $P(\text{Disease} / \text{Positive})$:

$$P(\text{Disease} / \text{Positive}) = 0.0099 / 0.0594 \approx 0.1667$$

Conclusion: Even though the test is 99% accurate, the probability that a person actually has the disease given a positive result is only **16.67%**. This is because the disease is very rare, and false positives affect the result significantly.



What is a Probability Distribution?

Imagine you're analyzing real-world data — sales, weather, clicks, heights, etc. You'll notice:

- Some values are more **likely** than others.
- There's a **pattern** to how data is spread.

That pattern is described by a **probability distribution**.

In short:

A probability distribution tells you how likely different outcomes are.

What is a Probability Distribution?

A **probability distribution** is a **mathematical function** or **table** that:

- Assigns **probabilities** to all possible outcomes of a random process.

There are two types:

1. Discrete Probability Distribution

- For outcomes you can **count** (e.g. number of heads in 3 coin tosses).
- Example: **Binomial Distribution**

Think: "What's the probability I get exactly 2 heads in 3 coin tosses?"

2. Continuous Probability Distribution

- For outcomes that can be **any number in a range** (e.g. height, weight).
- Example: **Normal Distribution**

Think: "What's the probability someone's height is between 165 cm and 170 cm?"

A Simple Analogy

Let's say you roll a die:

- Outcomes = {1, 2, 3, 4, 5, 6}
- Each has a probability of $\frac{1}{6}$

This is a **uniform distribution** (discrete).

Now imagine measuring people's **heights**:

- You don't get fixed values.
- Instead, you get a **curve** — most people around average height, fewer very short or very tall.

That's a **normal distribution** (continuous).

Why Are Distributions Useful in Data Analysis?

1. **Model real-world randomness** (user behavior, errors, arrivals, etc.)
2. **Make predictions** (how likely is a customer to buy?)
3. **Run simulations** (A/B testing, Monte Carlo)
4. **Assume underlying patterns** (e.g., linear regression assumes errors are normally distributed)

Summary:

Type	Example	Used For
Discrete	Binomial, Poisson	Count of events
Continuous	Normal, Uniform	Measuring quantities



What It Is:

A **uniform distribution** is when **every outcome is equally likely**.

Simple Example: Rolling a Fair Die

- Possible outcomes: {1, 2, 3, 4, 5, 6}
- Each number has a $1/6$ chance → That's a **discrete uniform distribution**

Continuous Version:

Let's say we randomly pick a number between 0 and 1.

- Every value in that range is **equally likely**.
- That's a **continuous uniform distribution**.

Graphs to Visualize**1. Discrete Uniform (like a die roll)**

Outcome: 1 2 3 4 5 6
Probability: | --- | --- | --- | --- | --- | --- |
 $1/6$ for each → flat bars

Discrete Uniform Distribution Formula:

$$P(X = x) = 1 / n$$

- Where n is the number of possible outcomes (e.g., for a 6-sided die, $P(\text{rolling a 4}) = 1/6$)

2. Continuous Uniform (0 to 1)

- It's just a **flat horizontal line** from $x = 0$ to $x = 1$
- The probability density is constant (say 1.0) across that interval

Continuous Uniform Distribution Formula:

$$f(x) = 1 / (b - a) \text{ for values between } a \text{ and } b$$

- Outside the range a to b , the probability is 0

Why It's Useful in Data Analytics:

- It models **pure randomness**
- It's used to **simulate random choices**
- Used in Generating random numbers (`np.random.uniform`)

In Code (Python):

```
import numpy as np
import matplotlib.pyplot as plt

# Continuous uniform from 0 to 1
samples = np.random.uniform(0, 1, 10000)

plt.hist(samples, bins=50, density=True, alpha=0.6, color='skyblue')
plt.title("Continuous Uniform Distribution (0 to 1)")
plt.xlabel("Value")
plt.ylabel("Probability Density")
plt.grid(True)
plt.show()
```

You'll see a **flat histogram** showing uniform probability.

Summary:

Property	Uniform Distribution
Type	Discrete or Continuous
Shape	Flat
Real-world example	Die roll, random number gen
Python function	<code>np.random.uniform(a, b)</code>

What is the Binomial Distribution?

The **binomial distribution** models the number of **successes** in a fixed number of **independent yes/no experiments**, where each has the **same probability** of success.

Think of this:

- Toss a coin 10 times
- What's the probability of getting **exactly 6 heads**?

That's a binomial problem.

Key Ingredients:

- n = number of trials (e.g., 10 tosses)
- p = probability of success (e.g., 0.5 for heads)
- x = number of successes (e.g., 6 heads)

Plain Text Formula:

$$P(X = x) = C(n, x) \times p^x \times (1 - p)^{(n - x)}$$

Where:

- $C(n, x)$ is "n choose x" = combinations = number of ways to pick x successes out of n
- p^x is the probability of x successes
- $(1 - p)^{(n - x)}$ is the probability of the remaining being failures

Example:

10 coin tosses, what's the probability of exactly 6 heads?

- $n = 10$
- $x = 6$
- $p = 0.5$

$$P(6 \text{ heads}) = C(10, 6) \times 0.5^6 \times 0.5^4 = 210 \times (0.015625) \times (0.0625) \approx 0.205$$

So there's a ~20.5% chance you'll get exactly 6 heads in 10 tosses.

When to Use:

- Email campaign: Will 40 out of 100 people click the link?
- Quality check: How many out of 10 products will be defective?
- A/B testing: Will 60 out of 200 visitors convert?

Summary:

Concept	Value
Type	Discrete
Formula	$P(X = x) = C(n, x) * p^x * (1 - p)^{(n - x)}$
Python	<code>scipy.stats.binom.pmf(x, n, p)</code>
Used for	Count of successes in repeated trials



Normal Distribution (a.k.a. Gaussian Distribution)

Before we dive into the details, let's understand two key concepts related to normal distribution, mean, and standard deviation.

1. What is Mean?

The **mean** (or average) of a dataset is the sum of all values divided by the number of values.

Formula:

$$\text{mean} = (x_1 + x_2 + x_3 + \dots + x_n) / n$$

It represents the central value of the data.

2. What is Standard Deviation?

The **standard deviation** measures how spread out the numbers are from the mean.

Steps to calculate standard deviation:

1. Find the mean
2. Subtract the mean from each value and square the result
3. Take the average of these squared differences (this is the variance)
4. Take the square root of the variance

Formula:

$$\text{standard deviation (sigma)} = \sqrt{(1/n) * \sum((x_i - \text{mean})^2)}$$

A smaller standard deviation means the data points are close to the mean. A larger standard deviation means the data is more spread out.

What is Normal Distribution?

A **normal distribution** is a continuous probability distribution that is **bell-shaped and symmetric** around the mean.

It describes variables where:

- Most values cluster around the **average (mean)**.
- Extreme values (very high or low) are **rare**.

Think of:

Chapter 11: Normal Distribution

SECTION 10

- Heights of people
- Test scores
- Measurement errors

All tend to follow a **normal curve**.

Key Properties:

- **Mean (μ):** Center of the distribution
- **Standard Deviation (σ):** Spread of the distribution

Shape of the Curve:

- Bell-shaped
- Symmetrical
- Peaks at the mean
- About **68%** of the data lies within $\pm 1\sigma$, **95%** within $\pm 2\sigma$, **99.7%** within $\pm 3\sigma$ → This is the famous **68–95–99.7 Rule**

Formula (Plain Text):

$$f(x) = (1 / (\sigma * \text{sqrt}(2\pi))) * e^{-(x - \mu)^2 / (2\sigma^2)}$$

Where:

- μ = mean
- σ = standard deviation
- e = Euler's number (≈ 2.718)
- π = pi (≈ 3.14159)

This gives the **probability density** for a given value x .

You **don't need to memorize** this formula — but understanding its shape and behavior is essential.

In Python:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Generate values
x = np.linspace(-4, 4, 1000)
mean = 0
std_dev = 1

# Get the probability density
y = norm.pdf(x, loc=mean, scale=std_dev)

# Plot
plt.plot(x, y)
plt.title("Standard Normal Distribution ( $\mu=0$ ,  $\sigma=1$ )")
plt.xlabel("x")
plt.ylabel("Probability Density")
plt.grid(True)
plt.show()
```

Summary

Property	Value
Type	Continuous
Shape	Bell curve
Key parameters	Mean (μ), Std. Dev (σ)
Formula	$f(x) = (1 / (\sigma\sqrt{2\pi})) * e^{-(x - \mu)^2 / 2\sigma^2}$
Python	<code>scipy.stats.norm.pdf(x, μ, σ)</code>

1. What is the Mean?

The **mean** (or average) of a dataset is the sum of all values divided by the number of values.

Formula:

$$\text{mean} = (x_1 + x_2 + x_3 + \dots + x_n) / n$$

It represents the central value of the data.

2. What is the Standard Deviation?

The **standard deviation** measures how spread out the numbers are from the mean.

Steps to calculate standard deviation:

1. Find the mean
2. Subtract the mean from each value and square the result
3. Take the average of these squared differences (this is the variance)
4. Take the square root of the variance

Formula:

$$\text{standard deviation (sigma)} = \sqrt{(1/n) * \sum((x_i - \text{mean})^2)}$$

A smaller standard deviation means the data points are close to the mean. A larger standard deviation means the data is more spread out.

3. Central Limit Theorem (CLT)

The **Central Limit Theorem** states:

If you take many random samples of size n from any population (with finite mean and variance), then the distribution of the sample means will tend to be **approximately normal** as n becomes large — regardless of the shape of the original population.

4. Mathematical Expression

Let $X_1, X_2, \dots, X_n$ be n independent, identically distributed (i.i.d) random variables with:

- Mean = μ
- Standard deviation = σ

Then the **sampling distribution of the sample mean** (denoted as $\bar{X}$) approaches a normal distribution with:

Chapter 12: Central Limit Theorem

SECTION 10

- Mean = μ
- Standard deviation = $\sigma / \sqrt{n}$

CodeWithHarry



Statistics is not about complex formulas, it's about **understanding data and making better decisions**.

As a data analyst, your goal is to:

- Understand what kind of data you have
- Summarize it correctly
- Avoid misleading conclusions
- Communicate insights clearly

Data Types

Before doing any analysis, the **first step is to identify the type of data** you're working with.

1. Numerical Data

Numerical data represents quantities and can be measured.

Examples:

- Age (25, 30, 45)
- Salary (₹50,000, ₹1,20,000)
- Number of users (1000, 5000)

You can perform mathematical operations on this data (like average, sum, etc.)

2. Categorical Data

Categorical data represents labels or categories.

Examples:

- Gender (Male, Female)
- City (Delhi, Mumbai)
- Product Category (Electronics, Clothing)

You cannot perform meaningful mathematical operations on this data

Discrete vs Continuous Data

This is a **sub-type of numerical data**.

1. Discrete Data

Discrete data consists of **countable values**.

Examples:



- Number of students in a class (30, 31, 32)
- Number of orders per day

These are whole numbers (no decimals)

2. Continuous Data

Continuous data can take **any value within a range**.

Examples:

- Height (170.5 cm, 172.2 cm)
- Weight (65.3 kg, 70.8 kg)
- Time taken to complete a task

These values can have decimals and are measured, not counted

Levels of Measurement

Understanding levels of measurement helps you decide **how to analyze and interpret data**.

1. Nominal Scale

- Data is categorized **without any order**
- Categories are just labels

Examples:

- Gender (Male, Female)
- Blood Group (A, B, AB, O)
- City names

You can only count frequency (no ranking possible)

2. Ordinal Scale

- Data has a **meaningful order**, but differences are not measurable

Examples:

- Ratings (1 ★, 2 ★, 3 ★, 4 ★, 5 ★)
- Education level (School, College, Postgraduate)
- Customer satisfaction (Low, Medium, High)

You can rank the data, but you cannot measure the exact difference between values

Key Takeaway

Before applying any statistical method, always ask:

"What type of data am I working with?"

Because:

- Data type determines the analysis method
- Wrong assumptions → wrong conclusions



In data analysis, averages help us summarize large amounts of data into a single representative value. However, not all averages tell the same story. Choosing the wrong one can lead to misleading conclusions.

The three most commonly used averages are **Mean**, **Median**, and **Mode**.

Mean

The mean is what most people refer to as the average. It is calculated by adding all values and dividing by the number of values.

Mean = (Sum of all values) / (Number of values)

Example: If salaries are 20k, 25k, 30k, 35k, 40k

Mean = $(20 + 25 + 30 + 35 + 40) / 5 = 30k$

The mean works well when data is evenly distributed and does not contain extreme values.

Problem with mean: If one value is very high or very low, it can distort the result.

Example with outlier: 20k, 25k, 30k, 35k, 200k

Mean = 62k

This suggests a typical salary of 62k, which is misleading.

Median

The median is the middle value when the data is arranged in order.

Example: 20k, 25k, 30k, 35k, 40k

Median = 30k

Example with outlier: 20k, 25k, 30k, 35k, 200k

Median = 30k

The median is not affected by extreme values and often gives a better representation of typical data.

Mode

The mode is the value that appears most frequently.

Example: 10, 20, 20, 30, 40

Mode = 20

Mode is useful for categorical data and for identifying the most common value.

When to Use What

- Use mean when data is evenly distributed and does not have outliers
- Use median when data is skewed or contains extreme values
- Use mode when you want to find the most common value or are working with categories

Real World Insight

In many real datasets such as income, sales, or user behavior, data is often skewed.

Using mean alone can lead to incorrect conclusions.

For example: - Average salary reported using mean may look higher than what most people actually earn - Median often gives a more realistic picture



Averages tell us the central value of data, but they do not tell us how spread out the data is. Two datasets can have the same mean but behave very differently.

Variance and standard deviation help us understand the **variability or spread** of data.

Why Spread Matters

Consider two datasets:

Dataset A: 30, 30, 30, 30, 30

Dataset B: 10, 20, 30, 40, 50

Both have the same mean of 30, but:

- Dataset A is completely consistent
- Dataset B is highly spread out

This difference is what variance and standard deviation capture.

Variance

Variance measures how far each value is from the mean, on average.

Steps to calculate variance:

1. Find the mean
2. Subtract the mean from each value
3. Square the differences
4. Take the average of those squared differences

Variance gives a sense of overall spread, but the values are squared, which makes interpretation less intuitive.

Standard Deviation

Standard deviation is the square root of variance.

This brings the value back to the **same unit as the original data**, making it easier to understand.

Sample Calculation

Consider the dataset: 10, 20, 30, 40, 50

1. Mean = $(10 + 20 + 30 + 40 + 50) / 5 = 30$
2. Differences from mean: -20, -10, 0, 10, 20
3. Squared differences: 400, 100, 0, 100, 400
4. Variance = $(400 + 100 + 0 + 100 + 400) / 5 = 200$
5. Standard Deviation = $\sqrt{200} \approx 14.14$

Intuition

- Low standard deviation means data points are close to the mean
- High standard deviation means data points are spread out

Example:

If average delivery time is 30 minutes:

- Standard deviation of 2 minutes means deliveries are very consistent
- Standard deviation of 15 minutes means delivery times are unpredictable

Real World Examples

Business Example

Two companies have the same average revenue per day:

- Company A: stable revenue every day
- Company B: huge ups and downs

Standard deviation helps identify which business is more consistent and predictable.

User Behavior Example

Average session time is 5 minutes:

- Low standard deviation means most users behave similarly
- High standard deviation means some users leave immediately while others stay much longer

When to Use Which?

- Use variance and standard deviation to understand consistency
- Use them to compare datasets with the same mean
- Use them to detect volatility or instability in data

Mean vs Standard Deviation

Mean tells you what is typical.

Standard deviation tells you how reliable that typical value is.

As a data analyst, both are equally important for making informed decisions.

CodeWithHarry



Section 11: Data Visualization using Power BI

Chapter 1: Introduction to Power BI

Power BI is Microsoft's business intelligence and data visualization platform. It helps users connect data from different sources, analyze the data, and create interactive reports and dashboards.

These dashboards can be shared securely across teams and organizations.

Why Not Just Use Excel Charts?

Excel is great for quick analysis and calculations, but Power BI is built for large-scale reporting and visualization.

In Excel, charts are mostly static. In Power BI, visuals are interactive. Clicking one chart automatically updates other charts and tables on the page.

Power BI also handles much larger datasets and supports automatic data refresh from cloud services, databases, and Excel files.

When Excel is Better

Excel is still the better choice for quick calculations, data entry, and complex financial models.

A simple way to think about it: Use Excel to work with data. Use Power BI to present and share insights.

When to Use Power BI

Power BI becomes useful when:

- Your data grows too large for Excel
- You need interactive dashboards
- Reports need automatic refresh
- Multiple people need access to the same report
- You want secure sharing and centralized reporting

In this section, we will focus on how to use Power BI to create interactive reports and dashboards that can be shared across your organization.



Let's download and install Power BI Desktop. For Windows users, Power BI Desktop is available for free from the official Microsoft website.

Download and Install Power BI Desktop

Follow these steps to download and install Power BI Desktop:

1. Go to the official Power BI website: <https://powerbi.microsoft.com/>
2. Click **Products** and select **Power BI Desktop**.
3. Click the **Download** button and download the installer.
4. Run the installer and follow the on-screen instructions to complete the installation.
5. Launch Power BI Desktop from the Start menu or by searching for **Power BI Desktop**.

Availability on Mac & Linux

Power BI Desktop is designed primarily for Windows. If you use macOS or Linux, you have a few alternatives:

1. **Use Power BI Service**
 - Access Power BI through any modern web browser.
 - Create, view, and share reports without installing the desktop application.
2. **Use Virtualization Software**
 - Install Windows using software such as VMware, VirtualBox, or Parallels Desktop.
 - Run Power BI Desktop inside the Windows virtual machine.
3. **Use Compatibility Layers (Linux)**
 - Some users run Power BI Desktop using Wine, although compatibility is not officially supported.



So far we saw that Power BI is the most common data visualization tool in the market. It is a product of Microsoft and it is used by many organizations to create interactive reports and dashboards.

We can use it to visualize and analyze our data, but before we can do that, we need to connect to our data sources and do some pre processing. Power BI supports a wide range of data sources, including:

- Excel
- SQL Server
- Azure
- SharePoint
- Salesforce

Ways to use Power BI

Power BI Desktop: This is a free application that you can download and install on your windows computer. It allows you to connect to various data sources, transform and model your data, and create reports and dashboards.

Power BI Mobile: This is a mobile application that allows you to view and interact with your reports and dashboards on your mobile device.

Power BI Cloud Service: This is a cloud-based service that allows you to publish and share your reports and dashboards with others. It also provides additional features such as data refresh, collaboration, and integration with other Microsoft services.

Loading a CSV file in Power BI Desktop

To load a CSV file in Power BI Desktop, follow these steps:

1. Open Power BI Desktop and click on "Get Data" from the Home tab.
2. Select "Text/CSV" from the list of data sources and click "Connect".
3. Browse to the location of your CSV file and select it.
4. Click "Load" to load the data into Power BI Desktop. You can also click "Transform Data" to open the Power Query Editor, where you can perform data transformations and cleaning before loading the data into Power BI Desktop.

Sample Data

You can download the sample data "harrykart_ecommerce_data.csv" which contains a sample, easy to understand ecommerce data.

There are three main views in Power BI Desktop that allow you to create and manage your reports and data models:

- 1. Report View:** This is the default view where you create and design your reports. You can add visualizations, arrange them on the canvas, and customize their appearance.
- 2. Data View:** This view allows you to see the underlying data that you have imported into Power BI. You can explore the data, create calculated columns, and manage relationships between tables.
- 3. Model View:** This view is used to manage the relationships between different tables in your data model. You can create and edit relationships, define hierarchies, and manage calculated measures.

Each view serves a specific purpose in the report creation process, and you can switch between them as needed to work on different aspects of your Power BI project.



We will now make changes to important settings in Power BI Desktop to ensure consistent behavior, better performance, and fewer unexpected issues while working with data.

Enable New Object Interaction

This feature improves the way you interact with visuals, making selection and formatting more intuitive.

1. Go to **File**
2. Click **Options and Settings**
3. Select **Options**
4. Navigate to **Preview Features**
5. Enable **New Object Interaction**

Configure Global Data Load Options

Disabling automatic detection prevents Power BI from making incorrect assumptions about your data, giving you full control.

1. Go to **Options**

Navigate to Global → Data Load

2. Set: **Never detect column types and headers**

Set Regional Settings

Setting a consistent region avoids issues with date formats, number formats, and separators.

1. Go to **Options**
2. Navigate to **Regional Settings**
3. Set: **English (United States)**

Configure Current File Settings

These settings ensure your current project behaves predictably and avoids unwanted automatic relationships.

Data Load

Disabling automatic relationships prevents Power BI from creating incorrect links between tables.

[Go to Current File](#) → [Data Load](#)

1. Under **Relationships**
2. Deselect all options

Regional Settings

Ensures the current file follows a consistent format for dates and numbers.

[Go to Current File](#) → [Regional Settings](#)

1. Set:
2. **English (United States)**

Saving PBIX File

Saving to cloud storage ensures easy access, backup, and collaboration.

1. Go to **File**
2. Click **Save As**
3. Choose: **OneDrive** or **SharePoint**



Raw datasets are rarely clean and ready for analysis.

In real-world projects, data often contains:

- Missing values
- Duplicate records
- Incorrect headers
- Wrong data types
- Unnecessary rows and columns
- Formatting inconsistencies

Before building dashboards or visualizations, the data must first be cleaned and transformed.

Power BI provides a powerful tool called the **Power Query Editor** that helps perform these transformations without writing code.

Opening the Power Query Editor

After importing a dataset into Power BI:

1. Click on **Transform Data**

This opens the Power Query Editor where all cleaning and preprocessing operations are performed.

Every transformation applied inside the Query Editor gets stored as a step, making the process repeatable and organized.

Renaming Columns

Column names should always be clear and meaningful.

Datasets sometimes contain confusing column names such as:

- Column1
- Unnamed
- Product_ID_123

These names make analysis difficult.

To rename a column:



1. Double-click the column name
 2. Type the new name
 3. Press Enter
-

Promoting Headers

Sometimes Power BI imports the first row as normal data instead of column headers.

This creates issues because the actual header names appear inside the dataset itself.

To fix this:

1. Go to the **Home** tab
2. Click on **Use First Row as Headers**

Power BI converts the first row into proper column names.

Keeping Rows

In some cases, only a specific portion of the dataset is required.

For example:

- Keeping only the top 100 rows for testing
- Keeping recent records only
- Keeping a sample dataset for analysis

Power BI allows filtering and keeping only required rows. This helps reduce unnecessary data and improves performance during analysis.

Removing Rows

Datasets often contain unwanted rows such as:

- Blank rows
- Extra titles
- Notes
- Invalid records

These rows should be removed before analysis.

To remove rows:



[Go to Home](#) → [Remove Rows](#)

1. Choose the required option

Examples:

- Remove Top Rows
- Remove Bottom Rows
- Remove Blank Rows

Cleaning unwanted rows improves dataset quality.

Removing Duplicates

Duplicate records can create incorrect analysis and misleading insights.

Example:

If the same transaction appears multiple times, total sales calculations become inaccurate.

Power BI allows removing duplicates from selected columns.

To remove duplicates:

1. Select one or more columns
2. Right-click
3. Choose **Remove Duplicates**

If multiple columns are selected, Power BI removes duplicate combinations across those columns.

Example:

City	Product
Delhi	Phone
Delhi	Phone
Mumbai	Laptop

After removing duplicates:

City	Product
Delhi	Phone
Mumbai	Laptop

This ensures cleaner and more reliable analysis.

Going Back to the Query Editor

Even after loading the dataset, transformations can still be modified later.

To reopen the Query Editor:

1. Click on **Transform Data**

This allows updating or adding new transformation steps anytime.

Using Locale to Identify US Dates

Date formats vary across countries.

Example:



Sometimes Power BI misinterprets dates because of regional formatting differences.

To correctly identify US dates:

1. Right-click the column
2. Select **Change Type > Using Locale**
3. Choose the correct data type
4. Select the locale: English (United States)

This ensures dates are interpreted correctly during analysis.

Replacing Errors

Datasets may contain error values because of:

- Invalid calculations
- Incorrect data types
- Missing values
- Conversion issues

These errors can break visualizations and calculations.

Power BI allows replacing errors with valid values.



To replace errors:

1. Select the column
2. Go to **Transform**
3. Click **Replace Values**
4. Choose **Replace Errors**

Example replacements:

- Replace with
- Replace with
- Replace with custom values

This helps maintain clean and consistent datasets for reporting and analysis.



Power BI allows us to add shapes to our reports to improve the design and structure of dashboards. Shapes can be used to highlight sections, display titles, or organize report elements neatly.

Adding Shapes in Power BI

To insert a shape in Power BI:

Insert > Shapes

From the Shapes menu, select a shape like:

- Rounded Rectangle
- Rectangle
- Circle
- Line

After selecting the shape, resize and place it anywhere on the report canvas.

Formatting the Shape

Power BI provides multiple customization options for shapes. To open formatting options:

1. Select the shape
2. Click on the three dots
3. Open the Format panel

The tutorial demonstrates:

- Changing the fill color
- Adjusting corner radius
- Adding padding
- Enabling shadow
- Changing size and position

Adding Text to Shapes

Text can also be added inside shapes.

Example used in the tutorial:

2027 quarter 1 report

The text appears inside the rounded rectangle and can be used as a report heading or section title.

Adding shapes and text enhances the visual appeal of Power BI reports and helps in organizing content effectively.

CodeWithHarry



A stacked bar chart displays **horizontal bars** divided into multiple colored sections.

It is used when you want to compare:

- Total value
- Contribution of sub-categories within that total

Example

Category-wise sales split by city:

Electronics → Delhi + Mumbai + Bangalore

Clothing → Delhi + Mumbai + Bangalore

Fields in Power BI

- **Y-axis:** Main category
- **X-axis:** Numeric value
- **Legend:** Sub-category for color split

Use case

Use it when category names are long and horizontal bars are easier to read, or when comparing categories across different cities or regions.

Stacked Column Chart

A stacked column chart displays **vertical columns** divided into multiple colored sections.

It is used for the same purpose as a stacked bar chart but in vertical format.

Example

Monthly sales split by product type:

January → Phones + Laptops + Accessories

February → Phones + Laptops + Accessories

Fields in Power BI

- **X-axis:** Main category
- **Y-axis:** Numeric value
- **Legend:** Sub-category for color split

Use case

Best when showing trends across months, years, or time-based data and when category names are short enough to fit under vertical columns.

Difference

Stacked Bar Chart	Stacked Column Chart
Horizontal bars	Vertical columns
Better for long labels	Better for time trends
Easier to read category names	Common for monthly comparisons



Clustered Charts

Clustered charts display values **side by side** for comparison.

Each sub-category gets its own separate bar or column.

They are used when you want to compare individual values across groups.

Types

Clustered Bar Chart

Shows **horizontal bars** placed side by side.

Example:

Sales by category across different cities

Electronics → Delhi, Mumbai, Bangalore

Clothing → Delhi, Mumbai, Bangalore

Fields:

Y-axis → Category

X-axis → Sales

Legend → City

Clustered Column Chart

Shows **vertical columns** placed side by side.

Example:

Monthly sales by product type

January → Phones, Laptops, Accessories

February → Phones, Laptops, Accessories

Fields:

X-axis → Month

Y-axis → Sales

Legend → Product Type

100% Stacked Charts

100% stacked charts show **percentage contribution instead of actual values**.

Each bar or column always totals **100%**.

They help compare proportions across categories.

100% Stacked Bar Chart

Shows horizontal bars divided by percentage.



Example:

Employee distribution by department



100% Stacked Column Chart

Shows vertical columns divided by percentage.

Example:

Market share by brand each year

- 2024 → Brand A 50%, Brand B 30%, Brand C 20%

Difference

Clustered Charts	100% Stacked Charts
Compare actual values	Compare percentages
Bars shown side by side	Bars combined into one
Total values visible	Total always equals 100%



Line charts and area charts are used to show changes over time.

They are useful when you want to see trends, growth, decline, or patterns in data.

Line Chart

A line chart connects data points using a line.

Example:

Monthly sales trend from January to December.

Fields:

- X-axis: Date or month
- Y-axis: Numeric value
- Legend: Category, if comparing multiple lines

Use case

Use a line chart when the focus is on movement or trend over time.

Area Chart

An area chart is similar to a line chart, but the area below the line is filled with color.

Example:

Revenue trend over multiple months.

Fields:

- X-axis: Date or month
- Y-axis: Numeric value
- Legend: Category, if needed

Use case

Use an area chart when you want to highlight the size or volume of change over time.

Difference

Line Chart	Area Chart
Shows trend using lines	Shows trend with filled area
Cleaner for multiple categories	Better for showing volume
Best for exact comparison	Best for visual impact

A Card Visual in Power BI is used to display a single important value in a report.

It is commonly used for showing:

- Total Sales
- Profit
- Revenue
- Number of Customers
- KPIs and summary metrics

Adding a Card Visual

1. Open your report in Power BI
2. Go to the **Visualizations** pane
3. Click the **Card** visual icon

A blank card visual will appear on the report canvas.

Adding Data to the Card

1. From the **Fields** pane
2. Drag a numeric field into the Card visual

Power BI will automatically calculate and display the value.

Example:

- Sales Amount
- Total Profit
- Quantity Sold

Formatting the Card Visual

You can customize the card using the **Format** pane.

Common formatting options include:



- Data label size
- Font color
- Background color
- Title
- Category label
- Alignment

Using Card Visuals

Card visuals are useful for highlighting key business insights.

They are commonly placed at the top of dashboards to quickly display important numbers.



Power BI provides data profiling tools inside the Power Query Editor.

These tools help us quickly understand the quality and structure of our data before creating reports.

Opening Data Profiling

1. Click **Transform Data**
2. Go to the **View** tab
3. Enable:
4. Column Quality
5. Column Distribution
6. Column Profile

Column Quality

Column Quality shows how much data is:

- Valid
- Error
- Empty

This helps find missing values and incorrect entries.

Column Distribution

Column Distribution shows unique and distinct values in a column.

It is useful for checking repeated values, categories, and unusual patterns.

Column Profile

Column Profile gives a deeper summary of the selected column.

It can show:

- Count
- Empty values
- Errors
- Minimum and maximum values
- Value distribution



Pie charts and donut charts show how a total is divided into parts.

They look simple, but they are often not the best choice for analysis.

Why Pie Charts Can Be Problematic

Pie charts become hard to read when there are many categories.

It is difficult for the human eye to compare slice sizes accurately.

Small differences between categories are especially hard to notice.

Problems with Donut Charts

Donut charts have the same issues as pie charts.

The hole in the middle makes the slices thinner, so comparison can become even harder.

They may look cleaner, but they do not always communicate data clearly.

When They Become Confusing

Pie and donut charts are not ideal when:

- There are too many categories
- Values are very similar
- Some values are very small
- Exact comparison is important
- Category labels are long

Better Alternatives

In many cases, use a bar chart or column chart instead.

Bar charts make comparison easier because all values start from the same baseline.

Example:

If you want to compare sales by product category, a bar chart is usually clearer than a pie chart.

When Pie or Donut Charts Are Acceptable

They can be used when:



- There are only 2 to 4 categories
- The difference between values is obvious
- You only want to show a simple share of total
- The visual is used for a quick summary, not deep analysis

Good Rule

Use pie and donut charts only when the audience can understand the message at a glance.

If the reader has to carefully inspect the slices, choose another chart.



Pie charts and donut charts are used to show part-to-whole relationships.

They help answer questions like:

- Which category contributes the most?
- What percentage of the total comes from each group?
- How is the total divided?

Creating a Pie Chart

1. Select the **Pie chart** visual from the Visualizations pane
2. Add a category field to **Legend**
3. Add a numeric field to **Values**

Example:

Sales share by product category.

Fields:

- Legend: Product Category
- Values: Sales

Power BI will divide the circle into slices based on the values.

Creating a Donut Chart

1. Select the **Donut chart** visual from the Visualizations pane
2. Add a category field to **Legend**
3. Add a numeric field to **Values**

Example:

Profit share by region.

Fields:

- Legend: Region
- Values: Profit

A donut chart works like a pie chart, but it has a blank center.

Formatting Options

Common formatting options include:



- Data labels
- Detail labels
- Legend position
- Slice colors
- Title

Data labels can show values, percentages, or both.

Best Practices

Keep the chart simple.

Use only a few categories so the chart remains readable.

If there are many categories, group smaller values into an **Others** category or use a bar chart instead.

Difference

Pie Chart	Donut Chart
Full circle divided into slices	Circle with a blank center
Good for simple share of total	Slightly cleaner visual appearance
Can become crowded quickly	Can also become hard to compare



Slicers in Power BI are used to filter data interactively. They help users view specific portions of data based on selected values.

When a slicer is applied, all connected visuals update automatically according to the selected filter.

Inserting a Slicer

1. Click on the **Slicer** visual from the Visualizations pane
2. A slicer visual will appear on the report canvas
3. Add a field to the slicer

Example: Filtering visuals category wise.

Choose the field **Category**

After selecting a category like **Clothing**, all visuals update to show only clothing-related data.

If **Electronics** is selected, all visuals display only electronics data.

How Slicers Work

Slicers filter all related visuals on the report page.

For example:

- Revenue values update
- Charts update
- Tables update
- KPIs update

All visuals show data only for the selected slicer value.

Dropdown Slicer

A slicer can also be displayed as a dropdown.

1. Select the slicer
2. Change its display type to **Dropdown**

This allows users to choose values from a compact dropdown list.

Date Slicer

A slicer can also be created using date fields.

1. Insert a new slicer
2. Add the **Date** field

Users can then filter visuals for a specific date range.

Example: Viewing data from one date to another date.

Additional Slicer Examples

Other fields can also be used inside slicers.

Examples:

- Category
- Date
- Order ID

Order ID slicers can filter records between selected values.

Formatting Options

Slicer formatting options include:

- Changing slicer colors
- Removing the slider
- Changing font of values
- Removing slicer header
- Adding a slicer title
- Changing size and position using properties

Number Formatting

Formatting settings can also control how values appear.

Examples:

- Showing values in percentage format
- Showing values with thousand separators

Organizing Filters

Text boxes can be added near slicers to organize filters visually.

Examples:



- Filters
- Category Filters
- Date Filters

This helps make the report layout cleaner and easier to understand.

CodeWithHarry



Filters in Power BI are used to display only specific data based on selected conditions.

They help users focus on particular values, categories, dates, or ranges without changing the entire dataset.

Unlike slicers, filters can be applied at different levels.

Types of Filters in Power BI

Power BI provides different types of filters:

- Filters on this visual
- Filters on this page
- Filters on all pages

Each filter level affects the report differently.

Filters on This Visual

Visual-level filters affect only one selected visual.

1. Select a visual
2. Open the Filters pane
3. Add a field to **Filters on this visual**
4. Choose the required values

Example: Filtering a chart to display sales data only for April.

In this example, the **Month** field is added to the visual-level filter.

After selecting **April**, only that particular visual updates to show April data.

Other visuals on the page remain unchanged.

Using Date Filters

Date fields can also be used inside filters.

1. Select the visual
2. Add the Date or Month field to the filter area
3. Choose the required dates or months

Example: Viewing data only for January, February, and March.

This helps analyze first-quarter performance separately.



Basic Filtering

Basic filtering allows users to manually select values from a list.

Examples:

- Selecting specific months
- Selecting particular categories
- Selecting certain dates

Only the selected values appear in the visual.

Top N Filtering

Power BI also supports Top N filtering.

This is used to display top or bottom values based on a measure.

Example: Showing top-selling categories based on sales amount.

Removing Filters

Filters can be cleared or removed anytime from the Filters pane.

Once removed, the visual returns to showing complete data.

Difference Between Slicers and Filters

Slicers	Filters
Visible directly on the report canvas	Applied through the Filters pane
Used mainly for interactive filtering	Used for more controlled filtering
Usually affects multiple visuals	Can affect one visual, one page, or all pages
Easier for end users to interact with	Commonly used while designing reports

Tables in Power BI are used to display detailed data in rows and columns.

They help users view exact values instead of summarized visuals.

Tables are useful when you want to analyze individual records, categories, products, cities, or revenue values.

Creating a Table Visual

1. Select the **Table** visual from the Visualizations pane
2. Add the required fields to the table

Example: Displaying revenue and product data city wise.

In this example, fields like **City**, **Product**, and **Revenue** are added to the table visual.

The table then displays detailed values for each city and product.

Sorting Data in a Table

Table data can be sorted based on any column.

1. Click on a column header
2. Power BI sorts the table according to that field

Example: Sorting cities by revenue.

This helps identify cities with the highest sales or revenue values.

Applying Filters to a Table

Filters can also be applied directly to table visuals.

1. Select the table visual
2. Open the Filters pane
3. Add a field to **Filters on this visual**
4. Apply the required filter

Using Top N Filter

Top N filtering helps display only the top or bottom records.

1. Add the required field to the filter section
2. Select **Top N**
3. Enter the required number

Example: Showing only the top 2 cities based on revenue.

After applying the filter, the table displays data only for the top-performing cities.

Modifying Table Data

Tables can be customized to display only useful information.

Showing too much data inside a table may make the report difficult to read.

Filters and sorting help make the table cleaner and easier to analyze.

Formatting Options

Power BI provides multiple formatting options for tables.

These include:

- Changing table colors
- Turning totals on or off
- Changing border colors
- Modifying horizontal grid lines
- Customizing text appearance

Grid Line Formatting

Grid lines can also be customized.

Users can:

- Remove horizontal grid lines
- Change grid line colors
- Enable or disable borders

This helps improve the visual appearance of the table.



A Matrix Visual is similar to a table but provides additional features such as grouping, hierarchies, and subtotals.

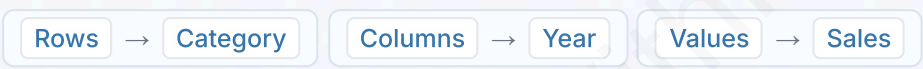
It is often compared to an Excel Pivot Table because it allows data to be summarized and analyzed in a structured format.

Creating a Matrix Visual

To create a Matrix Visual:

- 1. Select the **Matrix** visual from the Visualizations pane.
- 2. Drag fields into:
- 3. **Rows**
- 4. **Columns**
- 5. **Values**

Example:



Power BI automatically summarizes the data and displays it in a matrix format.

Benefits of Matrix Visuals

Matrix visuals allow you to:

- Group related data
- Display subtotals and grand totals
- Expand and collapse hierarchies
- Analyze data across rows and columns

This makes Matrix Visuals extremely useful for business reports and dashboards.

Table vs Matrix

Table	Matrix
Displays raw data	Displays summarized data
No grouping	Supports grouping and hierarchies
Similar to an Excel sheet	Similar to an Excel Pivot Table

Matrix visuals are commonly used when creating professional business reports and dashboards.

In real-world projects, data is rarely stored in a single table.

Instead, information is usually spread across multiple tables.

For example:

Customers Table

CustomerID	CustomerName
1	Harry
2	Amit
3	Sara

Orders Table

OrderID	CustomerID	Amount
101	1	500
102	1	700
103	2	300

Here, the customer names are stored in one table, while order information is stored in another.

Power BI needs a way to understand how these tables are connected.

This is where **relationships** come in.

What is a Relationship?

A relationship is a connection between two tables based on a common column.

In the example above:

```
Customers[CustomerID]
      ↓
Orders[CustomerID]
```

The `CustomerID` column exists in both tables.

Power BI uses this common column to connect the tables together.

Why are Relationships Important?

Relationships allow Power BI to:

- Combine data from multiple tables
- Create accurate reports
- Enable filtering across tables
- Build meaningful visualizations

Without relationships, Power BI treats each table separately and cannot correctly connect related information.

Opening Model View

To create relationships:

1. Open Power BI Desktop
2. Click on **Model View**

Model View displays all tables and their connections in a visual format.

This is where relationships are created and managed.

Creating a Relationship

To create a relationship:

1. Locate the common column in both tables
2. Drag the column from one table to the matching column in another table

Example:

Customers[CustomerID]



Orders[CustomerID]

Power BI automatically creates the relationship.

A line appears between the two tables indicating that they are now connected.

Understanding One-to-Many Relationships

The most common type of relationship in Power BI is the **One-to-Many Relationship**.

Consider the following:

Customers Table

CustomerID
1
2
3

Orders Table

CustomerID
1
1
1
2

A customer appears only once in the Customers table but can appear multiple times in the Orders table.

This means:

One Customer
↓
Many Orders

This is called a **One-to-Many Relationship**.

Examples:

Customer → Orders

Product → Sales

Department → Employees

Most Power BI projects use one-to-many relationships.

How Relationships Affect Visualizations

Once relationships are created, Power BI can combine information from different tables.

For example:

A table visual can display:



Customer Name	Total Sales
Harry	1200
Amit	300

Even though customer names and sales values are stored in different tables.

This becomes possible because Power BI understands the relationship between the tables.

Active Relationships

When a relationship is created, Power BI marks it as an **Active Relationship**.

An active relationship is automatically used by Power BI when creating reports and visualizations.

In Model View, active relationships appear as solid lines between tables.

Real-World Example

A typical business dashboard may contain multiple tables:

Customers
Products
Orders
Employees
Regions

Relationships connect these tables together and allow Power BI to generate meaningful insights.

Without relationships:

- Filters may not work correctly
- Totals may become inaccurate
- Visualizations may show incorrect results



DAX stands for **Data Analysis Expressions**.

It is the formula language used in Power BI to perform calculations and derive insights from data.

Although DAX is similar to Excel formulas, it is specifically designed for Power BI.

Sample Dataset Used

In this lesson, we will be working with a sample dataset.

The dataset contains columns such as:

City	Category	Product	Quantity	Price	Revenue

The data is stored in a CSV file and imported into Power BI.

Importing CSV Data into Power BI

To import a CSV file:

1. Open Power BI Desktop.
2. Create a new blank report.
3. Click **Get Data**.
4. Select **Text/CSV**.
5. Choose the CSV file.
6. Click **Load**.

The dataset will be available for analysis inside Power BI.

Creating Your First DAX Measure

Measures are used to perform calculations on data in Power BI.

To create a measure:

1. Select the table.
2. Click **New Measure**.
3. Enter a DAX formula.

Example:

```
Total Revenue = SUM(Sales[Revenue])
```

This measure calculates the total revenue from the Revenue column.

Displaying a Measure in a Card Visual

A measure can be displayed using a **Card** visual.

For example:

```
Total Revenue = SUM(Sales[Revenue])
```

Dragging this measure into a Card visual displays the total revenue value.

Basic Aggregation Functions in DAX

Common aggregation functions demonstrated in this lesson are:

- SUM()
- COUNT()
- DISTINCTCOUNT()
- AVERAGE()
- MAX()
- MIN()

Finding Unique Cities

To count the number of unique cities:

```
Unique Cities = DISTINCTCOUNT(Sales[City])
```

This measure returns the number of distinct cities in the dataset.

DAX Measures vs Calculated Columns

A calculated column creates a value for every row.

Example:

```
Revenue = Price × Quantity
```

This calculation is performed separately for each row.

A DAX measure works on the dataset as a whole and returns summarized results without creating a new column.

Renaming Tables

The dataset table can be renamed for better readability.

Example:

```
HarryKart E-commerce Data
      ↓
    Sales
```

When a table is renamed, Power BI automatically updates existing DAX formulas that reference that table.

Counting Total Orders

To count the total number of orders:

```
Total Orders = COUNT(Sales[OrderID])
```

This measure counts the Order IDs in the dataset.

Counting Unique Products

To count unique products:

```
Unique Products = DISTINCTCOUNT(Sales[Product])
```

This measure counts distinct product names instead of every occurrence.

Calculating Average Revenue

To calculate average revenue:

```
Average Revenue = AVERAGE(Sales[Revenue])
```


This measure returns the average revenue value and can be displayed in a Card visual.

Finding the Highest Sale

To determine the highest revenue value:

```
Highest Sale = MAX(Sales[Revenue])
```

This measure returns the maximum revenue recorded in the dataset.

Finding the Lowest Sale

To determine the lowest revenue value:

```
Lowest Sale = MIN(Sales[Revenue])
```

This measure returns the minimum revenue value in the dataset.

Using Existing Measures Inside Other Measures

DAX measures can reference other measures.

Example:

```
Revenue Per Order =  
DIVIDE(  
    [Total Revenue],  
    [Total Orders]  
)
```

Here, the measure divides Total Revenue by Total Orders to calculate revenue per order.

Filtering Calculations with CALCULATE

The `CALCULATE()` function performs calculations using specific conditions.

Example:



```
Delhi Revenue =  
CALCULATE(  
    SUM(Sales[Revenue]),  
    Sales[City] = "Delhi"  
)
```

Understanding the Formula

The formula contains:

1. An expression to calculate.
2. A filter condition.

Example:

```
Calculate Revenue  
    +  
City = Delhi
```

Only rows where the city is Delhi are included in the calculation.

Important Note About Conditions

Inside DAX conditions, a single equals sign is used:

```
Sales[City] = "Delhi"
```

Not:

```
Sales[City] = "Delhi"
```

Using IF Statements in DAX

DAX can return text based on conditions.

Example:



```
Performance =  
IF(  
    [Total Revenue] > 100000,  
    "Excellent",  
    "Performance Needs Improvement"  
)
```

Logic

```
If Total Revenue > 100000  
    ↓  
    Excellent  
  
Otherwise  
    ↓  
    Performance Needs Improvement
```

This measure can be displayed in a Card visual to show performance status.



When working with Power Query, every transformation you perform is recorded as a series of steps. These steps are stored behind the scenes as code written in **M Language**.

Power BI automatically generates this code whenever you transform data.

Importing Data for Transformation

Instead of directly loading data into Power BI, you can choose to transform it first.

To do this:

1. Click **Get Data**.
2. Select **Text/CSV**.
3. Choose the dataset.
4. Click **Transform Data** instead of **Load**.

This opens the Power Query Editor, where data transformations can be performed.

Understanding Applied Steps

When data is imported and transformed, Power Query records every action as an applied step.

For example:

```
Source
↓
Promoted Headers
↓
Changed Type
```

These steps are visible in the **Applied Steps** pane.

Examples of transformations include:

- Promoting headers
- Changing data types
- Removing rows
- Adding columns
- Other data-cleaning operations

Why M Language is Useful

Suppose another dataset has exactly the same structure but different values.

Instead of manually repeating every transformation, the same set of steps can be reused.

Power BI stores all transformation logic in M Language, making it possible to:

- View transformation code
- Reuse transformation logic
- Share transformation steps with others

Opening the Advanced Editor

To view the M Language code:

1. Open Power Query Editor.
2. Click **Advanced Editor**.

The Advanced Editor displays all applied steps as M Language code.

What is M Language?

M Language is the language used by Power Query to perform data transformations.

Just like Python, JavaScript, Java, and other programming languages have their own syntax, M Language also has its own syntax.

You do not need to memorize every detail of M Language syntax, but understanding the basics can help you read and modify Power Query transformations.

Understanding Let and In

The foundation of M Language is based on two keywords:

```
let
    Steps
in
    Final Result
```

Let

The `let` section contains all the transformation steps that Power Query performs.

Examples include:

- Importing data
- Promoting headers
- Changing data types
- Removing rows

In

The `in` section specifies the final result that should be returned.

Power Query executes all steps defined in the `let` section and returns the result specified after `in`.

Example Transformation Flow

A simple transformation process may look like this:

```
Source
↓
Promoted Headers
↓
Changed Type
```

In the Changed Type step, columns can be assigned appropriate data types.

For example:

Column	Data Type
Order ID	Int64
Date	Date
City	Text
Category	Text
Product	Text
Quantity	Int64
Price	Int64
Revenue	Int64

These transformations are represented as M Language code inside the Advanced Editor.

Understanding Pound (#) and Double Quotes

While reading M Language code, you may notice syntax like:

```
#"Promoted Headers"
```

This does **not** represent a comment.

Power Query automatically uses `#` and double quotes when step names contain:

- Spaces
- Special characters

For example:

```
#"Promoted Headers"  
#"Changed Type"
```

These are simply step names, not comments.

Writing Comments in M Language

M Language supports both single-line and multi-line comments.

Single-Line Comment

```
// This is a comment
```

Multi-Line Comment

```
/* This is a  
multi-line  
comment */
```

Comments are ignored when the code is executed.

Using ChatGPT with M Language

Since Power Query transformations are stored as M Language code, the code can be copied and shared with tools such as ChatGPT.

For example, you can ask:

Remove the top 3 rows from this M Language code and return the updated code.

ChatGPT can modify the code and return an updated version.

The updated code can then be pasted back into the Advanced Editor to apply the new transformation.

This can be useful when making changes to existing Power Query transformations without manually editing every step.

CodeWithHarry



After creating a report in Power BI Desktop, it can be published online using **Power BI Service**.

This allows stakeholders to view and interact with reports through a web browser without needing the `.pbix` file.

Power BI Desktop vs Power BI Service

Power BI Desktop is used to:

- Create reports
- Build visualizations
- Analyze data

Power BI Service is used to:

- Publish reports online
- Access reports through a browser
- Share reports with others

Publishing a Report

To publish a report:

1. Open the report in Power BI Desktop.
2. Click **Publish**.
3. Sign in to your Power BI account.
4. Select a workspace.
5. Complete the publishing process.

The report will then be available in Power BI Service.

Accessing Power BI Service

Power BI Service can be accessed through:

`app.powerbi.com`

Reports can be viewed and managed from the web portal after publishing.

Understanding Workspaces

A workspace is used to organize reports and projects.

By default, Power BI provides:

My Workspace

Additional workspaces can be created for different teams or projects.

Sharing Reports

Publishing a report does not automatically make it public.

Users must have appropriate access permissions to open and view a report.

Published

≠

Public

Publish to Web

Power BI includes a **Publish to Web** feature that creates a public link for a report.

To access it:

1. Open the published report.
2. Click **File**.
3. Select **Embed Report**.
4. Choose **Publish to Web**.

This makes the report accessible through a public web link.

Enabling Publish to Web

If the option is unavailable:

1. Open **Admin Portal**.
2. Go to **Tenant Settings**.
3. Find **Publish to Web** settings.
4. Enable:

Allow users to create embed codes

Changes may take some time to apply.

Important Warning

Before generating a public link, Power BI displays a warning.

Reports published using **Publish to Web** should not contain confidential or sensitive information, as anyone with the link can access them.



In this mini project, we will build a complete sales dashboard for **ClipKart**, using Power BI.

The project begins with importing and preparing the sales dataset. During this stage, we will perform basic data cleaning tasks, remove duplicate records, and recreate the Revenue column using existing fields to ensure the data is ready for analysis.

Once the data preparation is complete, we will design a professional dashboard layout by adding a title, company branding, and visual elements that improve the overall appearance of the report.

The dashboard will include several key business metrics and visualizations, such as:

- Total Revenue
- Total Orders
- Total Quantity Sold
- Revenue by Category
- Price vs Revenue Analysis
- Top Revenue Products
- Top Performing Cities

Using these visualizations, we will explore sales trends, compare category performance, identify high-revenue products, and analyze which cities contribute the most to overall revenue.

We will also customize and format the visuals to create a clean, organized, and business-friendly dashboard.

By the end of this project, you will have a complete e-commerce sales dashboard that transforms raw data into meaningful business insights and supports data-driven decision-making.



Section 12: Tableau for Data Analytics

Chapter 1: Introduction to Tableau

Tableau is one of the most popular data visualization and business intelligence tools used by organizations around the world.

It helps users transform raw data into meaningful insights through interactive charts, reports, and dashboards.

Tableau is widely used by data analysts, business analysts, and decision-makers to analyze data and communicate findings effectively.

Why Do We Need Tableau?

As businesses collect more data, analyzing information using spreadsheets alone becomes difficult.

Organizations often need to:

- Track sales performance
- Analyze customer behavior
- Monitor key business metrics
- Create reports for management
- Identify trends and patterns

Tableau helps present this information in a visual and interactive format that is easier to understand.

What Can Tableau Connect To?

Tableau can connect to a wide variety of data sources, including:

- Excel Files
- CSV Files
- SQL Databases
- Google Sheets
- Cloud Platforms
- Data Warehouses

This allows analysts to work with data from multiple systems in a single dashboard.

Tableau vs Excel

Excel is excellent for:



- Data entry
- Calculations
- Small datasets
- Financial modeling

Tableau is designed for:

- Data visualization
- Interactive dashboards
- Large datasets
- Business reporting

A simple way to think about it:

Use Excel to work with data. Use Tableau to visualize and communicate insights.

Tableau vs Power BI

Power BI and Tableau are two of the most widely used business intelligence tools.

Power BI	Tableau
Developed by Microsoft	Developed by Tableau (Salesforce acquired in 2019)
Strong integration with Microsoft products	Known for powerful visualizations
Popular in enterprises	Popular in analytics teams
Often more cost-effective	Often preferred for advanced dashboard design

Both tools are highly capable and widely used in the industry.

The choice often depends on organizational requirements and existing technology stacks.

Tableau Products

Tableau provides multiple products for different use cases.

Tableau Public

Tableau Public is a free version of Tableau.

It is commonly used for:

- Learning Tableau
- Practice projects
- Building a portfolio

Dashboards published using Tableau Public are publicly accessible.

Tableau Desktop

Tableau Desktop is the professional version of Tableau.

It is used to:

- Create dashboards
- Analyze data
- Connect to enterprise data sources

A paid license is required to use Tableau Desktop.

Tableau Cloud and Tableau Server

These products are used by organizations to share dashboards and reports with teams.

They allow secure access, collaboration, and centralized reporting.



In this section, we will install **Tableau Public**, the free version of Tableau.

Tableau Public is ideal for:

- Learning Tableau
- Practicing data visualization
- Building dashboards
- Creating portfolio projects

Since it is free, every student can follow along without purchasing a license.

Why Tableau Public?

Tableau offers multiple products, but Tableau Public is the best option for beginners.

Benefits:

- Free to use
- No credit card required
- Access to most core visualization features
- Allows publishing dashboards online

One important limitation is that dashboards published through Tableau Public are publicly accessible.

Therefore, do not upload confidential or sensitive business data.

Downloading Tableau Public

To download Tableau Public:

1. Open your browser.
2. Visit the official Tableau Public website:

<https://public.tableau.com/>

1. Click on **Download Tableau Public**.
2. Download the installer for your operating system.

Installing Tableau Public on Windows

After downloading the installer:



1. Double-click the downloaded file.
2. Accept the license agreement.
3. Follow the installation wizard.
4. Wait for the installation to complete.
5. Launch Tableau Public.

The installation process is similar to installing any standard Windows application.

Creating a Tableau Public Account

To publish dashboards online, you will need a Tableau Public account.

1. Open Tableau Public.
2. Click **Sign Up**.
3. Create a free account using your email address.
4. Verify your email if required.

After signing in, you can save and publish dashboards directly from Tableau.

Launching Tableau Public

After opening Tableau Public, you will see:

- Connect Pane
- Data Sources
- Sheets
- Dashboards
- Visualizations Workspace

This is where we will create charts and dashboards throughout this section.



In this section, we will connect Tableau to a dataset and understand how Tableau imports and organizes data for analysis.

We will use the same e-commerce dataset that was previously used in Power BI so that we can focus on learning Tableau without worrying about a new dataset.

Understanding Data Connections in Tableau

Before creating visualizations, Tableau needs to connect to a data source.

Tableau supports a wide variety of data sources, including:

- Excel Files
- CSV Files
- SQL Databases
- Google Sheets
- Cloud Data Sources

For this lesson, we will connect Tableau to an Excel dataset.

Connecting to an Excel File

To connect Tableau to an Excel file:

1. Open Tableau Public.
2. From the Connect pane, click on **Microsoft Excel**.
3. Browse to the location of your dataset.
4. Select the Excel file.
5. Click **Open**.

Once the file is loaded, Tableau will display the Data Source page.

Exploring the Data Source Page

After connecting the dataset, Tableau shows the Data Source page.

Here you can:

- View tables from the dataset
- Check column names
- Verify data types
- Preview records
- Manage relationships between tables

This page helps ensure that the data has been imported correctly before creating visualizations.

Understanding Dimensions and Measures

After loading the dataset, Tableau automatically categorizes fields into:

Dimensions

Dimensions contain descriptive information such as:

- City
- State
- Customer Name
- Product Category

These fields are generally used for grouping and categorization.

Measures

Measures contain numerical values such as:

- Sales
- Profit
- Quantity
- Revenue

These fields are generally used for calculations and visualizations.

Creating Your First Visualization

Now that the data has been connected successfully, we can create our first chart.

We will create a simple bar chart to analyze revenue across different cities.

This visualization will help us understand how Tableau automatically generates charts using dimensions and measures.

Saving the Workbook

After creating visualizations, it is important to save your work.

To save the workbook:

1. Click on **File**.
2. Select **Save As**.
3. Choose a location.
4. Enter a workbook name.
5. Save the file.

Tableau workbooks are typically saved using the **.twb** format.

These concepts form the foundation for building more advanced charts and dashboards in Tableau.



In this section, we will explore how to create different types of visualizations in Tableau using the dataset we imported earlier.

Tableau makes it easy to switch between chart types and quickly discover insights from your data with just a few clicks.

Opening the Existing Workbook

To continue working with your dataset:

1. Open Tableau Public.
2. Open your previously saved workbook or drag the workbook file into Tableau.
3. Wait for the data and visualizations to load.

This allows you to continue building on your existing analysis without importing the data again.

Using the Show Me Panel

One of Tableau's most useful features is the **Show Me** panel.

It allows you to instantly convert your data into different types of visualizations.

Some of the available chart types include:

- Bar Charts
- Pie Charts
- Heat Maps
- Tree Maps
- Line Charts
- Scatter Plots

Simply select the required fields and choose a visualization from the Show Me panel.

Creating a Pie Chart

To create a pie chart:

1. Select the required dimensions and measures.
2. Open the **Show Me** panel.
3. Click on **Pie Chart**.

Tableau will automatically generate the visualization based on the selected data.

If required, you can also display the chart legend from the **Analysis** → **Legends** menu.

Displaying Labels on Charts

Labels make charts easier to understand by displaying values directly on the visualization.

To show labels:

1. Open the **Marks** card.
2. Click on **Label**.
3. Enable **Show Mark Labels**.

You can also drag a measure such as Revenue onto the Label section to display numerical values on the chart.

Creating and Sorting a Bar Chart

Bar charts are one of the most commonly used visualizations for comparing values across categories.

For example, you can display Revenue by City using a bar chart.

You can also sort the bars:

- Highest to Lowest
- Lowest to Highest

Sorting helps quickly identify the best-performing and lowest-performing categories.

Using Colors for Better Insights

Tableau allows you to color visualizations based on another field.

For example:

1. Drag the **Category** field onto **Color** in the Marks card.
2. Tableau will automatically assign different colors to each category.
3. The visualization becomes easier to interpret and compare.

This makes it simple to understand which product categories contribute the most revenue in different cities.

Understanding Data Through Visualizations

By experimenting with different chart types, labels, colors, and sorting options, you can quickly uncover meaningful patterns and business insights from your data.

Tableau's interactive interface encourages exploration and makes data analysis both efficient and intuitive.

CodeWithHarry



In this section, we will learn how to use filters in Tableau to display only the data that is relevant for our analysis.

Filters allow us to narrow down large datasets and focus on specific categories, cities, dates, or any other field available in the data.

Understanding Filters

Filters help display only a selected subset of data instead of the complete dataset.

For example, you may want to view:

- Only Electronics sales
- Only Clothing sales
- Data for a specific city
- Data for a particular quarter
- Multiple selected values together

This makes analysis more focused and meaningful.

Applying a Category Filter

To filter data by category:

1. Locate the **Category** field.
2. Drag it to the **Filters** shelf.
3. A dialog box will appear.
4. Select the desired category.
5. Click **Apply** and then **OK**.

The chart will now display data only for the selected category.

For example, selecting **Clothing** will show only clothing-related data in the visualization.

Changing Filter Values

Filters can be modified at any time.

Simply:



1. Double-click the existing filter.
2. Select a different value such as **Electronics**.
3. Click **Apply**.
4. Click **OK**.

The visualization updates instantly to reflect the new selection.

Applying a City Filter

You can also filter data based on cities.

To do this:

1. Drag the **City** field to the **Filters** shelf.
2. Select one or more cities.
3. Click **Apply**.
4. Click **OK**.

The chart will now display data only for the selected locations.

Using Multiple Filters

Tableau allows multiple filters to be applied simultaneously.

For example, you can combine:

- Category = Electronics
- City = Mumbai
- Quarter = Q1 and Q2

By combining multiple filters, you can analyze very specific subsets of your data with ease.

Applying a Date Filter

Date filters are useful for time-based analysis.

To apply one:



1. Drag the Date field to the **Filters** shelf.
2. Choose the desired date option.
3. Select values such as specific quarters or months.
4. Apply the filter.

For example, selecting **Q1** and **Q2** will display data only for those quarters.

Removing a Filter

If you no longer need a filter:

1. Select the filter from the Filters shelf.
2. Drag it outside the shelf or remove it.
3. The visualization will return to displaying the full dataset.

This allows you to quickly switch between filtered and complete views.



In the previous lectures, we created a basic visualization using Tableau.

Tableau supports a wide variety of chart types that help us analyze data from different perspectives.

One of the easiest ways to create charts in Tableau is by using the **Show Me** panel.

Creating a Line Chart

Line charts are commonly used to visualize trends over time.

Steps:

1. Drag **Date** to Columns.
2. Drag **Revenue** to Rows.
3. If Tableau does not automatically create a line chart, open **Show Me** and select **Line Chart**.

Line charts are useful for analyzing how values change over time.

Creating a Pie Chart

Pie charts show the contribution of different categories to a whole.

Steps:

1. Select **Category** and **Revenue**.
2. Open **Show Me**.
3. Select **Pie Chart**.

Pie charts should be used when comparing a small number of categories.

Creating a Scatter Plot

Scatter plots help identify relationships between numerical variables.

Steps:

1. Drag **Price** to Columns.
2. Drag **Revenue** to Rows.
3. Open **Show Me**.
4. Select **Scatter Plot**.

Each point in the chart represents a data record.

Creating a Treemap

Treemaps display data using rectangles whose sizes represent values.

Steps:

1. Select **Category** and **Revenue**.
2. Open **Show Me**.
3. Select **Treemap**.

Treemaps are useful for comparing the contribution of different categories.

Using Show Me

The **Show Me** panel suggests suitable visualizations based on the fields selected.

Instead of memorizing every chart type, you can select the desired fields and allow Tableau to recommend appropriate visualizations.

Tableau provides many visualization options, and the Show Me panel makes it easy to discover and create them.



In the provided `data.csv` file, we have geographical information such as **City** and **State**, along with business metrics like **Revenue** and **Orders**. Tableau can use this geographical data to create map visualizations.

Symbol Map

A Symbol Map displays locations on a map using markers or bubbles.

Using our dataset, we can create a Symbol Map to visualize revenue generated by different cities.

Example:

- Mumbai
- Delhi
- Bangalore
- Chennai
- Hyderabad

Each city appears as a point on the map.

To create a Symbol Map:

1. Double-click the **City** field.
2. Tableau automatically generates a map.
3. Drag **Revenue** to **Color**.
4. Drag **Revenue** to **Size**.

Cities with higher revenue will appear as larger and darker bubbles.

This visualization helps us quickly identify which cities contribute the most revenue.

Filled Map

A Filled Map colors entire geographical regions based on a measure.

Using our dataset, we can visualize revenue generated by different states.

Example:



- Maharashtra
- Karnataka
- Gujarat
- Tamil Nadu
- Uttar Pradesh

Each state is shaded according to its revenue.

To create a Filled Map:

1. Double-click the **State** field.
2. Open the **Show Me** panel.
3. Select **Filled Map**.
4. Drag **Revenue** to **Color**.

States with higher revenue will appear darker, while states with lower revenue will appear lighter.

This visualization is useful for comparing business performance across different regions.

Symbol Map vs Filled Map

Symbol Map	Filled Map
Uses City data	Uses State data
Displays bubbles on a map	Colors entire states
Useful for location-level analysis	Useful for regional analysis
Shows exact city locations	Shows state-wise performance

In our `data.csv` dataset, Symbol Maps are best suited for analyzing revenue by city, while Filled Maps are useful for comparing revenue across states.



In the provided `data.csv` file, we have fields such as **City**, **State**, **Revenue**, and **Orders**. Tableau allows us to combine multiple worksheets into a single dashboard for analysis and reporting.

Revenue by City

This worksheet shows revenue generated by different cities.

To create it:

1. Drag **Revenue** to Columns.
2. Drag **City** to Rows.

This helps compare revenue across cities.

Revenue by State

This worksheet shows revenue generated by different states.

To create it:

1. Drag **Revenue** to Columns.
2. Drag **State** to Rows.

This helps compare business performance across states.

Creating a Dashboard

A Dashboard combines multiple worksheets into a single view.

To create a dashboard:

1. Click **New Dashboard**.
2. Drag worksheets onto the dashboard.
3. Arrange them as needed.

This provides a centralized view of important insights.

Containers

Containers help organize dashboard elements.

Horizontal Container

Places worksheets side by side.

Vertical Container



Places worksheets one below another.

Containers help create clean and structured layouts.

Orders by City Map

A map can be created using the **City** field.

To create it:

1. Double-click **City**.
2. Drag **Orders** onto the map.

This helps visualize order distribution across locations.

Scatter Plot

A Scatter Plot can be used to analyze the relationship between Revenue and Orders.

To create it:

1. Drag **Orders** to Columns.
2. Drag **Revenue** to Rows.

This helps identify patterns between the two measures.

Aggregate Measures

By default, Tableau aggregates measures such as:

- SUM(Revenue)
- SUM(Orders)

To view individual records:

1. Open **Analysis**.
2. Uncheck **Aggregate Measures**.

This is especially useful for Scatter Plots.

Final Dashboard

The dashboard combines:



- Revenue by City
- Revenue by State
- Orders by City Map
- Orders vs Revenue Scatter Plot

These visualizations provide a complete overview of business performance.

CodeWithHarry



After creating a dashboard in Tableau, it can be published to Tableau Public so that it can be viewed and shared online.

Creating a Tableau Public Account

Before publishing a dashboard, you need a Tableau Public account.

To create an account:

1. Visit Tableau Public.
2. Click **Sign Up**.
3. Enter your details and create an account.
4. Verify your email address.
5. Sign in to Tableau Public.

Publishing a Dashboard

To publish a dashboard:

1. Open your workbook in Tableau.

[Click File](#) → [Save to Tableau Public.](#)

2. Sign in to your Tableau Public account.
3. Tableau will upload and publish the workbook.

Once published, Tableau generates a public link for the dashboard.

Downloading Dashboards

Published dashboards can be downloaded in different formats.

Available formats include:

- PDF
- PowerPoint
- Tableau Workbook
- Image

This makes it easy to share dashboards with others.

Sharing and Embedding Dashboards

Tableau Public provides a shareable link for every dashboard.

You can also copy the **Embed Code** and add it to a website.

This allows the dashboard to be displayed directly on web pages.

Important Note About Tableau Public

Dashboards published on Tableau Public are publicly accessible.

This means:

- Anyone with the link can view the dashboard.
- Dashboards can be shared and embedded by others.
- Sensitive or confidential company data should never be published.

Always ensure that only non-confidential data is uploaded to Tableau Public.

Tableau Desktop vs Tableau Public

Tableau Desktop	Tableau Public
Used for creating dashboards	Used for sharing dashboards online
Files remain local	Dashboards are publicly available
Suitable for confidential data	Suitable for public data
Requires local storage	Provides shareable links

Tableau Public is an excellent platform for showcasing dashboards, sharing insights, and building a public portfolio of data visualization projects.



In this section, we learned how to connect data sources, create visualizations, build interactive dashboards, and publish our work using Tableau Public. These are some of the most important skills required for turning raw data into meaningful insights.

Like any tool, Tableau becomes easier with practice. Keep experimenting with different datasets and dashboard designs to strengthen your understanding.

As you continue to explore Tableau, remember to always consider the audience for your dashboards and ensure that the visualizations you create effectively communicate the intended message.

CodeWithHarry

